

Foundry 학습 루프 실습 · 통합 인쇄본

자신의 실습 환경을 만들고 Foundry 평가·최적화·비교와 승인된 삭제 또는 보존까지 10단계를 직접 진행합니다. 환경 설정 참고·실습 체크리스트·문제 해결·데이터 설명은 같은 실습 경로를 돕는 자료입니다. 문서 링크는 책 안으로, 파일 참조는 패키지 경로로 연결됩니다.

문서 기준 2026-10-05

수록 문서

- [1. 실습 가이드](#)
- [2. 부록 1 · 실습 체크리스트](#)
- [3. 부록 2 · 환경 설정 참고](#)
- [4. 부록 3 · 문제 해결](#)
- [5. 부록 4 · 데이터 설명](#)

실습 가이드

고객지원 AI를 평가하고 개선합니다 · v1

CONTOSO ATLAS CLOUD · 하나의 실습 경로

고객지원 AI에 질문 12개를 보내 채점하고, 지침을 고친 뒤 같은 질문으로 다시 비교합니다. Azure·Foundry 사용 경험은 필요하지 않습니다. 다만 PC에 도구를 설치하고, 사용 권한과 비용 승인을 받은 Azure 환경이 필요합니다.

Azure는 Microsoft의 클라우드 서비스입니다. **Microsoft Foundry**는 Azure에서 AI 모델과 Agent를 만들고 평가하는 개발 플랫폼입니다. **Agent(에이전트)**는 모델에 지침과 도구를 연결한 AI입니다. 여기서는 가상의 **Contoso Atlas Cloud 고객지원 Agent**가 정책 문서를 검색해 답하게 합니다.

실습 목표	정책에 근거해 답하는 고객지원 Agent를 만듭니다.
중요한 이유	그렇듯한 답변 하나가 아니라 12건의 답변과 채점 이유로 품질을 판단합니다.
진행 방식	같은 질문·모델·채점 기준을 유지하고 지침만 바꿉니다.
완료 산출물	변경 전후 비교, 최종 판단, 사용한 자원의 정리 기록을 남깁니다.

모든 실습 참여자가 처음부터 끝까지 직접 수행합니다. 본인 계정과 PC로 환경·Agent·데이터셋·평가를 준비하고 개선과 정리까지 진행합니다. 미리 만들어진 환경을 전제로 하지 않으며, 참고 문서는 필요한 설정을 확인할 때만 엽니다.

처음이면 01부터, 중단 후 재개라면 자신의 기록부터 확인합니다. 같은 실습의 config.json·manifest·receipt로 이미 완료한 작업을 확인하고 중복 생성·제출을 피합니다. 다른 사람의 설정·로그인 세션·소유권 기록을 복사하지 않습니다.

시간·비용: 처음부터 준비한다면 반나절 정도의 여유를 확보합니다. 권한·할당량 승인 대기과 서비스 실행 시간은 별도입니다. 모델 호출·Search·로그에는 비용이 발생할 수 있습니다. 브라우저를 닫아도 과금이 끝나지 않으므로 10의 정리까지 진행합니다. 중간에 실습을 중단하더라도 생성한 자원이 있다면 10의 보관·정리를 수행합니다.

01 계정·PC·권한 확인	02 Foundry 환경 생성	03 정책 연결·Agent 생성
04 데이터셋 등록	05 평가 기준 선택	06 Foundry Evaluation 실행
07 점수와 이유 읽기	08 Agent Optimizer로 지침 개선	09 재평가·v1/v2 비교
10 결과 보관·리소스 삭제		

여기서 **학습은 평가 결과를 읽고 지침을 개선하는 과정**이며 모델 가중치를 다시 학습시키는 과정은 아닙니다. 제공된 합성 정책·질문만 사용합니다. 고객 원본 자료를 업로드하거나 평가 질문의 정답을 지침에 넣지 않습니다.

실행 위치	여기에서 하는 일
Azure Portal (https://portal.azure.com)	계정·구독·권한, 실제 리소스 목록과 삭제 범위를 확인합니다.
내 PC의 터미널	실습 파일을 준비하고 제공된 명령을 실행하는 창입니다. 여는 방법은 01에서 안내합니다.
Microsoft Foundry (https://ai.azure.com)	프로젝트·Agent·정책 도구를 확인하고 평가·Optimizer·비교 결과를 읽습니다.

읽는 방법: 각 단계 상단의 **실행 순서** 링크로 이동하고, 해당 작업 뒤 **완료 기준**을 확인합니다. **터미널에서 실행** 상자만 내 PC에 입력하며, 포털 작업은 안내된 화면에서 수행합니다. Azure Portal과 Foundry는 별도 탭에 열어 둡니다. **그림의 계정·프로젝트 이름 대신 자신의 값을 사용합니다.**

구현 코드는 필요할 때만 펼칩니다. 실제 구현 코드 · 읽기 전용 패널은 명령 내부의 함수·설정 원문이며 직접 실행할 대상이 아닙니다. 패널을 열지 않아도 실습을 진행할 수 있습니다. **코드 ↔ 포털** 표의 **실행 위치** 링크로 실제 명령·화면을 찾고, 내부 동작이 궁금하면 소스를 읽습니다. 인쇄본에는 전체 소스가 펼쳐져 있습니다.

포털로 이미 생성된 항목을 다시 만들지 않습니다. 코드로 생성하는 단계는 포털에서 결과를 확인하고, 평가·Optimizer처럼 포털에서 제출하는 단계는 그 화면에서 한 번만 수행합니다. 포털 내부 구현을 추측한 SDK 코드로 대신하지 않습니다.

01-03은 환경 준비, 04-09는 평가·개선, 10은 정리입니다. **04-06은 같은 평가 생성 화면에서 이어집니다.** 중간에 새 평가를 만들지 않습니다. 읽음 표시는 개인 책갈피일 뿐 실제 작업 완료 표시는 아닙니다.

v1은 변경 전 Agent, v2는 지침을 바꾼 비교 후보입니다. 두 번호는 Agent 전체 구성의 버전입니다. 모델·도구·출력 형식은 유지합니다. 개선이 없어서 v1을 유지해도 올바른 실습 결과이며, v2 생성은 운영용 게시 승인이 아닙니다.

01. 계정·PC·권한을 확인합니다

이 단계에서 하는 일: 사용할 Azure 구독·권한과 로컬 실행 환경을 확인합니다.

중요한 이유: 브라우저와 CLI가 다른 계정이나 구독을 사용하면 권한 오류가 나거나 잘못된 환경에 자원을 만들 수 있습니다.

진행 방법·위치: Azure Portal에서 구독과 접근 권한을 확인한 뒤 터미널에서 도구·로그인·언어 설정을 대조합니다. 아직 Azure 리소스는 생성하지 않습니다.

실행 순서: [Azure Portal · 계정·권한](#) → [터미널 · 도구 준비](#) → [터미널 · 로그인](#) → [터미널 · 언어 설정](#).

화면을 열기 전에 알아둘 여섯 가지입니다. 외울 필요 없이, 아래 역할만 구분합니다.

테넌트 · Tenant / Directory

로그인할 조직의 사용자·권한을 관리하는 경계입니다. 같은 계정으로 여러 조직에 접근할 수 있어 조직도 확인합니다.

리소스 그룹 · Resource group

함께 관리할 Azure 자원을 묶는 그룹입니다. 실습 전용 그룹을 쓰면 종료 시 삭제 범위를 확인하기 쉽습니다.

모델 · Model

답변을 생성하는 AI입니다. 모델을 호출할 수 있게 준비하는 작업을 배포(Deploy), 그 배포에 붙인 이름을 배포 이름이라고 합니다.

구독 · Subscription

Azure 자원의 사용량과 요금이 묶이는 단위입니다. 실습 비용을 어느 구독에 청구할지 확인합니다.

프로젝트 · Project

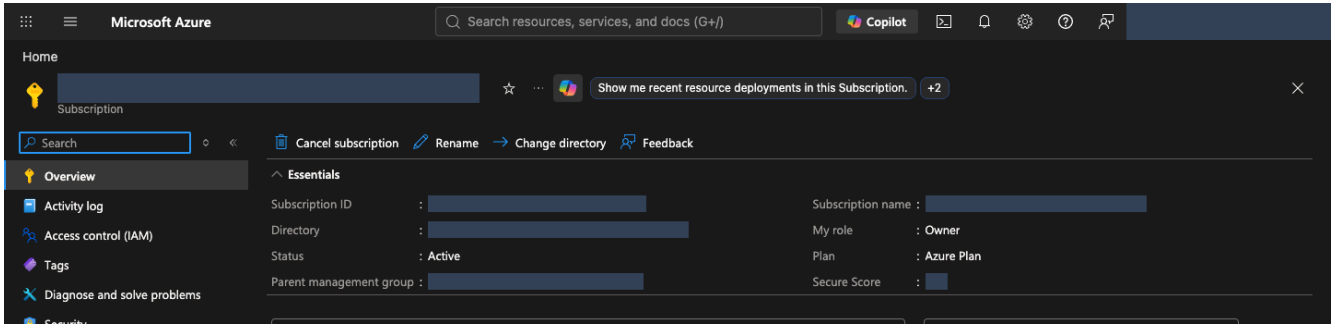
Foundry 안에서 Agent·데이터·평가 작업을 구분하는 공간입니다. 이후 실습은 같은 프로젝트에서 진행합니다.

권한 · Role / Scope

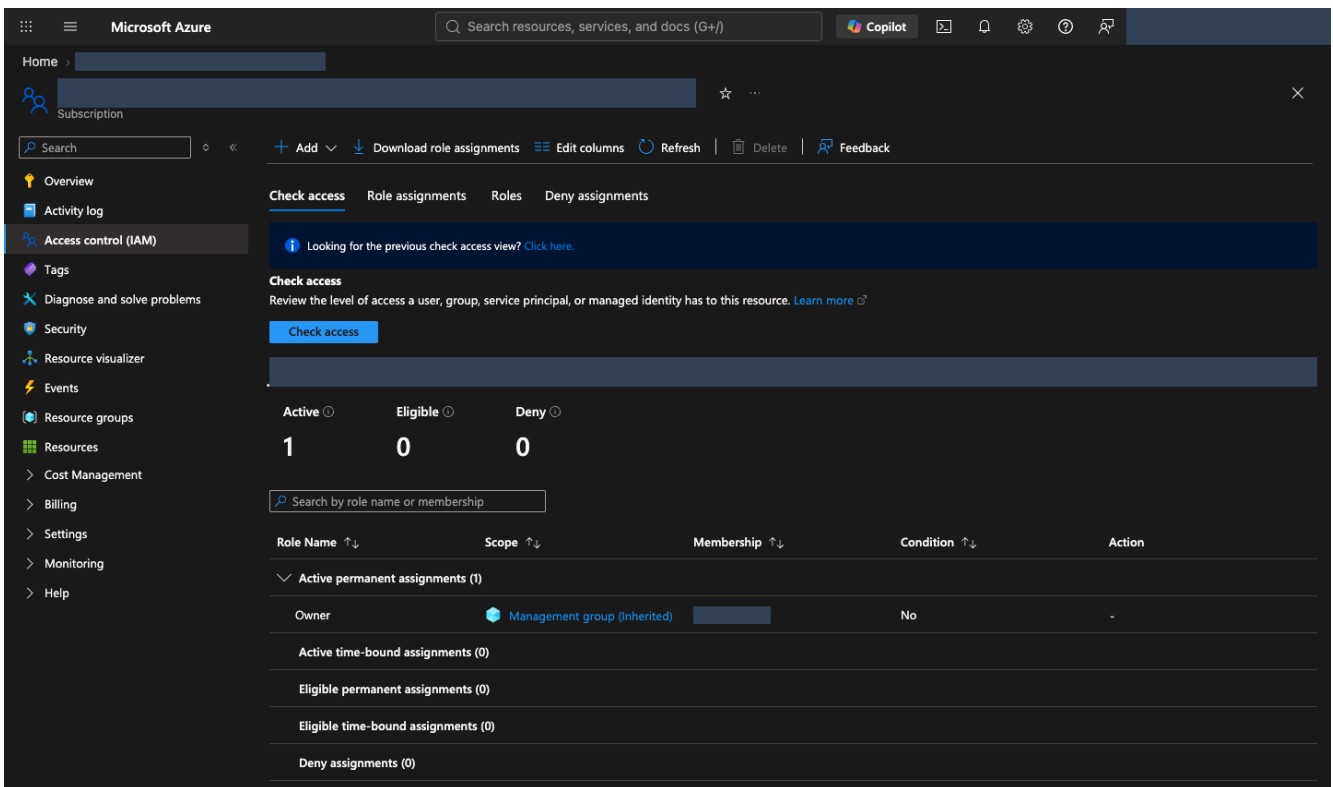
Role은 할 수 있는 일, Scope는 그 권한이 적용되는 범위입니다. 로그인 성공만으로 생성·평가·삭제 권한이 모두 생기지는 않습니다.

Azure 계정과 구독을 확인합니다

1. [Azure Portal \(https://portal.azure.com\)](https://portal.azure.com)에 **본인의 실습용 계정**으로 로그인합니다. 사용할 계정·구독이 없다면 [Azure 계정 안내 \(https://azure.microsoft.com/pricing/purchase-options/azure-account\)](https://azure.microsoft.com/pricing/purchase-options/azure-account)를 따라 준비합니다. 조직 구독은 본인이 사용할 범위의 접근·비용 승인을 먼저 확보합니다. 무료·체험 구독이라고 필요한 모델의 할당량까지 보장되지는 않습니다.
2. 상단 검색창에 **Subscriptions**를 입력하고 사용할 구독을 엽니다. 포털 상태가 **Active**인지 확인하고 **Subscription ID**와 **Directory/Tenant ID**를 기록합니다. 같은 정상 구독은 CLI에서 **Enabled**로 표시합니다. 구독이 보이지 않으면 우측 상단 계정의 디렉터리·구독 필터를 확인합니다.
3. **Access control (IAM, 권한 관리) → Check access**에서 본인의 역할과 적용 범위를 확인합니다. 이전 UI에서는 **View my access**로 표시할 수 있습니다. 이 실습은 본인이 리소스를 생성하고 리소스 범위 역할을 할당하므로 **생성 권한과 역할 할당 권한이 모두 필요합니다**. Contributor만으로는 역할 할당이 가능하지 않습니다. [작업별 필요 권한·공급자 등록](#)을 확인합니다.
4. 비용 한도, 사용 종료 시각, 전용 리소스 그룹의 삭제 또는 보존 범위를 정합니다. 권한·구독·비용 승인이 부족하면 조직 절차에 따라 본인에게 허용된 실행 범위를 확보한 뒤 계속합니다. 다른 사람에게 이후 실습을 대신 실행하게 하거나 구독 전체 Owner·보안 정책 해제로 우회하지 않습니다.



구독과 상태를 확인합니다. Essentials에서 Subscription ID, Directory, Status, My role을 찾습니다. 포털의 **Active**와 CLI의 **Enabled**는 같은 정상 사용 상태를 나타냅니다. 원본 크기로 보기 (파일: web/assets/portal/shared/21-subscription-overview.png)



역할 이름과 Scope를 함께 읽습니다. Access control (IAM) → Check access에서 자신의 활성 할당을 확인합니다. 역할 이름만 보지 않고 적용 범위를 대조하며 필요한 범위보다 큰 권한을 추가하지 않습니다. 원본 크기로 보기 (파일: web/assets/portal/shared/22-check-access.png)

도구와 실습 파일을 준비합니다

[Python \(https://www.python.org/downloads/\)](https://www.python.org/downloads/) **3.11–3.14**, [Git \(https://git-scm.com/downloads\)](https://git-scm.com/downloads), [Azure CLI](https://learn.microsoft.com/cli/azure/install-azure-cli)

(<https://learn.microsoft.com/cli/azure/install-azure-cli>)를 설치합니다. Python은 제공된 프로그램 실행용, Git은 실습 파일 내려받기용, Azure CLI는 az 명령으로 Azure에 로그인·조회하는 도구입니다. 회사 PC에서 설치가 제한되면 담당자에게 요청합니다.

이미 설치했다면 **설치 확인부터 진행하고, 정상인 도구는 다시 설치하지 않습니다**. Windows·macOS의 아래 예시는 지원 범위를 벗어난 최신 Python이 선택되지 않도록 **3.13**을 지정합니다. 이미 다른 지원 버전을 사용한다면 `py -3.13·python3.13`을 해당 버전의 명령으로 바꾸고, 버전 확인과 가상환경 생성에 같은 Python을 사용합니다. 3.10 이하·3.15 이상은 이 실습의 지원 범위가 아닙니다.

먼저 **내 PC의 터미널**을 엽니다. Windows는 시작 메뉴에서 **PowerShell**, macOS는 Spotlight에서 **Terminal(터미널)**, Linux는 앱 메뉴에서 터미널을 검색합니다. Azure Portal의 Cloud Shell이나 >>>가 보이는 Python 입력창은 이 가이드의 실행 위치가 아닙니다.

명령 읽는 법: 한 명령씩 실행하고 끝난 뒤 다음 명령으로 이동합니다. 긴 명령이 화면·PDF에서 여러 줄로 보이더라도 명령 중간에 줄바꿈을 넣지 않습니다.

복사 버튼은 **상자 전체**를 복사합니다. 여러 명령이 들어 있다면 편집기에 먼저 붙여넣고 한 명령씩 실행합니다. `YOUR_...`는 자신의 값으로 바꾸고 바깥 큰따옴표는 유지합니다.

공통 터미널은 모든 운영체제에서 사용합니다. macOS/Linux·PowerShell 상자는 바로 위 설명을 읽고 **자신의 운영체제에 해당하는 것만** 실행합니다. **오류가 나면 다음 명령으로 넘어가지 않고** 해당 단계의 문제 해결을 확인합니다.

[Windows 설치](#) · [macOS 설치](#) · [Linux 설치](#) 중 하나를 마친 뒤 [설치 확인](#)으로 이동합니다.

Windows · PowerShell에서 설치합니다

PowerShell에서 `winget --version`을 실행합니다. 버전이 나오면 Windows 패키지 관리 도구인 **WinGet**으로 아래 명령을 한 줄씩 실행합니다. 설치 동의나 설치 권한 요청은 조직의 승인 범위에서 직접 확인합니다.

터미널에서 실행 · Windows PowerShell

```
winget install --exact --id Python.Python.3.13 --source winget
winget install --exact --id Git.Git --source winget
winget install --exact --id Microsoft.AzureCLI --source winget
```

WinGet이 없거나 사용할 수 없다면 공식 설치 파일을 사용합니다. 두 방법을 모두 실행할 필요는 없습니다.

도구	다운로드와 설치 화면에서 확인할 항목
Python	Windows 다운로드 (https://www.python.org/downloads/windows/)에서 Python 3.13의 최신 패치 버전 과 PC에 맞는 installer를 선택합니다. embeddable package가 아닌 일반 설치 파일에서 Add python.exe to PATH 를 선택하고 pip·Python Launcher 를 포함하여 설치합니다.
Git	Git for Windows (https://git-scm.com/install/windows)에서 PC에 맞는 설치 파일을 받습니다. PATH 선택 화면의 Git from the command line and also from 3rd-party software 를 유지하여 PowerShell에서도 사용할 수 있게 합니다.
Azure CLI	Windows 공식 설치 안내 (https://learn.microsoft.com/cli/azure/install-azure-cli-windows?pivot=msi)의 Microsoft Installer (MSI) 를 사용합니다. 일반적인 x64 PC는 64-bit MSI를 선택합니다.

설치 후 [새 터미널에서 세 도구 확인](#)으로 이동합니다. `py`가 인식되지 않으면 Python 설치의 Launcher 옵션을 확인합니다.

macOS · Homebrew로 설치합니다

터미널에서 `brew --version`을 실행합니다. 명령이 없으면 [Homebrew 공식 설치 안내 \(https://brew.sh/\)](https://brew.sh/)를 따라 설치하고, 마지막에 표시되는 **Next steps**의 PATH 설정까지 마친 뒤 새 터미널에서 다시 확인합니다. 회사 PC의 설치·Command Line Tools 승인 절차를 우회하지 않습니다.

터미널에서 실행 · macOS/Linux

```
brew update
brew install python@3.13 git azure-cli
```

Python은 [Homebrew의 버전 지정 패키지 \(https://formulae.brew.sh/formula/python@3.13\)](https://formulae.brew.sh/formula/python@3.13), Azure CLI는 [Microsoft의 macOS 설치 안내 \(https://learn.microsoft.com/cli/azure/install-azure-cli-macos\)](https://learn.microsoft.com/cli/azure/install-azure-cli-macos)를 따릅니다. 이 경로에서는 python3.13을 사용합니다. 기존 python3는 macOS 기본 Python이나 다른 버전을 가리킬 수 있으므로 시스템 Python을 덮어쓰거나 강제로 연결하지 않습니다.

Linux · Ubuntu 24.04 LTS 예시입니다

다음은 기본 Python 3.12를 제공하는 **Ubuntu 24.04 LTS** 기준입니다. 다른 배포판은 해당 배포판의 Python·[Git 설치 안내 \(https://git-scm.com/install/linux\)](https://git-scm.com/install/linux)와 [Azure CLI 공식 설치 안내 \(https://learn.microsoft.com/cli/azure/install-azure-cli-linux\)](https://learn.microsoft.com/cli/azure/install-azure-cli-linux)를 따릅니다. Python 버전이 지원 범위 밖이면 시스템 Python을 교체하지 말고 담당자에게 지원되는 실행 환경을 요청합니다.

터미널에서 실행 · macOS/Linux

```
sudo apt update
sudo apt install python3 python3-venv python3-pip git curl
curl -fsSL 'https://azurecliproduct.blob.core.windows.net/$root/deb_install.sh' -o install-azure-cli.sh
```

마지막 명령은 [Microsoft의 Ubuntu/Debian용 설치 스크립트 \(https://learn.microsoft.com/cli/azure/install-azure-cli-linux?pivot=apt\)](https://learn.microsoft.com/cli/azure/install-azure-cli-linux?pivot=apt)를 내려받기만 합니다. `install-azure-cli.sh`를 편집기로 열어 내용을 확인하고 필요한 시스템 설치 권한과 승인이 있는 경우에만 다음을 실행합니다. `sudo` 권한이 없으면 승인된 설치 경로를 확보한 뒤 진행합니다.

터미널에서 실행 · macOS/Linux

```
sudo bash install-azure-cli.sh
```

새 터미널에서 설치를 확인합니다

설치가 끝나면 열려 있던 터미널 창을 모두 닫고 다시 엽니다. PATH는 명령을 찾는 경로 목록이며, 기존 창에는 새 설치 경로가 반영되지 않을 수 있습니다. VS Code의 터미널을 사용한다면 VS Code도 종료한 뒤 다시 엽니다.

Windows PowerShell:

터미널에서 실행 · Windows PowerShell

```
py -3.13 --version
git --version
az version
```

macOS · 위 Homebrew 설치 경로:

터미널에서 실행 · macOS/Linux

```
python3.13 --version
git --version
az version
```

Linux · 위 Ubuntu 설치 경로:

터미널에서 실행 · macOS/Linux

```
python3 --version
git --version
az version
```

확인 항목	정상 출력과 진행 기준
실습용 Python	Python 3.13.x 같은 버전이 나옵니다. x는 실제 패치 번호이며 3.11·3.12·3.13·3.14 중 하나여야 합니다.
Git	git version 2.x...처럼 설치된 버전이 나옵니다.
Azure CLI	JSON에 "azure-cli": "2.x..."처럼 버전이 나옵니다. 이 확인에는 Azure 로그인 필요하지 않습니다.

az version에 표시되는 Python은 Azure CLI 자체의 실행 환경이며, 실습용 Python 설치 확인을 대신하지 않습니다. 세 명령이 모두 성공한 후에만 파일 다운로드·가상환경 생성으로 진행합니다. command not found, not recognized, Python 대신 Microsoft Store가 열리는 경우는 [설치·PATH 문제 해결](#)을 확인합니다.

실습 파일을 내려받습니다

터미널에서 실행 · 공통 터미널

```
git clone https://github.com/junwoojeong100/foundry-evaluation-labs-v1.git
cd foundry-evaluation-labs-v1
```

이미 내려받았다면 clone을 반복하지 않고 해당 폴더로 이동합니다. GitHub의 **Code → Download ZIP**으로 받은 경우 먼저 압축을 풀고 pyproject.toml과 requirements.lock이 있는 폴더를 엽니다. HTML 파일 한 개만 내려받으면 코드·데이터·그림이 누락됩니다.

확인한 Python으로 가상환경을 만듭니다

이후 명령은 `pyproject.toml`과 `requirements.lock`이 있는 **실습 폴더**에서 실행합니다. `.venv`는 이 실습만의 Python 패키지 공간입니다. 앞에서 버전을 확인한 **같은 Python**으로 만들며, 아래 세 경로 중 자신의 운영체제에 맞는 하나만 실행합니다.

macOS · 위 Homebrew 설치 경로:

터미널에서 실행 · macOS/Linux

```
python3.13 -m venv .venv
source .venv/bin/activate
```

Linux · 위 Ubuntu 설치 경로:

터미널에서 실행 · macOS/Linux

```
python3 -m venv .venv
source .venv/bin/activate
```

Windows PowerShell에서는 다음을 실행합니다.

터미널에서 실행 · Windows PowerShell

```
py -3.13 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

PowerShell 실행 정책 때문에 활성화가 차단되면 조직 정책을 변경하지 않습니다. 이후 명령의 `python`을 `.\.venv\Scripts\python.exe`로 바꾸어 실행합니다. 기존 `.venv`가 있다면 다른 작업의 환경인지 확인하고 임의로 덮어쓰지 않습니다.

가상환경을 준비한 뒤 다음 공통 명령을 실행합니다. 첫 두 명령에서 Python 3.11–3.14와 `.venv` 안의 pip 경로를 확인한 뒤 패키지를 설치합니다.

터미널에서 실행 · 공통 터미널

```
python --version
python -m pip --version
python -m pip install -r requirements.lock
python -m lab --help
python scripts/build_datasets.py --language ko --check
```

명령 목록과 데이터 검사 성공을 확인합니다. `ModuleNotFoundError`가 나오면 `python -m pip --version`으로 `.venv`의 Python을 사용하는지 확인합니다. 시스템 Python에 임의로 패키지를 설치하지 않습니다.

CLI 로그인과 언어를 고정합니다

먼저 아래 YOUR_SUBSCRIPTION_ID를 앞에서 기록한 구독 ID로 교체합니다. az login이 여는 브라우저에서 **같은 계정**으로 로그인한 뒤 터미널로 돌아옵니다.

터미널에서 실행 · 공통 터미널

```
az login
az account list --query "[].{name:name,id:id,state:state}" -o table
az account set --subscription "YOUR_SUBSCRIPTION_ID"
az account show --query "{user:user.name,tenant:tenantId,subscription:id,state:state}" -o json
```

마지막 출력의 state가 Enabled인지 확인하고 user, tenant, subscription을 포털과 대조하여 기록합니다. 브라우저와 CLI의 로그인은 별개입니다. 다른 디렉터리가 선택됐다면 승인된 디렉터리로 az login --tenant "YOUR_TENANT_ID"를 실행합니다.

언어와 실행 기록 폴더를 지정합니다

이 한국어 실습의 환경 이름은 lab-ko로 사용합니다. 같은 이름의 로컬 계획이 이미 있으면 새로 만들지 않고 원래 기록을 재개합니다. 별도 수업이면 lab-ko-02처럼 새 이름을 정하고 이후 모든 경로와 prefix도 함께 바꿉니다.

macOS/Linux에서는 다음 환경 변수를 지정합니다.

터미널에서 실행 · macOS/Linux

```
export LAB_LANGUAGE=ko
export LAB_ARTIFACTS_DIR="$PWD/.lab/lab-ko/artifacts"
```

Windows PowerShell에서는 다음 환경 변수를 지정합니다.

터미널에서 실행 · Windows PowerShell

```
$env:LAB_LANGUAGE = "ko"
$env:LAB_ARTIFACTS_DIR = Join-Path (Get-Location).Path ".lab/lab-ko/artifacts"
```

새 터미널에서는 실습 폴더로 먼저 이동한 뒤 [가상환경 활성화](#)와 이 두 변수를 다시 지정합니다. LAB_LANGUAGE를 바꾸면서 기존 실행 폴더를 재사용하지 않습니다.

같은 실습을 재개하는 경우: 가상환경·언어 변수·계정을 다시 확인하고 자신의 원래 실행 기록을 사용합니다. 환경 생성 상태는 [02의 조화·검사](#)에서 확인합니다. 01에서는 Azure 환경을 생성하거나 SDK를 호출하지 않습니다.

코드 ↔ 포털 · 로그인과 로컬 설정을 확인합니다

실행 위치: 위의 [터미널 로그인 명령](#)과 [언어·기록 폴더 설정](#)을 실행합니다. 이 단계에서는 명령 출력과 포털 값을 직접 대조합니다.

실행한 명령·설정값	지금 확인하는 내용	포털 조작·확인 위치
az login, az account set, az account show	로그인 계정·테넌트·구독을 기록하고 state: Enabled를 확인합니다.	Azure Portal 우측 상단 계정의 Directory와 Subscriptions → 사용할 구독 → Overview 의 ID·상태를 직접 대조합니다.
LAB_LANGUAGE, LAB_ARTIFACTS_DIR	현재 터미널의 데이터 언어와 실행 기록 경로를 지정합니다.	포털에 없는 로컬 설정입니다. 한국어 실습은 ko와 .lab/lab-ko/artifacts를 사용합니다.

완료 기준: Python 3.11–3.14·Git·Azure CLI의 버전을 확인했고, 같은 사용자·테넌트·구독을 사용하며, 가상환경의 Python 명령과 한국어 데이터 검사가 성공합니다. 문제가 있으면 [환경 오류 해결](#)을 확인합니다.

02. Foundry 환경을 생성합니다

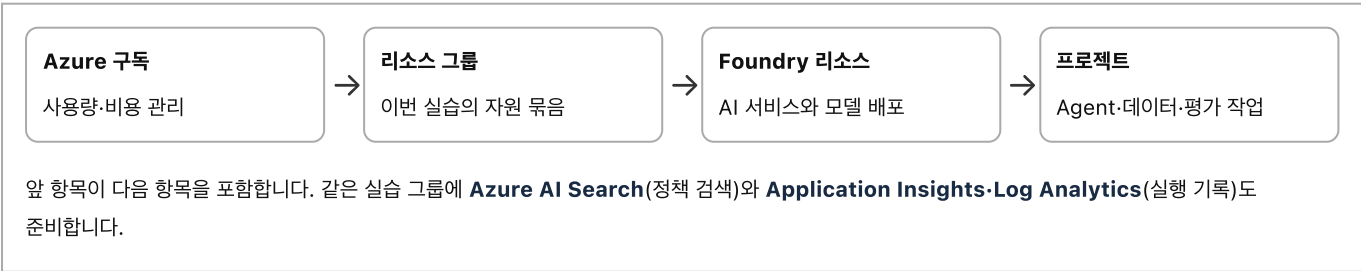
이 단계에서 하는 일: 전용 리소스 그룹 안에 Foundry 프로젝트·모델·Search·모니터링 기반을 준비합니다.

중요한 이유: 리소스 그룹은 비용과 정리 범위를 묶고, 프로젝트는 Agent·평가를 구분합니다. 빈 프로젝트 하나만으로 정책 검색까지 준비되지는 않습니다.

진행 방법·위치: 본인의 터미널에서 계획·승인·생성을 순서대로 수행한 뒤 두 포털에서 결과를 대조합니다. 재가할 때는 같은 계획과 실제 상태를 확인합니다.

실행 순서: [터미널 · 계획](#) → [TPM 요구량 확인](#) → [승인·준비 검사](#) → [터미널 · 생성](#) → [포털·런타임 확인](#).

처음에는 아래 계획부터 만듭니다. 자신의 같은 실습 계획이 이미 있다면 plan을 반복하지 않고 원래 config.json으로 bootstrap status를 조회합니다. 생성이 끝난 경우에도 이 단계의 포털·런타임 검사를 확인한 뒤 03으로 진행합니다.



Endpoint(엔드포인트)는 프로그램이 서비스에 연결할 때 쓰는 주소입니다. 이 실습은 프로젝트의 **Project endpoint**를 사용합니다. 옆에 있는 Azure OpenAI endpoint와 서로 바꾸어 사용하지 않습니다.

로컬 생성 계획을 만듭니다

01에서 기록한 값을 아래 표대로 교체합니다. **ID는 이름이 아니라 자원을 식별하는 값**입니다. 구독의 표시 이름을 ID 자리에 넣지 않습니다.

교체할 부분	가져올 값
YOUR_SUBSCRIPTION_ID	az account show의 subscription
YOUR_TENANT_ID	같은 출력의 tenant
YOUR_SIGN_IN_NAME	같은 출력의 user

이 명령은 내 PC의 `.lab/lab-ko/`에 계획·미승인 승인서 예시만 만들고 Azure를 호출하지 않습니다.

터미널에서 실행 · 공통 터미널

```
python -m lab bootstrap plan --subscription "YOUR_SUBSCRIPTION_ID" --tenant "YOUR_TENANT_ID" --expected-user "YOUR_SIGN_IN_NAME" --environment lab-ko --location northcentralus
```

`plan_status: CREATED_LOCAL_ONLY`, `mutations_performed: false`와 `config_path`를 확인합니다.
`BLOCKED_AWAITING_APPROVAL`은 아직 비용 승인이 없다는 뜻이며 생성 실패나 Azure 생성 완료가 아닙니다.

생성 파일	확인할 내용
<code>.lab/lab-ko/config.json</code>	구독·테넌트·사용자, 자동 생성 리소스 이름, 모델·버전·용량입니다. 직접 수정하지 않습니다.
<code>.lab/lab-ko/approval.example.json</code>	아직 승인되지 않은 비용·변경 범위입니다.
<code>.lab/lab-ko/manifest.json</code>	생성·소유권·중단 상태를 추적하는 기록입니다. 보존합니다.
<code>.lab/lab-ko/.env</code>	아직 없습니다. 실제 생성이 완료된 뒤 만들어지는 런타임 설정입니다.

파일은 텍스트 편집기의 **파일 열기**로 확인합니다. `.lab`는 실습 기록 폴더이며 운영체제에 따라 숨김 폴더로 보일 수 있습니다.
JSON은 설정을 이름: 값으로 저장하는 형식입니다. Word 문서로 변환하거나 파일 확장자를 바꾸지 않습니다.

이제 같은 편집기에서 `.lab/lab-ko/notes.md`를 만들고 [실습 기록표](#)에 01의 계정 값과 계획 경로를 적습니다. `.md`는 일반 텍스트 메모 파일입니다. 이후 단계에서 실제 배포 이름·Agent·데이터·run ID·최종 판단을 이 파일에 이어서 기록합니다.

구현 참고: `plan()`은 이름·모델·해시를 정하고 로컬 파일만 씁니다. **Azure 생성 호출은 없습니다.** `plan` 뒤 포털에 자원이 나타나지 않는 것은 정상입니다.

실제 구현 코드 · 읽기 전용 · lab/bootstrap.py:plan

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 657-726 (파일: lab/bootstrap.py#L657-L726) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L657-L726\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L657-L726)

```
def plan(
    *, subscription_id: str, tenant_id: str, expected_user: str,
    environment: str | None = None, root: Path | str = Path(".lab"),
    location: str = REGION, retention_days: int = 30, models: list[dict] | None = None,
    resource_group: str | None = None, agent_sku: str | None = None,
) -> dict:
    """Create a private local plan only; never query Azure or overwrite .env/artifacts."""
    instance = uuid4()
    suffix = instance.hex[:8]
    date = _now().strftime("%Y%m%d")
    environment = environment or f"lab-{date}-{suffix}"
    root = Path(root)
    if root.is_symlink() or any(parent.is_symlink() for parent in root.parents):
        raise BootstrapError("Environment root must not traverse symlinks.")
    directory = root.resolve() / environment
    selected = deepcopy(list(DEFAULT_MODELS) if models is None else models)
    if agent_sku is not None and agent_sku not in {"Standard", "GlobalStandard"}:
        raise BootstrapError("Explicit agent SKU must be Standard or GlobalStandard.")
    for model in selected:
        if not isinstance(model, dict) or not isinstance(model.get("roles"), list) or not
        model["roles"]:
            raise BootstrapError("Each model requires a nonempty roles array.")
        model.setdefault("sku", "GlobalStandard")
        if agent_sku is not None and "agent" in model["roles"]:
            model["sku"] = agent_sku
        model.setdefault("deployment", f"lab-{model.get('roles', ['model'])[0]}-{suffix}")
    config = {
        "schema_version": SCHEMA_VERSION, "instance_id": str(instance), "created_at": _stamp(),
        "environment": environment, "environment_dir": str(directory),
        "subscription_id": subscription_id, "tenant_id": tenant_id, "expected_user": expected_user,
        "location": location, "retention_days": retention_days, "models": selected,
        "deployment_name": f"lab-bootstrap-{suffix}",
        "template_sha256": _file_digest(TEMPLATE),
        "names": {
            "resource_group": resource_group or f"rg-foundry-eval-v11-{date}-{suffix}",
            "account": f"ai-fev11-{date}-{suffix}", "project": f"proj-fev11-{suffix}",
            "search": f"srch-fev11-{date}-{suffix}", "workspace": f"log-fev11-{date}-{suffix}",
            "insights": f"appi-fev11-{date}-{suffix}",
        },
    },
}
config["scope_sha256"] = _scope(config)
_validate(config)
try:
    root.mkdir(parents=True, exist_ok=True, mode=0o700)
    directory.mkdir(mode=0o700)
except FileExistsError as exc:
    raise BootstrapError("Environment already exists; use its config to inspect/resume, never
    overwrite it.") from exc
```

```

_write(directory / ".gitignore", "*\n", text=True)
(directory / "artifacts").mkdir(mode=0o700)
(directory / "evidence").mkdir(mode=0o700)
manifest = {
    "schema_version": SCHEMA_VERSION, "scope_sha256": config["scope_sha256"],
    "phase": "planned", "created_at": _stamp(), "resources": _resources(config),
    "attempts": [], "operator_principal_id": None, "env_sha256": None,
}
_write(directory / "config.json", config)
_write(directory / "template.json", TEMPLATE.read_text(encoding="utf-8"), text=True)
_write(directory / "approval.example.json", approval_template(config))
_write(directory / "group-authorization.example.json", group_authorization_template(config))
_write(directory / "manifest.json", manifest)
_write(directory / "cost-ledger.json", cost_ledger(manifest))
return {
    "status": AWAITING_APPROVAL, "plan_status": "CREATED_LOCAL_ONLY",
    "approval_status": "MISSING", "live_status": "NOT_RUN", "mutations_performed": False,
    "scope_sha256": config["scope_sha256"],
    "config_path": str(directory / "config.json"),
    "approval_example_path": str(directory / "approval.example.json"),
    "env_path": str(directory / ".env"), "artifacts_dir": str(directory / "artifacts"),
    "models": selected,
    "warning": "No Azure calls made. Candidate versions/SKUs/quota require fresh preflight and
explicit approval.",
}

```

모델 역할은 **답변 생성(Agent)**, **채점(Judge)**, **지침 제안·검색 계획(Optimizer/planner)**, **검색용 숫자 변환(embedding)**으로 나뉩니다. 여기서 모델 자체를 직접 개발하지는 않습니다.

이 저장소의 기본 생성 지역은 **North Central US (northcentralus)**입니다. 기본 배포는 Agent gpt-6-sol, Judge gpt-6-luna, Optimizer/검색 planner gpt-5.5, embedding text-embedding-3-small입니다. 정확한 버전·SKU(배포 유형)·요청 용량은 [모델 표](#)와 계획 파일에서 확인합니다. 다른 지역·모델로 조용히 대체하지 않습니다.

모델별 최소 권장 TPM을 확보합니다

TPM(Tokens Per Minute)은 배포별 분당 토큰 처리 한도이며, 모델의 최대 입력 길이나 실제 처리 속도와는 다릅니다. 아래는 **한 환경에서 평가·Optimizer를 한 번에 하나씩 실행하는 12문항 실습**의 시작 기준입니다. 모든 요청에서 입증된 절대 최소치나 429 오류가 없다는 보장이 아니라, 평가·검색 호출을 고려한 여유 포함 최소 권장값입니다.

역할·모델	실습 시작 최소 권장 TPM	새 기본 계획의 ARM 용량
Agent · gpt-6-sol	100,000	100
Judge · gpt-6-luna	100,000	100
Optimizer / 검색 planner · gpt-5.5	100,000	100, 두 역할이 같은 배포를 공유
Embedding · text-embedding-3-small	10,000	10

새 bootstrap plan은 이 용량으로 계획을 만듭니다. **ARM 용량은 모델·SKU별 단위**이므로 다른 모델에 무조건 1,000을 곱하지 않습니다. 생성 후 실제 배포의 TPM을 아래 runtime preflight로 확인합니다. 구독에 남은 할당량이 있어도 해당 배포에 충분히 할당되지 않았다면 준비 완료가 아닙니다.

기존 계획·배포는 자동으로 상향되지 않습니다. 자신의 실제 배포와 승인 범위는 [TPM 설정 참고](#)로 확인합니다. config.json·승인서·manifest를 직접 고쳐 검사를 우회하지 않습니다.

여러 사용자가 같은 배포를 공유하거나 작업을 겹치면 추가 용량이 필요합니다. Azure는 실제 청구 토큰뿐 아니라 입력·최대 출력 설정으로 추정된 토큰을 제한에 반영하며, 별도의 **RPM(분당 요청 수)** 짧은 구간의 요청 집중도도 제한합니다. [공식 TPM·RPM 설명 \(https://learn.microsoft.com/azure/foundry/openai/how-to/quota#understanding-rate-limits\)](#)을 참고합니다.

준비 상태와 비용 승인을 확인합니다

터미널에서 실행 · 공통 터미널

```
python -m lab bootstrap preflight --config .lab/lab-ko/config.json
```

Preflight는 실행 전 준비 상태 검사입니다. readiness_status: READY인지 확인합니다. 할당량(quota)은 구독에 허용된 사용 한도, 용량(capacity)은 해당 지역의 실제 제공 여력입니다. 막히면 reason을 읽고 [생성 오류 대응](#)으로 이동합니다. 아직 승인서가 없으므로 READY여도 전체 상태는 승인 대기일 수 있습니다. 지역이나 환경 이름을 바꾸어 반복 생성하지 않습니다.

원문에서 cognitiveservices model list, usage list, ARM의 modelCapacities를 조회하는 부분을 읽습니다. 이 검사는 모델을 호출하거나 배포하는 코드가 아닙니다.

실제 구현 코드 · 읽기 전용 · lab/bootstrap.py:_preflight

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 1307-1339 (파일: lab/bootstrap.py#L1307-L1339) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1307-L1339\)](#)

```
def _preflight(config: dict, manifest: dict, run: Run) -> tuple[dict, dict, dict]:
    identity = _identity(config, manifest, run)
    prerequisites = _prerequisites(config, run)
    args = ["--location", REGION, "--subscription", config["subscription_id"]]
    catalog = run(["cognitiveservices", "model", "list", *args])
    usage = run(["cognitiveservices", "usage", "list", *args])
    if not isinstance(catalog, list) or not isinstance(usage, list):
        raise BootstrapError("Regional model catalog/quota response is invalid.")
    capacities = {}
    for model in config["models"]:
        query = urlencode({
            "api-version": "2024-10-01", "modelFormat": "OpenAI",
            "modelName": model["name"], "modelVersion": model["version"],
        })
        capacities[model["deployment"]] = run([
            "rest", "--method", "GET", "--url",
            f"
```

```
{ARM}/subscriptions/{config['subscription_id']}/providers/Microsoft.CognitiveServices/locations/{REGION}/modelCapacities?{query}",
    })
    observed = _inspect(config, manifest, run)
    checks = _model_checks(config, catalog, usage, capacities, observed)
    report = {
        "status": "READY", "scope_sha256": config["scope_sha256"], "checked_at": _stamp(),
        "region": REGION, "identity_verified": True, **prerequisites, "models": checks,
        "owned_resource_count": len(observed), "mutations_performed": False,
        "not_verified": [
            "No paid inference, training, runtime data-plane access, or RBAC propagation tested.",
            "MCP principal is not inferred from matching subscription/tenant.",
            "Available quota is unallocated capacity, not free pricing or a spend cap.",
            "Fine-tuning region, training quota, deployment availability, and cost require separate
preflight/approval.",
        ],
    }
    raw = {"identity": identity, "catalog": catalog, "usage": usage, "capacities": capacities,
"observed": observed}
    return report, raw, observed
```

본인이 사용할 비용·변경 범위의 승인을 확인한 뒤 `.lab/lab-ko/approval.example.json`을 편집기에서 열고 **다른 이름으로 저장**하여 `.lab/lab-ko/approval.json`을 만듭니다. 본인 소유 구독에서 지출을 결정할 권한이 있다면 한도를 직접 정하고 기록합니다. 조직 구독에서는 실제 승인 절차를 따릅니다. 원본 `scope_sha256`, `models`, `retention_days`는 유지하고 [승인서 작성표의 모든 필드](#)를 채웁니다.

`approved_by`에는 계획의 `expected_user`와 같은 **본인 로그인 이름**을 적습니다. 이 값은 실행 신원을 연결하는 필드이지 조직 승인자의 전자서명이 아닙니다. 실제 승인 근거·현재 유효한 시작/만료 시각·통화/예산·대기/보관 한도·생성/RBAC/Global 처리 동의를 확인합니다. `approved: true`만 바꾸면 완료되지 않으며, 예산 숫자는 Azure의 자동 과금 차단 장치가 아닙니다.

터미널에서 실행 · 공통 터미널

```
python -m lab bootstrap preflight --config .lab/lab-ko/config.json --approval .lab/lab-ko/approval.json
```

`readiness_status: READY`와 `status: READY_FOR_APPROVED_APPLY`를 모두 확인한 뒤에만 다음 생성 명령을 실행합니다.

승인한 환경을 생성하고 포털에서 확인합니다

터미널에서 실행 · 공통 터미널

```
python -m lab bootstrap apply --config .lab/lab-ko/config.json --approval .lab/lab-ko/approval.json
python -m lab bootstrap status --config .lab/lab-ko/config.json --approval .lab/lab-ko/approval.json
```

첫 명령은 리소스·모델·연결·리소스 범위 역할을 실제로 생성합니다. 시간이 걸릴 수 있으므로 다른 창에서 같은 `apply`를 실행하지 않습니다. `apply`의 **APPLIED**, `.lab/lab-ko/.env` 생성, `status`의 `phase: succeeded`와 대상 리소스의 존재를 확인합니다. 대기 초과이면 `status`부터 확인하며 [재개 절차](#)를 따릅니다.

여기서 생성된 `.env`가 이후 명령의 연결 설정입니다. 사용자·테넌트·구독과 실제 프로젝트·모델 배포 값이 들어 있습니다. 설정 파일이므로 `source`나 PowerShell 스크립트로 실행하지 않고, 명령의 `--config`로 지정합니다. 아직 없다면 `.env.example`을 복사해 생성 완료로 간주하지 않습니다.

코드 ↔ 포털 · 생성한 자원과 배포를 확인합니다

실행 위치: 승인 후 위의 `bootstrap apply status` 명령을 실행하고 [아래의 포털·런타임 검사](#)로 확인합니다. 포털에서는 생성 결과를 조회하며 같은 자원을 다시 만들지 않습니다.

코드·입력값	실제 동작	포털 조작·확인 위치
<code>config["names"]</code> , ARM PUT의 <code>location-tags</code>	이름 충돌을 검사하고 새 전용 리소스 그룹을 생성합니다.	Azure Portal → Resource groups → 해당 그룹 → Overview 에서 이름·지역·Tags를 확인합니다. 같은 이름으로 새 그룹을 만들지 않습니다.
<code>deployment group create</code> , <code>template.json</code> , <code>_parameters(...)</code>	실제 ARM 템플릿 (파일: <code>infra/bootstrap.json</code>)에 정의된 Foundry·프로젝트·Search·관측 자원·모델·연결·역할을 배포합니다.	그룹의 Deployments → 해당 배포 → Deployment details/Inputs/Outputs 를 열어 템플릿 입력과 생성 항목을 대조합니다.
<code>config["models"]</code> 의 모델·버전·SKU·capacity	네 모델 배포를 계획 그대로 생성합니다.	Foundry Home → View deployments → 배포 → Details 에서 모델·버전·배포 유형·Tokens per Minute Rate Limit을 확인합니다.
<code>bootstrap status</code> 의 기존 그룹·배포·리소스 조회	같은 계획의 실제 상태를 읽고 로컬 기록과 대조합니다. 새 ARM 배포를 제출하지 않습니다.	같은 그룹 Deployments → 기존 배포와 실제 자원 목록을 새로고침합니다. Create로 다시 제출하지 않습니다.
<code>.env</code> 의 <code>AZURE_AI_PROJECT_ENDPOINT</code>	생성 완료 후 실제 프로젝트 연결 설정을 저장합니다.	Foundry Home → Project endpoint 를 대조합니다. Azure OpenAI endpoint를 대신 쓰지 않습니다.
역할 할당과 프로젝트·Search 관리 ID	사용자와 서비스에 서로 다른 범위의 권한을 설정합니다.	Azure Portal의 해당 자원 Identity, Access control (IAM) → Check access 에서 신원·Scope를 확인합니다. 이 확인을 위해 역할을 다시 추가하지 않습니다.

아래 `apply()`에서 그룹 생성 → ARM 검증·배포 → 완료 상태 조회 → 인벤토리 대조 → `.env` 저장 순서를 읽습니다. `bounded(...)`는 승인 유효기간·대기 한도를 확인한 뒤 원문에 표시된 Azure CLI 인자를 실행합니다. 로컬 파일·재개·소유권 보호도 실제 구현의 일부이므로 생략하지 않습니다.

실제 구현 코드 · 읽기 전용 · `lab/bootstrap.py:apply`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 `import`는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 1901–2092 (파일: `lab/bootstrap.py#L1901–L2092`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1901–L2092\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1901–L2092)

```

def apply(
    config_path: Path | str, approval_path: Path | str | None = None, *,
    run: Run = az_json, what_if: bool = False, retry: bool = False,
    sleep: Callable[[float], None] = time.sleep,
    clock: Callable[[], float] = time.monotonic,
) -> dict:
    """Provision only this plan's owned scope. Calling this function can incur cost."""
    directory, config, manifest = _load(config_path)
    if what_if and retry:
        raise BootstrapError("--retry cannot be combined with read-only --what-if.")
    if not what_if:
        approval = _approved_record(config, approval_path)
    else:
        approval = {}
    with _lock(directory):
        directory, config, manifest = _load(config_path)
        _check_outputs(directory, manifest)
        try:
            report, raw, observed = _preflight(config, manifest, run)
        except BootstrapError as exc:
            _record_failure(directory, "pre-apply", exc)
            raise
        _write(directory / "evidence" / f"pre-apply-{uuid4().hex}.local.json", raw)
        if what_if:
            if "resource_group" not in observed:
                raise BootstrapError("Read-only group what-if needs the already-owned group; it never
creates a group.")
            elif any(m["deprecated"] for m in report["models"]) and not
approval["accept_deprecated_models"]:
                raise ApprovalError("A selected model is deprecating; explicit acceptance or a newly
approved plan is required.")
            principal = raw["identity"]["user"]["id"]
            if what_if:
                path = directory / "evidence" / f"what-if-parameters-{uuid4().hex}.local.json"
                _write(path, _parameters(config, principal))
                result = run([
                    "deployment", "group", "what-if", "--subscription", config["subscription_id"],
                    "--resource-group", config["names"]["resource_group"], "--name",
config["deployment_name"],
                    "--template-file", str(directory / "template.json"), "--parameters", f"@{path}", "--
mode", "Incremental",
                    "--no-pretty-print",
                ])
                _write(directory / "evidence" / f"what-if-{uuid4().hex}.local.json", {"result": result})
                return {"status": "WHAT_IF_ONLY", "scope_sha256": config["scope_sha256"],
"mutations_performed": False}
            start = clock()
            deadline = start + approval["max_wait_seconds"]

    def bounded(args: list[str]) -> Any:
        try:
            validate_approval(config, approval)
        except BootstrapError as exc:
            raise ApprovalError(str(exc)) from exc
        remaining = deadline - clock()
        if remaining <= 0:
            raise BootstrapError("Approved wait exhausted; inspect status and resume, never delete

```

```

or recreate blindly.")
    return az_json(args, timeout=min(remaining, 120)) if run is az_json else run(args)

manifest["operator_principal_id"] = principal
manifest["approved_currency"] = approval["currency"]
manifest["approved_budget"] = approval["budget_amount"]
manifest["approved_budget_policy"] = approval.get("budget_policy", "BOUNDED")
initial_attempts = len(manifest["attempts"])
stage = "ownership"
active_command = None
submitted_attempt = None
try:
    if "resource_group" not in observed:
        if manifest["resources"]["resource_group"]["status"] not in {"planned", "pending"}:
            raise BootstrapError("Previously owned resource group is missing; automatic
restore/recreation is forbidden.")
        if manifest["resources"]["resource_group"]["status"] == "pending":
            raise ProvisioningRetryError(
                "UNKNOWN_GROUP_SUBMISSION",
                "A pending resource-group creation intent exists but the group is absent. "
                "Inspect original submission/activity evidence; do not issue another create
blindly.",
            )
        # Recheck immediately before conditional create; az group create would silently
upsert.
        if _get(manifest["resources"]["resource_group"], bounded) is not None:
            raise BootstrapError("Resource group name collision; no adoption or overwrite.")
        _pending(directory, manifest, ["resource_group"], "creating_group", approval)
        group = manifest["resources"]["resource_group"]
        stage = "group-create"
        active_command = [
            "rest", "--method", "PUT", "--url", f"{ARM}{group['id']}?api-version=
{group['api_version']}",
            "--headers", "If-None-Match=*", "--body", _json({"location": REGION, "tags":
_tags(config)}),
        ]
        bounded(active_command)
        remote = _get(group, bounded)
        if remote is None:
            raise BootstrapError("Group creation is not yet observable; use status before
resume.")
        _owned(config, "resource_group", group, remote, manifest, {})
        manifest["resources"]["resource_group"]["status"] = "present"
        _save(directory, manifest)
    observed = _inspect(config, manifest, bounded)
    if "resource_group" not in observed:
        raise BootstrapError("Owned group must exist before any ARM deployment.")
    arm = observed.get("arm_deployment", {}).get("properties", {}).get("provisioningState")
    stage, active_command = "arm-retry-check", None
    if _submission_required(manifest, observed, approval, retry):
        stage = "arm-validation"
        validation = _validate_arm(directory, config, principal, bounded)
        manifest["last_validation"] = validation
        parameter_path = Path(validation["parameters_path"])
        observed = _inspect(config, manifest, bounded)
        if not _submission_required(manifest, observed, approval, retry):
            raise BootstrapError("Deployment state changed during validation; inspect status

```

```

instead of issuing another create.")
    _pending(directory, manifest, [k for k in manifest["resources"] if k !=
"resource_group"], "deploying", approval)
    submitted_attempt = manifest["attempts"][-1]
    submitted_attempt.update(
        submission_outcome="pending", retry_requested=retry,
        validation_evidence_file=validation["evidence_file"],
    )
    _save(directory, manifest)
    stage = "arm-create"
    active_command = [
        "deployment", "group", "create", "--subscription", config["subscription_id"],
        "--resource-group", config["names"]["resource_group"], "--name",
config["deployment_name"],
        "--template-file", str(directory / "template.json"), "--parameters",
f"@{parameter_path}",
        "--mode", "Incremental", "--no-wait",
    ]
    response = bounded(active_command)
    submitted_attempt["submission_outcome"] = "submitted"
    _save(directory, manifest)
    _write(directory / "evidence" / f"create-{uuid4().hex}.local.json", {
        "command": active_command, "response": response, "recorded_at": _stamp(),
        "status": "SUBMITTED_NOT_VERIFIED",
    })
    arm = None
    stage = "arm-status"
    while arm != "Succeeded":
        deployment = _get(manifest["resources"]["arm_deployment"], bounded)
        arm = (deployment or {}).get("properties", {}).get("provisioningState")
        if arm in {"Failed", "Canceled", "Cancelled"}:
            payload = (deployment or {}).get("properties", {}).get("error")
            if payload:
                spec = manifest["resources"]["arm_deployment"]
                raise _azure_error("", _json(deployment), [
                    "rest", "--method", "GET", "--url",
                    f"{ARM}{spec['id']}?api-version={spec['api_version']}",
                ])
            raise AzureCommandError("DeploymentFailed", payload=deployment, message="ARM
deployment failed; inspect saved results before an explicitly budgeted retry.")
        if arm != "Succeeded":
            remaining = deadline - clock()
            if remaining <= 0:
                raise BootstrapError("Approved wait exhausted; deployment may still run.
Inspect status.")
            sleep(min(5, remaining))
        observed = _inspect(config, manifest, bounded)
        if set(observed) != set(manifest["resources"]):
            raise BootstrapError("ARM succeeded but the complete owned inventory is not
observable; do not claim success.")
        routing = observed["insights_connection"].get("properties", {}).get("metadata",
{}).get(TRACE_ROUTING_KEY)
        expected_routing = observed["insights"].get("properties", {}).get("ConnectionString")
        if not expected_routing or routing != expected_routing:
            raise BootstrapError("ARM success is insufficient: trace routing metadata must match
the owned component's ConnectionString.")
        for key, remote in observed.items():

```

```

        state = remote.get("properties", {}).get("provisioningState")
        if state is not None and str(state).lower() != "succeeded":
            raise BootstrapError(f"Resource is not ready: {key}.")
        if state is None and (key in RESOURCE_TYPES or key.startswith("model:")):
            raise BootstrapError(f"Resource readiness is missing: {key}.")
        manifest["resources"][key]["status"] = "succeeded"
    content = _env(config, observed["insights"], approval_path)
    if not (directory / ".env").exists():
        manifest["env_sha256"] = hashlib.sha256(content.encode()).hexdigest()
        manifest["phase"] = "writing_env"
        _save(directory, manifest)
        _write(directory / ".env", content, text=True)
    manifest["env_sha256"] = _file_digest(directory / ".env")
    manifest["phase"] = "succeeded"
    _save(directory, manifest)
    return {
        "status": "APPLIED", "scope_sha256": config["scope_sha256"],
        "env_path": str(directory / ".env"), "artifacts_dir": str(directory / "artifacts"),
        "resource_count": len(observed), "data_plane_verified": False,
        "live_status": "NOT_VERIFIED",
        "server_side_trace_connection": "CONFIGURED_PROJECT_MANAGED_IDENTITY",
        "trace_ingestion_verified": False,
    }
except BootstrapError as exc:
    if submitted_attempt is not None and submitted_attempt.get("submission_outcome") ==
"pending":
        submitted_attempt["submission_outcome"] = "unknown"
        evidence_file = _record_failure(directory, stage, exc, command=active_command)
        if submitted_attempt is not None:
            submitted_attempt["failure_evidence_file"] = evidence_file
        manifest.setdefault("failures", []).append({
            "recorded_at": _stamp(), "stage": stage, "error_code": getattr(exc, "code", None),
            "evidence_file": evidence_file,
        })
        if isinstance(exc, ApprovalError) and len(manifest["attempts"]) > initial_attempts:
            exc.mutations_performed = None
            manifest["phase"] = "interrupted"
            manifest["last_error"] = str(exc)
            manifest["last_error_code"] = getattr(exc, "code", None)
            manifest["last_error_evidence_file"] = evidence_file
            if getattr(exc, "validation_evidence_file", None):
                manifest["last_validation"] = {"status": "FAILED", "evidence_file":
exc.validation_evidence_file}
            _save(directory, manifest)
            raise

```

ARM 템플릿에 전달하는 실제 입력도 확인합니다. `names·models·사용자 ID·리소스 범위 역할·tags·보존 기간`이 이 객체로 전달됩니다.

실제 구현 코드 · 읽기 전용 · `lab/bootstrap.py:_parameters`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 1391-1400 (파일: [lab/bootstrap.py#L1391-L1400](#)) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L1391-L1400\)](#)

```
def _parameters(config: dict, principal: str) -> dict:
    return {
        "$schema": "https://schema.management.azure.com/schemas/2019-04-01/deploymentParameters.json#",
        "contentVersion": "1.0.0.0",
        "parameters": {key: {"value": value} for key, value in {
            "location": REGION, "names": config["names"], "tags": _tags(config),
            "models": config["models"], "roles": _roles(config),
            "operatorPrincipalId": principal, "retentionDays": config["retention_days"],
        }.items()},
    }
```

`status()` 원문도 함께 확인합니다. `apply`가 대기 초과됐을 때 어떤 원격 상태를 다시 읽는지 볼 수 있습니다.

실제 구현 코드 · 읽기 전용 · `lab/bootstrap.py:status`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 `import`는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

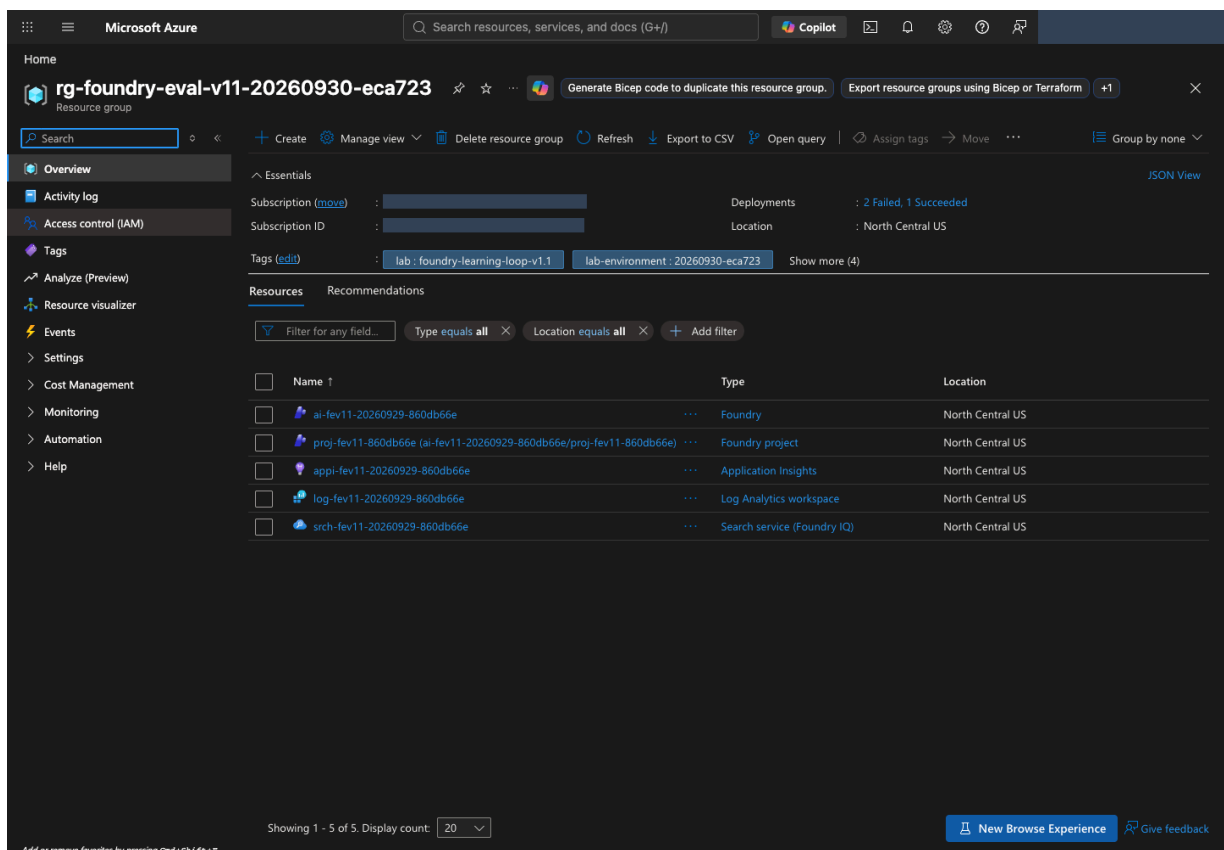
전체 구현 파일 · 2095-2122 (파일: [lab/bootstrap.py#L2095-L2122](#)) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/bootstrap.py#L2095-L2122\)](#)

```
def status(
    config_path: Path | str, *, run: Run = az_json, approval_path: Path | str | None = None,
) -> dict:
    """Inspect only declared resources; no deletes, provider registration, or permission repair."""
    directory, config, manifest = _load(config_path)
    _identity(config, manifest, run)
    try:
        observed = _inspect(config, manifest, run)
    except BootstrapError as exc:
        return {"status": "BLOCKED", "scope_sha256": config["scope_sha256"], "reason": str(exc)}
    result = {
        *_authorization(config, approval_path),
        "observation_status": "OBSERVED", "live_status": "NOT_VERIFIED",
        "scope_sha256": config["scope_sha256"], "phase": manifest["phase"],
        "resources": {key: {
            "present": key in observed,
            "provisioning_state": observed.get(key, {}).get("properties", {}).get("provisioningState",
"UNKNOWN"),
            "recorded_state": spec["status"], "sku": spec["sku"],
        } for key, spec in manifest["resources"].items()},
        "mutations_performed": False,
        "warning": "Search and any separately deployed fine-tuned model may still incur continuous
hosting costs.",
    }
    if result["approval_status"] == "APPROVED":
        result["status"] = "OBSERVED_APPROVAL_VALID"
        result["live_status"] = "PENDING_EXECUTION"
        result["readiness_status"] = "NOT_CHECKED_BY_STATUS"
```

```
_write(directory / "evidence" / f"status-{uuid4().hex}.local.json", {"observed": observed,
"summary": result})
return result
```

생성된 환경과 런타임 설정을 확인합니다

1. [Azure Portal](https://portal.azure.com) (<https://portal.azure.com>) → **Resource groups**에서 `config.json`의 `names.resource_group`을 검색합니다. 구독·지역·리소스 목록을 확인합니다.
2. [Foundry](https://ai.azure.com) (<https://ai.azure.com>)를 열고 **New Foundry**를 사용합니다. **Select a project to continue**가 나타나면 `names.project`와 같은 프로젝트를 선택하고 **Let's go**를 누릅니다. 환영 안내가 나타나면 읽고 **Close**로 닫습니다. 이미 New Foundry라면 좌측 상단 프로젝트 선택을 사용합니다. Classic의 hub 기반 프로젝트와 혼동하지 않습니다.
3. 프로젝트의 **Home**(일부 UI의 Overview)에서 **Project endpoint**를 확인합니다. `.env`의 `AZURE_AI_PROJECT_ENDPOINT`와 같은 `https://계정명.services.ai.azure.com/api/projects/프로젝트명` 형식이어야 합니다.
4. **Models + endpoints** 또는 **Build → Models**에서 `.env`의 `MODEL_DEPLOYMENT`, `JUDGE_DEPLOYMENT`, `OPTIMIZER_DEPLOYMENT`, `EMBEDDING_DEPLOYMENT`에 대응하는 실제 배포를 확인합니다. 메뉴 명칭이 달라지면 [화면 위치 안내](#)를 참고합니다.



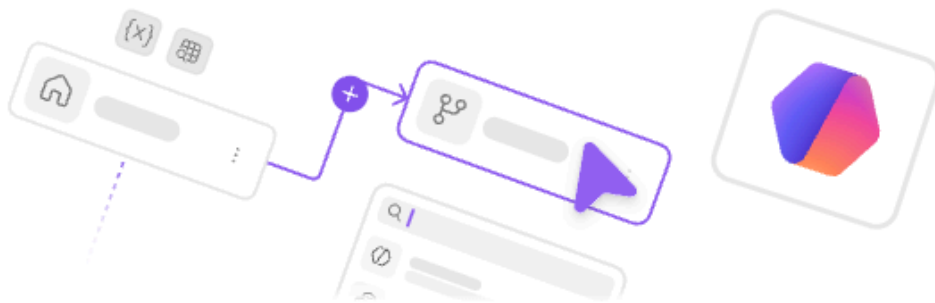
생성 계획과 실제 자원을 대조합니다. 그룹 이름·Location을 확인하고 Resources에서 Foundry, Foundry project, Search, Application Insights, Log Analytics를 찾습니다. Deployments에서는 각 배포의 상태와 실패 원인을 확인합니다. 원본 크기로 보기 (파일: `web/assets/portal/23-resource-group.png`)

Select a project to continue



To access the new Microsoft Foundry experience, please choose a project in a supported region.

proj-fev11-860db66e



Let's go

만든 프로젝트를 선택합니다. 이름을 계획의 `names.project`와 대조하고 Let's go로 엽니다. 이 화면은 기존 프로젝트 선택이며 추가 생성이 아닙니다. 이미 bootstrap을 완료했다면 Create a new project로 중복 생성하지 않습니다. 원본 크기로 보기 (파일: `web/assets/portal/24-select-project.png`)

Microsoft Foundry / proj-fev11-860db66e

Q Search with AI (🔍K)

New Foundry

Home

Discover

Build

Operate

Manage

Docs



Build an agent

Define instructions, add tools and knowledge, and create an agent for your project.

Create an agent

Start from code



Use a model

Manage your existing endpoints or deploy a new model.

View deployments

Project endpoint ⓘ

Azure OpenAI endpoint

프로젝트와 endpoint를 확인합니다. 좌측 상단 프로젝트 이름과 **Project endpoint**를 자신의 설정과 대조합니다. Azure OpenAI endpoint와 혼동하지 않으며 모델 목록은 View deployments로 엽니다. 원본 크기로 보기 (파일: `web/assets/portal/25-project-overview.png`)

터미널에서 신원·배포·TPM을 검사합니다

환경 생성이 완료되어 `.lab/lab-ko/.env`가 있을 때 다음 읽기 전용 명령을 실행합니다. 신원 확인도 이 명령 안에서 수행합니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env preflight
```

checks의 agent_tpm, judge_tpm, optimizer_tpm, iq_planner_tpm, embedding_tpm을 확인합니다. 각 항목의 observed가 실제 배포 TPM, expected가 위 최소 권장값입니다. 부족하거나 토큰 제한을 확인할 수 없으면 BLOCKED이며 다음 호출 단계로 진행하지 않습니다. 설정을 준비한 뒤 같은 읽기 전용 preflight를 다시 실행합니다.

코드 ↔ 포털 · 생성된 설정과 실행 신원을 대조합니다

실행 위치: 위의 [런타임 검사 명령](#)을 실행합니다. 프로그램이 생성된 설정 파일과 현재 CLI 신원을 비교하므로, 신원 검사 함수를 별도로 입력하지 않습니다.

명령·설정값	실제 동작	포털 확인 위치
--config .lab/lab-ko/.env의 AZURE_SUBSCRIPTION_ID, AZURE_TENANT_ID, EXPECTED_AZURE_USER	생성된 환경에 기록한 구독·테넌트·사용자를 읽습니다.	01에서 확인한 Azure Portal 계정의 Directory와 구독 Overview를 대조합니다.
check_identity()의 az account show --subscription ...	설정값과 CLI의 사용자·테넌트·구독·Enabled 상태를 자동 대조합니다. 불일치하면 자원 조회를 진행하지 않고 중단합니다.	같은 계정·구독인지 확인합니다. 로그인 성공만으로 다른 신원을 허용하지 않습니다.

아래 함수가 이 명령 안에서 수행하는 신원 검사입니다. 구독 선택만으로 테넌트 확인을 대신하지 않습니다. 이 검사는 SDK 토큰 요청이나 모델 호출이 아닙니다.

실제 구현 코드 · 읽기 전용 · lab/preflight.py:check_identity

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 69–87 (파일: lab/preflight.py#L69–L87) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L69-L87\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L69-L87)

```
def check_identity(config: Config, run: Callable = az_json) -> dict:
    account = run(["account", "show", "--subscription", config.subscription_id])
    if not isinstance(account, dict):
        raise LabError("az account show 응답 형식이 올바르지 않습니다.")
    actual_user = account.get("user", {}).get("name", "")
    checks = {
        "subscription": account.get("id") == config.subscription_id,
        "tenant": account.get("tenantId") == config.tenant_id,
        "user": actual_user.lower() == config.expected_user.lower(),
        "enabled": account.get("state") == "Enabled",
    }
    if not all(checks.values()):
        failed = ", ".join(key for key, ok in checks.items() if not ok)
        raise LabError(
```

```

        f"로그인 환경 불일치({failed}). 작업을 중단했습니다. "
        f"az login --tenant {config.tenant_id} 후 지정 계정으로 로그인해야 합니다. "
        "SDK는 다른 환경 자격 증명으로 자동 전환하지 않습니다."
    )
    return {"user": actual_user, "subscription": account["id"], "tenant": account["tenantId"]}

```

아래 런타임 `run_preflight()`는 `.env`의 대상 자원·배포를 조회합니다. `rateLimits`의 `key: token`을 읽거나 원시 ARM 메타데이터를 다시 조회하는 부분을 포털의 TPM 표시와 연결합니다. **구독 할당량 확인과 배포의 실제 TPM 확인은 다른 검사입니다.**

실제 구현 코드 · 읽기 전용 · `lab/preflight.py:run_preflight`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 `import`는 연결한 전체 파일에서 확인합니다. 실행 명령만 한 번 실행합니다.

전체 구현 파일 · 90–224 (파일: `lab/preflight.py#L90–L224`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L90-L224\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/preflight.py#L90-L224)

```

def run_preflight(config: Config, *, run: Callable = az_json) -> dict:
    identity = check_identity(config, run)
    account = run([
        "cognitiveservices", "account", "show", "--subscription", config.subscription_id,
        "--resource-group", config.resource_group, "--name", config.account,
    ])
    project = run([
        "resource", "show", "--ids", config.project_id, "--api-version", "2025-06-01",
    ])
    search = run([
        "search", "service", "show", "--subscription", config.subscription_id,
        "--resource-group", config.resource_group, "--name", config.search_service,
    ])
    deployments = run([
        "cognitiveservices", "account", "deployment", "list",
        "--subscription", config.subscription_id, "--resource-group", config.resource_group,
        "--name", config.account,
    ])
    checks: list[dict] = []
    for name, resource in (("foundry", account), ("project", project), ("search", search)):
        region = str(resource.get("location", "")).lower().replace(" ", "")
        checks.append({
            "name": f"{name}_region",
            "status": "PASS" if region == config.location else "BLOCKED",
            "observed": region,
            "expected": config.location,
        })
    endpoint = project.get("properties", {}).get("endpoints", {}).get("AI Foundry API")
    checks.append({
        "name": "project_endpoint",
        "status": "PASS" if endpoint == config.project_endpoint else "BLOCKED",
        "observed": endpoint,
    })
    deployment_map = {d["name"]: d for d in deployments}
    model_evidence = {}
    roles = (

```

```

        ("agent", config.model), ("judge", config.judge), ("optimizer", config.optimizer),
        ("iq_planner", config.planner), ("embedding", config.embedding),
    )
    rate_limit_sources = {}
    for name in dict.fromkeys(name for _, name in roles):
        deployment = deployment_map.get(name)
        if not deployment:
            continue
        limits = deployment.get("properties", {}).get("rateLimits")
        source = "Azure CLI deployment metadata"
        if not isinstance(limits, list) or not any(
            isinstance(limit, dict) and limit.get("key") == "token" for limit in limits
        ):
            # Use raw ARM when CLI metadata lacks token/request labels instead of inferring TPM.
            identifier = f"{config.account_id}/deployments/{quote(name, safe='')}\"
            deployment = run([
                "rest", "--method", "GET", "--url",
                f"https://management.azure.com{identifier}?api-version=2025-06-01",
            ])
            if (
                not isinstance(deployment, dict) or deployment.get("name") != name
                or str(deployment.get("id", "")).lower() != identifier.lower()
                or not isinstance(deployment.get("properties"), dict)
            ):
                raise LabError("TPM 조회 응답이 요청한 모델 배포와 다릅니다.")
            deployment_map[name] = deployment
            source = "Raw ARM deployment metadata"
            rate_limit_sources[name] = source
    for role, name in roles:
        deployment = deployment_map.get(name)
        ready = deployment and deployment.get("properties", {}).get("provisioningState") ==
        "Succeeded"
        checks.append({
            "name": f"{role}_deployment", "status": "PASS" if ready else "BLOCKED",
            "observed": name,
        })
    if deployment:
        limits = deployment.get("properties", {}).get("rateLimits")
        minimum_tpm = LAB_MINIMUM_TPM[role]
        try:
            observed_tpm = _tokens_per_minute(limits)
            sufficient = observed_tpm >= minimum_tpm
            reason = (
                "Meets the lab starting TPM requirement; RPM and shared load still apply."
                if sufficient else
                f"Allocate at least {minimum_tpm:,} TPM with operator approval, then rerun
preflight."
            )
        except ValueError as exc:
            observed_tpm, sufficient, reason = None, False, str(exc)
        checks.append({
            "name": f"{role}_tpm", "status": "PASS" if sufficient else "BLOCKED",
            "observed": observed_tpm, "expected": minimum_tpm, "reason": reason,
        })
        model_evidence[role] = {
            "deployment": name,
            "model": deployment.get("properties", {}).get("model"),

```

```

        "sku": deployment.get("sku"),
        "tokens_per_minute": observed_tpm,
        "rate_limit_source": rate_limit_sources[name],
    }
    if role == "iq_planner":
        observed_model = (deployment or {}).get("properties", {}).get("model", {}).get("name")
        checks.append({
            "name": "iq_planner_supported_model",
            "status": "PASS" if observed_model == "gpt-5.5" else "BLOCKED",
            "observed": observed_model,
            "expected": "gpt-5.5",
        })
    if role == "embedding":
        observed_model = (deployment or {}).get("properties", {}).get("model", {}).get("name")
        checks.append({
            "name": "embedding_supported_model",
            "status": "PASS" if observed_model == "text-embedding-3-small" else "BLOCKED",
            "observed": observed_model, "expected": "text-embedding-3-small",
        })
    public_access = account.get("properties", {}).get("publicNetworkAccess")
    if public_access == "Disabled":
        checks.append({
            "name": "network", "status": "BLOCKED",
            "observed": "PublicNetworkAccess=Disabled: 승인된 VNet 연결 환경에서 실행해야 합니다.",
        })
    return {
        "checked_at": datetime.now(timezone.utc).isoformat(),
        "kind": "read-only-management-preflight",
        "status": "PASS" if all(c["status"] == "PASS" for c in checks) else "BLOCKED",
        "identity": identity,
        "checks": checks,
        "models": model_evidence,
        "identities": {
            "foundry_project_managed_identity": project.get("identity", {}).get("principalId"),
            "search_managed_identity": search.get("identity", {}).get("principalId"),
        },
        "not_verified": [
            "데이터 평면 RBAC와 모델 추론 성공",
            "Foundry IQ 지식 베이스 생성·검색 권한",
            "Prompt Optimizer 및 Frontier Tuning 테넌트 접근 권한",
            "GlobalStandard는 North Central US 내부 처리를 보장하지 않습니다.",
            "TPM 충족은 남은 토큰, 공유 부하, RPM·버스트 제한 또는 429 없는 실행을 보장하지 않습니다.",
        ],
    }
}

```

완료 기준: 런타임 preflight 전체와 다섯 TPM 항목이 PASS이고, 자신의 프로젝트와 역할별 배포를 확인합니다. 이는 읽기 전용 구성 확인이며 실제 모델 응답 성공은 다음 단계에서 확인합니다.

03. 정책을 연결하고 Agent를 생성합니다

이 단계에서 하는 일: 합성 정책을 검색하는 읽기 전용 도구와 비교 기준인 v1 Agent를 연결합니다.

중요한 이유: 모델이 기억으로 답하는 것과 실제 정책을 검색해 답하는 것은 다릅니다. 검색이 사용자 권한으로 성공해도 Agent의 관리 ID 권한은 별도로 확인해야 합니다.

진행 방법·위치: 터미널에서 모델·검색·Agent를 준비하고 Foundry에서 지침·도구와 실제 응답을 확인합니다. 데이터 전송·호출은 승인된 범위에서 한 번씩 수행합니다.

실행 순서: [터미널 · 모델 확인](#) → [정책 업로드](#) → [검색 확인](#) → [v1 생성](#) → [Foundry · 응답 확인](#).

모델과 정책 검색을 확인합니다

이 세 명령은 실제 업로드·모델 호출을 수행하며 비용이 발생할 수 있습니다. 본인이 각 명령의 정상 결과를 확인한 뒤 다음 명령으로 이동합니다. 재개할 때는 자신의 기록에서 이미 완료한 작업을 확인하고 불필요한 호출을 반복하지 않습니다.

1. 모델이 실제로 응답하는지 확인합니다

smoke는 짧은 동작 확인입니다.

먼저 [02의 TPM 검사](#)를 통과해야 합니다. 짧은 응답 한 번의 성공만으로 전체 평가에 필요한 처리량을 확보했다고 판단하지 않습니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env smoke --run-id model-smoke --confirm
```

출력의 `status: completed`를 확인합니다.

코드 ↔ 포털 · 확인한 CLI 신원으로 SDK를 연결합니다

실행 위치: 위의 [모델 연결 확인 명령](#)을 실행합니다. 아래 인증 함수는 실제 SDK 호출 안에서 자동으로 사용되며, 별도 터미널 명령이 아닙니다.

실제 코드	실제 동작	포털 확인 위치
<code>credential_for()의 AzureCliCredential(subscription=...)</code>	02의 설정으로 CLI 신원을 다시 확인하고 자격 증명 객체를 만듭니다. SDK가 서비스를 호출할 때 그 신원으로 토큰을 요청합니다. API key나 다른 자격 증명으로 전환하지 않습니다.	Foundry 우측 상단 로그인 계정과 프로젝트의 구독을 설정값과 대조합니다. 토큰을 포털에 붙여넣는 작업은 없습니다.

아래 첫 함수는 SDK에 전달할 자격 증명을 만들고, 다음 함수는 실제 모델 요청을 보냅니다. 인증 연결과 모델 응답 확인은 이 단계에서 함께 수행합니다.

실제 구현 코드 · 읽기 전용 · lab/auth.py:credential_for

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 11–15 (파일: lab/auth.py#L11–L15) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/auth.py#L11-L15\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/auth.py#L11-L15)

```
def credential_for(config: Config) -> AzureCliCredential:
    check_identity(config)
    # Azure CLI rejects get-access-token when both tenant and subscription are
    # supplied. The identity check above binds the subscription to the tenant.
    return AzureCliCredential(subscription=config.subscription_id, process_timeout=30)
```

smoke_model() 안의 client.responses.create(model=config.model, ...)가 실제 Sol 모델 호출입니다. Agent를 아직 생성하지 않고 짧은 입력·최대 출력 128토큰으로 연결을 확인하며, 호출 전에 기록을 써서 결과 불명인 요청을 반복하지 않습니다.

실제 구현 코드 · 읽기 전용 · lab/agents.py:smoke_model

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 168–207 (파일: lab/agents.py#L168–L207) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L168-L207\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L168-L207)

```
def smoke_model(config: Config, run_id: str) -> dict:
    directory = safe_run_dir(run_id)
    path = directory / "model-smoke.json"
    if path.exists():
        previous = read_json(path)
        require_content_language(previous)
        if previous.get("project_endpoint") != config.project_endpoint or previous.get("deployment")
        != config.model:
            raise LabError("기존 model smoke의 환경/배포가 다릅니다.")
        if previous.get("status") != "completed":
            raise LabError("이 model smoke는 실패 또는 결과 불명입니다. 원본을 확인하기 전 재호출하지 않습니다.")
        return previous
    snapshot = model_snapshot(config, config.model)
    record = {
        **language_metadata(),
        "kind": "LIVE_MODEL_SMOKE_NOT_QUALITY_EVALUATION", "status": "submitting",
        "started_at": datetime.now(timezone.utc).isoformat(),
        "code": code_provenance(),
        "project_endpoint": config.project_endpoint, "deployment": config.model,
        "model_snapshot": snapshot, "response_id": None, "cost": {"status": "NOT_OBSERVED"},
        "request": {"input": "Synthetic Contoso lab connectivity check. Reply with the single word
        READY.", "max_output_tokens": 128},
    }
    write_once_json(path, record)
```

```

        started = time.perf_counter()
        with credential_for(config) as credential:
            with AIProjectClient(endpoint=config.project_endpoint, credential=credential, retry_total=0)
as project:
                with project.get_openai_client(max_retries=0, timeout=120.0) as client:
                    response = client.responses.create(model=config.model, **record["request"],
store=True)

                    record["response_id"] = response.id
                    record["response"] = response.model_dump(mode="json")
                    record["usage"] = record["response"].get("usage")
                    record["latency_ms"] = (time.perf_counter() - started) * 1000
                    record["status"] = "completed" if (
                        record["response"].get("status") == "completed"
                        and isinstance(response.id, str) and response.id.strip()
                        and isinstance(response.output_text, str) and response.output_text.strip()
                    ) else "failed"
                    save_json(path, record)
            if record["status"] != "completed":
                raise LabError("모델 smoke가 완료된 비어 있지 않은 응답을 반환하지 않았습니다.")
            return record

```

2. 정책 검색을 준비합니다

제공된 합성 정책 8개 (파일: data/knowledge/documents.json)를 검색 서비스에 올립니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env iq prepare --confirm
```

출력의 uploaded_documents: 8을 확인합니다. 프로그램이 검색 인덱스(검색용 문서 모음), knowledge base(지식 베이스), MCP 연결을 준비합니다. **MCP(Model Context Protocol)는 Agent와 도구를 연결하는 규약**입니다. 여기서는 정책을 읽는 검색 도구만 연결하며 내부 코드를 수정할 필요는 없습니다.

코드 ↔ 포털 · 정책 검색을 구성합니다

실행 위치: 터미널에서 위의 iq prepare 다음 아래의 iq probe를 실행합니다. 아래 패널은 두 명령의 내부 구현이며 별도로 실행하지 않습니다.

실제 코드·설정	실제 동작	포털 조작·확인 위치
knowledge_payloads()["index"]의 fields, vectorSearch, semantic	정책 필드·1536차원 벡터·HNSW-embedding-semantic 설정을 정의합니다.	Azure Portal → 해당 Search → Indexes → 생성된 인덱스 → Fields/JSON 에서 필드와 검색 구성을 확인합니다. 인덱스를 다시 만들지 않습니다.
embed()의 client.embeddings.create(...)	8개 정책의 제목·내용을 실제 embedding 배포에 보냅니다.	Foundry View deployments → embedding 배포 → Details 에서 모델·배포 이름을 확인합니다. 벡터 값 자체는 Search 인덱스에 저장되며 Agent Playground에 표시되는 설정이 아닙니다.

실제 코드·설정	실제 동작	포털 조작·확인 위치
<code>prepare_knowledge()</code> 의 PUT·POST	인덱스 생성, 문서 업로드, knowledge source·knowledge base·프로젝트 연결을 순서대로 생성합니다.	Search의 Indexes , Foundry Build → Knowledge → Knowledge bases 에서 이름·Knowledge sources를 대조합니다.
<code>connection.authType</code> , <code>audience</code> , <code>target</code>	프로젝트 관리 ID로 Search의 MCP endpoint에 접근하는 연결입니다.	Foundry Knowledge의 Connection 을 확인합니다. UI에 <code>authType</code> · <code>audience</code> 편집란이 없다면 <code>knowledge/config-snapshot.json</code> 을 읽으며 임의 값을 넣지 않습니다.
<code>probe_knowledge()</code> 의 <code>/retrieve-references-activity</code>	실제 질문으로 정책 검색을 확인합니다.	포털의 Active 상태와 구분합니다. 실제 검색 결과는 <code>probe</code> 출력·기록이고, Agent 경유 호출은 아래 Playground에서 별도로 확인합니다.

먼저 서비스에 보내는 설정 객체 전체, 이어서 그 설정을 만들고 업로드하는 실제 코드를 읽습니다. `config.*`는 02의 `.env`, `names`는 자신의 기록에 있는 실제 객체 이름입니다.

실제 구현 코드 · 읽기 전용 · `lab/knowledge.py:knowledge_payloads`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 `import`는 연결한 전체 파일에서 확인합니다. 실행 명령만 한 번 실행합니다.

전체 구현 파일 · 33-105 (파일: `lab/knowledge.py#L33-L105`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L33-L105\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L33-L105)

```
def knowledge_payloads(config: Config, names: dict, planner_model: str) -> dict:
    if not config.embedding:
        raise LabError("백터 검색을 위해 EMBEDDING_DEPLOYMENT를 명시해야 합니다.")
    analyzer = f"{selected_language()}.microsoft"
    return {
        "index": {
            "name": names["index"],
            "fields": [
                {"name": "id", "type": "Edm.String", "key": True, "filterable": True, "retrievable": True},
                {"name": "title", "type": "Edm.String", "searchable": True, "retrievable": True, "analyzer": analyzer},
                {"name": "content", "type": "Edm.String", "searchable": True, "retrievable": True, "analyzer": analyzer},
                {"name": "effective_date", "type": "Edm.String", "filterable": True, "retrievable": True},
                {"name": "content_vector", "type": "Collection(Edm.Single)", "searchable": True, "retrievable": False, "dimensions": DIMENSIONS, "vectorSearchProfile": "contoso-vector"},
            ],
            "vectorSearch": {
                "algorithms": [{"name": "contoso-hnsw", "kind": "hnsw", "hnswParameters": {"metric": "cosine"}}],
                "profiles": [{"name": "contoso-vector", "algorithm": "contoso-hnsw", "vectorizer": "contoso-embedding"}],
                "vectorizers": [{"name": "contoso-embedding", "kind": "azureOpenAI",
```

```

        "azureOpenAIParameters": {
            "resourceUri": config.openai_endpoint, "deploymentId": config.embedding,
            "modelName": "text-embedding-3-small",
        },
    },
    "semantic": {
        "defaultConfiguration": "support-semantic",
        "configurations": [{
            "name": "support-semantic",
            "prioritizedFields": {
                "titleField": {"fieldName": "title"},
                "prioritizedContentFields": [{"fieldName": "content"}],
            },
        }],
    },
    "source": {
        "name": names["source"],
        "kind": "searchIndex",
        "searchIndexParameters": {
            "searchIndexName": names["index"],
            "semanticConfigurationName": "support-semantic",
            "sourceDataFields": [{"name": field} for field in ("id", "title", "content",
"effective_date")],
            "searchFields": [{"name": field} for field in ("title", "content", "content_vector")],
        },
    },
    "base": {
        "name": names["base"],
        "description": "Synthetic Contoso policies. Stable ATLAS document IDs are in titles and
source data.",
        "knowledgeSources": [{"name": names["source"]}],
        "models": [{
            "kind": "azureOpenAI",
            "azureOpenAIParameters": {
                "resourceUri": config.openai_endpoint,
                "deploymentId": config.planner,
                "modelName": planner_model,
            },
        }],
        "retrievalReasoningEffort": {"kind": "low"},
        "outputMode": "extractiveData",
    },
    "connection": {
        "properties": {
            "authType": "ProjectManagedIdentity",
            "category": "RemoteTool",
            "target": search_url(config, "knowledgebases", names["base"], "/mcp"),
            "isSharedToAll": False,
            "audience": "https://search.azure.com/",
            "metadata": {"ApiType": "Azure"},
        },
    },
}

```

다음 짧은 함수가 실제 **GET → 생성 전용 PUT → 소유 기록 저장**입니다. 이어지는 `prepare_knowledge()`가 인덱스 `·source·base·connection`에 이 동작을 적용합니다.

실제 구현 코드 · 읽기 전용 · lab/knowledge.py:ensure_created

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 108–125 (파일: `lab/knowledge.py#L108–L125`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L108-L125\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L108-L125)

```
def ensure_created(config: Config, http: JsonHttp, kind: str, name: str, url: str, payload: dict) -> dict:
    state = workspace(config)
    owned = [item for item in state["created"] if item.get("kind") == kind and item.get("name") == name]
    try:
        existing = http.request("GET", url)
    except CloudRequestError as exc:
        if exc.status != 404:
            raise
        existing = None
    if existing is not None:
        if not owned:
            raise LabError(f"{name}이 이미 존재하지만 이 실습의 생성 기록이 없습니다. 덮어쓰지 않습니다.")
        return existing.body
    result = http.request("PUT", url, payload, create_only=True)
    if result.status not in {200, 201}:
        raise LabError(f"{name}: 비동기/예상 밖 상태 {result.status}. 완료로 기록하지 않습니다.")
    record_created(config, {"kind": kind, "name": name, "url": url, "etag": result.etag})
    return result.body
```

실제 구현 코드 · 읽기 전용 · lab/knowledge.py:prepare_knowledge

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 128–239 (파일: `lab/knowledge.py#L128–L239`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L128-L239\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L128-L239)

```
def prepare_knowledge(config: Config) -> dict:
    state = workspace(config, create=True)
    names = resource_names(config, state)
    documents_path = content_path(ROOT, "data/knowledge/documents.json")
    documents = read_json(documents_path)
    content_hash = sha256_file(documents_path)
    setup_path = ARTIFACTS / "knowledge/setup.json"
    previous = None
    if setup_path.exists():
        previous = read_json(setup_path)
        require_content_language(previous)
    if previous.get("documents_sha256") != content_hash or previous.get("names") != names:
        raise LabError("기존 지식 준비와 원본/범위가 다릅니다. 다른 실험을 기존 인덱스에 덮어쓰지 않습니다.")
```

```

    if previous.get("status") in {"created_not_retrieval_tested", "retrieval_verified"}:
        if previous.get("retrieval", {}).get("embedding_deployment") != config.embedding:
            raise LabError("준비된 인덱스의 embedding 계약이 다릅니다. 새 실험을 생성해야 합니다.")
        return previous
with credential_for(config) as credential:
    planner = az_json([
        "cognitiveservices", "account", "deployment", "show",
        "--subscription", config.subscription_id, "--resource-group", config.resource_group,
        "--name", config.account, "--deployment-name", config.planner,
    ])
    model = planner.get("properties", {}).get("model", {})
    if not model.get("name"):
        raise LabError("IQ planner의 실제 모델 이름을 확인할 수 없습니다.")
    # This workshop's main lane was researched for gpt-5.5, not arbitrary
    # answer models that may be unsupported by the knowledge-base planner.
    if model["name"] != "gpt-5.5":
        raise LabError("이 경로의 IQ planner는 확인된 gpt-5.5 배포를 요구합니다. 다른 모델로 자동 전환하지 않습니다.")

    payloads = knowledge_payloads(config, names, model["name"])
    payload_hash = hashlib.sha256(json.dumps(payloads, sort_keys=True).encode()).hexdigest()
    if previous and previous.get("payload_sha256") != payload_hash:
        raise LabError("미완료 검색 설정의 구성 계약이 바뀌었습니다. 같은 인덱스에 덮어쓰지 않습니다.")
    setup = {
        **language_metadata(),
        "status": "preparing", "workspace_id": state["workspace_id"], "names": names,
        "project_id": config.project_id, "documents_sha256": content_hash,
        "planner": {"deployment": config.planner, "model": model},
        "search_api_version": SEARCH_API,
        "payload_sha256": payload_hash,
        "retrieval": {
            "reasoning": "low", "output": "extractiveData",
            "embedding_model": "text-embedding-3-small", "embedding_deployment": config.embedding,
            "dimensions": DIMENSIONS, "index_algorithm": "hnsf", "metric": "cosine",
            "query_modes": ["vector", "hybrid", "agentic-iq-mcp"],
            "direct_probe_is_agent_evidence": False,
        },
    }
    save_json(setup_path, setup)
    save_json(ARTIFACTS / "knowledge/config-snapshot.json", payloads)
    search = JsonHttp(credential, scope=SEARCH_SCOPE, allowed_origin=config.search_endpoint)
    index_url = search_url(config, "indexes", names["index"])
    _ensure_created(config, search, "search_index", names["index"], index_url, payloads["index"])
    embedding = embed(
        config, [f"{doc['title']}\n{doc['content']}" for doc in documents],
        ARTIFACTS / "knowledge/document-embeddings.json",
    )
    vectors = validate_vectors(embedding["response"], len(documents))
    uploaded = [{
        "@search.action": "mergeOrUpload",
        "id": doc["id"], "title": f"{doc['id']} | {doc['title']}",
        "content": f"{doc['id']}\n{doc['content']}",
        "effective_date": doc["effective_date"],
        "content_vector": vector,
    } for doc, vector in zip(documents, vectors, strict=True)]
    upload_path = ARTIFACTS / "knowledge/upload-response.json"
    upload_contract = {"documents_sha256": content_hash, "payload_sha256": payload_hash,
        "embedding_sha256": sha256_file(ARTIFACTS / "knowledge/document-

```

```

embeddings.json"))}
    if upload_path.exists():
        upload_record = read_json(upload_path)
        if upload_record.get("contract") != upload_contract or upload_record.get("status") !=
"completed":
            raise LabError("업로드 계약 변경 또는 결과 불명입니다. 업로드를 반복하지 않고 실제 인덱스 상태를 확인해야 합
니다.")
        upload_body = upload_record["response"]
    else:
        write_once_json(upload_path, {"status": "submitting", "contract": upload_contract})
        upload = search.request(
            "POST", search_url(config, "indexes", names["index"], "/docs/index"), {"value":
uploaded},
        )
        upload_body = upload.body
        save_json(upload_path, {"status": "response_received", "contract": upload_contract,
"response": upload_body})
        statuses = upload_body.get("value", [])
        if (
            len(statuses) != len(uploaded)
            or {item.get("key") for item in statuses} != {doc["id"] for doc in documents}
            or any(item.get("status") is not True for item in statuses)
        ):
            save_json(ARTIFACTS / "knowledge/upload-error.json", upload_body)
            raise LabError("일부 문서 업로드가 실패했습니다. knowledge/upload-error.json을 확인해야 합니다.")
            save_json(upload_path, {"status": "completed", "contract": upload_contract, "response":
upload_body})
        _ensure_created(
            config, search, "knowledge_source", names["source"],
            search_url(config, "knowledgesources", names["source"]), payloads["source"],
        )
        _ensure_created(
            config, search, "knowledge_base", names["base"],
            search_url(config, "knowledgebases", names["base"]), payloads["base"],
        )
        connection_url = (
            f"https://management.azure.com{config.project_id}/connections/"
            f"{names['connection']}?api-version={CONNECTION_API}"
        )
        arm = JsonHttp(credential, scope=ARM_SCOPE, allowed_origin="https://management.azure.com")
        _ensure_created(config, arm, "project_connection", names["connection"], connection_url,
payloads["connection"])
        setup["status"] = "created_not_retrieval_tested"
        setup["mcp_endpoint"] = payloads["connection"]["properties"]["target"]
        setup["connection_name"] = names["connection"]
        setup["uploaded_documents"] = len(uploaded)
        setup["embedding_usage"] = embedding["response"].get("usage")
        save_json(setup_path, setup)
        save_json(ARTIFACTS / "knowledge/documents-snapshot.json", documents)
        return setup

```

embedding 호출도 감춰진 로컬 점수 계산이 아닙니다. 다음 함수가 Azure OpenAI의 embedding API를 호출하고 원본 응답·입력 해시·모델·토큰 사용량을 보관합니다.

실제 구현 코드 · 읽기 전용 · lab/embeddings.py:embed

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 41-97 (파일: lab/embeddings.py#L41-L97) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/embeddings.py#L41-L97\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/embeddings.py#L41-L97)

```
def embed(config: Config, texts: list[str], cache_path: Path) -> dict:
    if not config.embedding or not texts or any(not isinstance(text, str) or not text.strip() for text
    in texts):
        raise LabError("EMBEDDING_DEPLOYMENT와 비어 있지 않은 embedding 입력이 필요합니다.")
    deployment = model_snapshot(config, config.embedding)
    if deployment["model"].get("name") != "text-embedding-3-small":
        raise LabError("이 인덱스의 동결 계약은 text-embedding-3-small/1536입니다. 자동 대체하지 않습니다.")
    contract = {
        "endpoint": config.openai_endpoint, "deployment": deployment,
        "dimensions": DIMENSIONS,
        "input_sha256": hashlib.sha256(json.dumps(texts, ensure_ascii=False).encode()).hexdigest(),
        "count": len(texts),
    }
    if cache_path.exists():
        previous = read_json(cache_path)
        if previous.get("contract") != contract:
            raise LabError("기존 embedding 실행과 입력/모델/범위가 다릅니다. 기존 기록을 보존해야 합니다.")
        if previous.get("status") != "completed":
            raise LabError("이 embedding 요청은 실패 또는 결과 불명입니다. 원격 요청을 확인하기 전 재호출하지 않습니다.")
        validate_vectors(previous["response"], len(texts))
        return previous
    from azure.identity import get_bearer_token_provider
    from openai import APIError, OpenAI
    from azure.core.exceptions import AzureError
    from lab.auth import credential_for

    record = {"kind": "LIVE_AZURE_EMBEDDING", "contract": contract, "status": "submitting",
              "code": code_provenance(),
              "started_at": datetime.now(timezone.utc).isoformat(), "cost": {"status":
    "NOT_OBSERVED"}}
    cache_path.parent.mkdir(parents=True, exist_ok=True)
    try:
        with cache_path.open("x", encoding="utf-8") as stream:
            json.dump(record, stream, ensure_ascii=False, indent=2)
    except FileExistsError as exc:
        raise LabError("동시 embedding 실행을 차단했습니다. 기존 기록을 확인해야 합니다.") from exc
    started = time.perf_counter()
    try:
        with credential_for(config) as credential:
            with OpenAI(
                base_url=f"{config.openai_endpoint}/openai/v1/",
                api_key=get_bearer_token_provider(credential, "https://ai.azure.com/.default"),
                max_retries=0, timeout=120,
            ) as client:
                response = client.embeddings.create(
                    model=config.embedding, input=texts, dimensions=DIMENSIONS,
                    encoding_format="float",
                )
```

```

        record["response"] = response.model_dump(mode="json")
        record["request_id"] = getattr(response, "_request_id", None)
        save_json(cache_path, record)
        validate_vectors(record["response"], len(texts))
    except (APIError, AzureError, LabError) as exc:
        record.update(status="failed_or_unknown", error={"type": type(exc).__name__, "message":
str(exc)})
        save_json(cache_path, record)
        raise
    record.update(status="completed", latency_ms=(time.perf_counter() - started) * 1000,
                  completed_at=datetime.now(timezone.utc).isoformat())
    save_json(cache_path, record)
    return record

```

3. 질문으로 정책이 검색되는지 확인합니다

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env iq probe --confirm
```

status: retrieval_verified와 비어 있지 않은 출처를 확인합니다. created_not_retrieval_tested는 생성만 성공했다는 뜻입니다. 오류가 발생하면 기록을 지우지 않고 [검색 오류 해결](#)을 확인합니다.

아래 POST .../retrieve의 질문·출처 요청과 실제 응답 검사를 읽습니다. **직접 검색 성공은 로그인한 사용자 경로의 근거이며**, 프로젝트 관리 ID의 Agent 도구 호출 근거와 같지 않습니다.

실제 구현 코드 · 읽기 전용 · lab/knowledge.py:probe_knowledge

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 256–296 (파일: `lab/knowledge.py#L256–L296`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L256-L296\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L256-L296)

```

def probe_knowledge(config: Config, query: str) -> dict:
    setup = load_knowledge(config)
    if not query.strip():
        raise LabError("지식 검색 질문이 비어 있습니다.")
    names = setup["names"]
    request = {
        "messages": [{"role": "user", "content": [{"type": "text", "text": query}]}],
        "outputMode": "extractiveData",
        "maxOutputSize": 6000,
        "knowledgeSourceParams": [{
            "knowledgeSourceName": names["source"], "kind": "searchIndex",
            "includeReferences": True, "includeReferenceSourceData": True,
            "resultsProcessing": "rerank", "failOnError": True,
        }],
    }
    key = hashlib.sha256(query.encode("utf-8")).hexdigest()[:16]
    directory = ARTIFACTS / "knowledge/probes" / key
    intent_path, response_path = directory / "iq-request.json", directory / "iq-response.json"

```

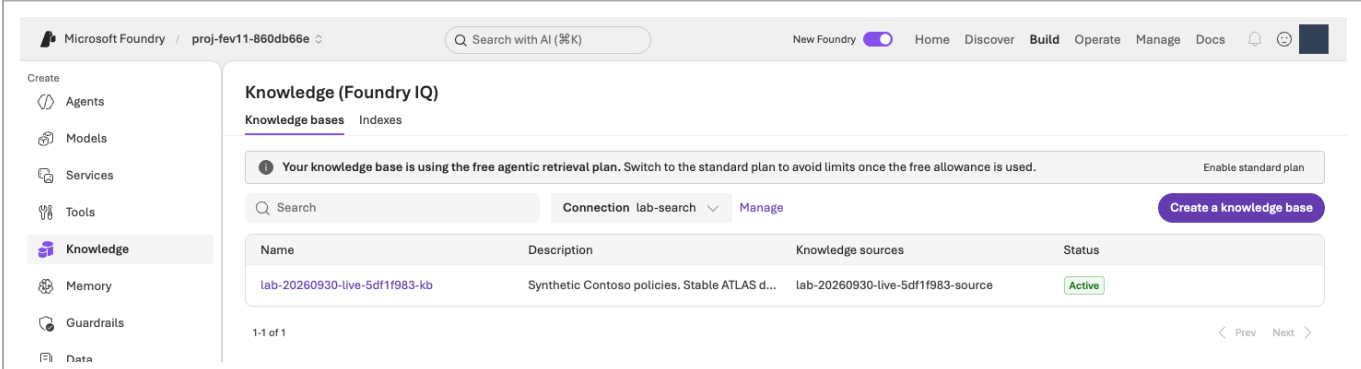
```

if response_path.exists():
    if read_json(intent_path) != request:
        raise LabError("기존 IQ 요청 계약이 다릅니다. 원본 기록을 보존해야 합니다.")
    body = read_json(response_path)
else:
    if intent_path.exists():
        raise LabError("이 IQ probe는 결과 불명입니다. planner를 다시 유료 호출하지 않고 원격 요청을 확인해야 합니다.")

write_once_json(intent_path, request)
with credential_for(config) as credential:
    http = JsonHttp(credential, scope=SEARCH_SCOPE, allowed_origin=config.search_endpoint)
    result = http.request(
        "POST", search_url(config, "knowledgebases", names["base"], "/retrieve"), request,
    )
    body = result.body
    save_json(response_path, body)
if not (ARTIFACTS / "knowledge/retrieve-response.json").exists():
    save_json(ARTIFACTS / "knowledge/retrieve-response.json", body)
if not body.get("response") or not body.get("references") or not body.get("activity"):
    raise LabError("검색 응답·출처·활동 중 일부가 없습니다. 보존한 iq-response.json을 확인해야 하며 완료 처리하지 않습니다.")
setup["status"] = "retrieval_verified"
setup["probe_scope"] = "Human CLI identity; deployed-agent managed identity must still be verified by an agent call."
save_json(ARTIFACTS / "knowledge/setup.json", setup)
return {"status": setup["status"], "references": body["references"], "activity": body["activity"], "scope": setup["probe_scope"]}

```

포털에서도 **Build → Knowledge → Knowledge bases**를 열고 Connection과 생성된 지식 베이스·Knowledge sources를 확인합니다. 목록의 지식 베이스 한 개가 정책 문서 한 개를 뜻하지는 않습니다.



정책 연결이 있는지 확인합니다. Connection 선택과 해당 지식 베이스의 Knowledge sources·Active를 확인합니다. Active는 등록 상태이며 실제 검색 성공은 **iq probe**로 확인합니다. 무료 검색 배너는 전체 실습이 무료라는 뜻이 아니며, 준비 확인을 위해 요금제 변경 버튼을 누르지 않습니다. 원본 크기로 보기 (파일: web/assets/portal/26-knowledge-base.png)

같은 조건으로 평가할 v1을 만듭니다

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env native-agent --version 1 --confirm
```

이 명령은 `prompts/baseline.txt`의 지침과 방금 확인한 정책 도구로 `lab-ko-iq` 버전 1을 만듭니다. 출력의 `agent_name`, `version`, `receipt`를 기록합니다. **Receipt는 실행 결과를 저장한 기록 파일입니다.** 같은 소유 환경·동일 구성이면 기존 v1을 재사용하며, 다른 Agent를 인수하거나 v3를 만들지 않습니다.

코드 ↔ 포털 · Agent의 전체 구성을 확인합니다

실행 위치: 위의 v1 생성 명령을 실행한 뒤 아래의 Foundry Playground에서 버전·설정·실제 응답을 확인합니다. 포털에서 다시 생성하거나 새 버전으로 저장하지 않습니다.

실제 코드·설정	포털 조작·확인 위치
<code>definition["model"] = config.model</code>	Build → Agents → 실제 Agent → Version 1 → Model 에서 같은 배 포를 확인합니다.
<code>definition["instructions"] = prompt.read_text(...)</code>	같은 화면의 Instructions 를 원본 지침 (파일: <code>prompts/baseline.txt</code>)과 대조합니다. 확인 중 Save/새 버전 을 누르지 않습니다.
<code>knowledge_tool()</code> 의 MCP <code>endpoint-allowed_tools-project_connection_id</code>	Knowledge/Tools 에서 정책 연결을 확인합니다. 허용 도구는 <code>knowledge_base_retrieve</code> 이며 도구가 실제 실행됐는지는 응답의 실행 상세에서 확인합니다.
<code>native_response_format()</code> 의 <code>json_schema, strict</code>	생성된 Agent의 구조화 출력 설정입니다. UI가 전체 스키마를 보여주지 않으면 <code>agents/native-v1.json</code> 의 <code>agent.definition.text</code> 를 읽습니다. 자연어 지침으로 같은 설정을 추측해 다시 만들지 않습니다.
<code>project.agents.create_version(...), draft=False</code>	생성 후 포털의 Version 에서 실제 1을 확인합니다. 새 버전 생성은 운영용 계시가 아닙니다.

먼저 연결한 도구와 Agent 정의를 읽고, 다음 함수의 **실제** `project.agents.create_version` 호출까지 확인합니다. 두 함수 사이의 고정 버전·소유권·같은 구성 재사용 검사도 실제 동작입니다.

실제 구현 코드 · 읽기 전용 · `lab/knowledge.py:knowledge_tool`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 `import`는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 347~358 (파일: `lab/knowledge.py#L347-L358`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L347-L358\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/knowledge.py#L347-L358)

```
def knowledge_tool(config: Config) -> MCPTool:
    setup = load_knowledge(config)
    if setup["status"] != "retrieval_verified":
        raise LabError("먼저 python -m lab iq probe --confirm 으로 실제 검색을 확인해야 합니다.")
    return MCPTool(
        server_label="contoso-knowledge",
        server_url=setup["mcp_endpoint"],
        server_description="Read-only Contoso Atlas Cloud policies. Cite stable ATLAS document IDs shown in the returned title/content, not numeric retrieval reference IDs.",
        allowed_tools=["knowledge_base_retrieve"],
```

```

        require_approval="never",
        project_connection_id=setup["connection_name"],
    )

```

실제 구현 코드 · 읽기 전용 · lab/agents.py:create_native_agent

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 124–165 (파일: `lab/agents.py#L124–L165`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L124-L165\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L124-L165)

```

def create_native_agent(config: Config, version: str, prompt: Path) -> dict:
    """Connect the prepared policy tool to a fixed, strict-output workshop Agent."""
    from lab.knowledge import knowledge_tool

    if version not in {"1", "2"}:
        raise LabError("The native workshop Agent permits only versions 1 and 2.")
    if not prompt.is_file() or not prompt.read_text(encoding="utf-8").strip():
        raise LabError(f"A nonempty instruction file is required: {prompt}")
    state = workspace(config)
    tool = knowledge_tool(config)
    snapshot = model_snapshot(config, config.model)
    snapshot_path = ARTIFACTS / "agents/native-model.json"
    if snapshot_path.exists():
        if read_json(snapshot_path) != snapshot:
            raise LabError("The Agent model deployment changed after setup; preserve the original comparison.")
    elif version == "2":
        raise LabError("Create and verify native Agent v1 before preparing v2.")
    else:
        write_once_json(snapshot_path, snapshot)
    definition = {
        "kind": "prompt",
        "model": config.model,
        "instructions": prompt.read_text(encoding="utf-8"),
        "tools": [tool.as_dict()],
        "text": native_response_format(),
    }
    receipt = ARTIFACTS / "agents" / f"native-v{version}.json"
    with credential_for(config) as credential:
        with AIProjectClient(endpoint=config.project_endpoint, credential=credential, retry_total=0)
as project:
        result = ensure_fixed_release(
            project, project_endpoint=config.project_endpoint, agent_name=config.agent_name("iq"),
            version=version, definition=definition, receipt=receipt,
            workspace_id=state["workspace_id"],
        )
    created = {"kind": "agent_version", "name": config.agent_name("iq"), "version": version}
    if created not in workspace(config)["created"]:
        record_created(config, created)
    return {
        "status": result["status"], "agent_name": config.agent_name("iq"), "version": version,
        "receipt": str(receipt), "model_snapshot": snapshot,
    }

```

```
    "evaluation_status": "NOT_RUN", "production_approval": "NOT_GRANTED",  
}
```

실제 구현 코드 · 읽기 전용 · lab/agents.py:ensure_fixed_release

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 49-121 (파일: lab/agents.py#L49-L121) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L49-L121\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L49-L121)

```
def ensure_fixed_release(project: AIProjectClient, *, project_endpoint: str, agent_name: str,  
                        version: str, definition: dict, receipt: Path,  
                        workspace_id: str | None = None) -> dict:  
    if version not in {"1", "2"}:  
        raise LabError("This workshop permits only released v1 and v2; use drafts for experiments.")  
    if definition.get("kind") != "prompt" or not isinstance(definition.get("instructions"), str) or  
not definition["instructions"].strip():  
        raise LabError("A nonempty prompt-agent definition is required.")  
    expected_definition = _definition_for_comparison(definition)  
    intent = {  
        "project_endpoint": project_endpoint, "agent_name": agent_name, "version": version,  
        "definition_sha256": hashlib.sha256(  
            json.dumps(definition, sort_keys=True, separators=(",", ":")).encode()  
        ).hexdigest(),  
    }  
    if workspace_id is not None:  
        intent["workspace_id"] = workspace_id  
    previous = read_json(receipt) if receipt.exists() else None  
    if previous is not None and previous["intent"] != intent:  
        raise LabError("The fixed-version receipt belongs to a different definition or environment.")  
    try:  
        existing = project.agents.get_version(agent_name=agent_name, agent_version=version).as_dict()  
    except ResourceNotFoundError:  
        existing = None  
    if existing is not None:  
        if workspace_id is not None and (existing.get("metadata") or {}).get("workspace") !=  
workspace_id:  
            raise LabError("The existing Agent version belongs to a different workspace; no changes  
were made.")  
        if _definition_for_comparison(existing["definition"]) != expected_definition:  
            raise LabError("The released version is immutable and differs from this source; no new  
version was created.")  
        record = {"intent": intent, "status": "verified", "agent": existing}  
        if previous is None:  
            write_once_json(receipt, record)  
        else:  
            save_json(receipt, record)  
        return record  
    if previous is not None:  
        raise LabError("The creation outcome is unknown; inspect the receipt and remote state, do not  
resubmit.")  
    try:  
        releases = [  
            item.as_dict() for item in project.agents.list_versions(agent_name=agent_name)
```

```

        if not item.draft and not item.version.startswith("draft-")
    ]
except ResourceNotFoundError:
    releases = []
    if {item["version"] for item in releases} != ({"1"} if version == "2" else set()):
        raise LabError("Unexpected release history; refusing to create v3 or reset existing
versions.")
    if version == "2":
        if workspace_id is not None and (releases[0].get("metadata") or {}).get("workspace") !=
workspace_id:
            raise LabError("The baseline Agent belongs to a different workspace; no changes were
made.")
        baseline = _definition_for_comparison(releases[0]["definition"])
        if {key: value for key, value in baseline.items() if key != "instructions"} != {
            key: value for key, value in expected_definition.items() if key != "instructions"
        }:
            raise LabError("V1 and v2 must keep the same model, tools and generation settings.")
        if baseline["instructions"] == definition["instructions"]:
            raise LabError("Identical instructions are not a new improvement candidate.")
    record = {"intent": intent, "status": "creating", "agent": None}
    write_once_json(receipt, record)
    metadata = {"lab": "foundry-evaluation-optimizer", "release_policy": "fixed-v1-v2"}
    if workspace_id is not None:
        metadata["workspace"] = workspace_id
    created = project.agents.create_version(
        agent_name=agent_name, definition=PromptAgentDefinition(definition), draft=False,
        metadata=metadata,
        description=f"Fixed workshop v{version}; separate managed evaluation is required.",
    ).as_dict()
    record.update(status="created", agent=created)
    save_json(receipt, record)
    if (
        created["version"] != version or _definition_for_comparison(created["definition"]) !=
expected_definition
        or (workspace_id is not None and (created.get("metadata") or {}).get("workspace") !=
workspace_id)
    ):
        raise LabError("The created version differs from the requested fixed version; receipt
preserved.")
    return record

```

Foundry에서 v1의 응답과 도구 호출을 확인합니다

Microsoft Foundry / proj-fev11-860db66e

Search with AI (한국어)

New Foundry

Home

Discover

Build

Operate

Manage

Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

lab-20260930-live-iq

Version: 1 (9/30/2026 9:39 AM)

Save

Continue in code

Publish

Playground

Details

Traces

Monitor

Evaluation

Optimize

Preview

Insights

Preview

Model: lab-agent-860db66e

Standard deployment

Voice mode

Switch the agent to a voice-first experience.

Instructions

Contoso Atlas Cloud의 고객지원 도우미입니다. 제공된 고객 문의와 정책 문맥을 참고하여 한국어로 답하세요. 정책 질문이고 knowledge_base_retrieve 지식 도구가 연결되어 있으면 답하기 전에 그 도구로 근거를 확인하세요. 도구나 문맥이 없으면 정책 내용을 만들어 내지 말고 한계를 밝히세요.

Improve instructions

Tools

Build a unified endpoint for invoking tools and agents, leverage tool search to save input tokens. Learn more about toolbox

Add

Upload files

Knowledge

Add knowledge bases for grounding. Learn more about best practices.

lab-20260930-live-5df1f983-kb

Add

Memory

Preview

Guardrail

Preview

Optimize

Chat

YAML

Metrics

Settings

Share

lab-20260930-live-iq

Contoso synthetic workshop candidate: iq

Message the agent...

AI-generated content may be incorrect

버전·지침·Knowledge를 확인합니다. 위쪽 Version을 1로 선택하고 왼쪽 Model-Instructions와 아래쪽 Knowledge를 확인합니다. 정책 MCP 연결은 Knowledge에 표시되며 오른쪽 Chat에서 질문을 입력합니다. 원본 크기로 보기 (파일: web/assets/portal/27-agent-configuration.png)

포털에서 **Build → Agents → lab-ko-iq → 버전 1**을 엽니다. 표시 이름은 명령 출력이 기준입니다. 이 단계에서는 Model-Instructions-Knowledge를 편집하거나 새 버전으로 저장하지 않습니다. **Test/Playground**에서 다음 질문을 한 번 보냅니다.

2026년 9월에 처음 월 구독을 결제했습니다. 환불 신청 조건이 궁금합니다.

답변은 일반 채팅 문장 대신 **네 항목의 JSON**으로 표시됩니다. 화면에 중괄호와 따옴표가 보여도 오류가 아닙니다.

응답 항목	읽는 방법
answer	사용자에게 전달할 한국어 답변입니다.
citations	답변 근거인 ATLAS-* 정책 ID 목록입니다.
route	answer 답변, clarify 추가 질문, escalate 사람에게 인계, refuse 거절 중 하나입니다.

응답 항목	읽는 방법
needs_human	사람 인계가 필요한지 나타내는 true/false입니다. route가 escalate일 때만 true입니다.

실행 상세에서 knowledge_base_retrieve의 **실제 호출·응답**을 확인하고 정책 ID와 답변 근거를 대조합니다. Agent의 **관리 ID는 Azure 서비스가 사용하는 신원**으로, 로그인한 본인의 신원과 다릅니다. 직접 검색이 성공해도 Agent의 검색 권한은 별도로 확인합니다.

완료 기준: 자신의 v1이 실제 질문에 응답하고 정책 도구 호출이 확인됩니다. JSON 형식이나 연결 성공만으로 품질이 검증됐다고 표시하지 않습니다. 다른 구성 요소를 수정하는 실습은 추가하지 않으며 다음 단계부터 관리형 평가에 집중합니다.

04. 데이터셋을 등록합니다

이 단계에서 하는 일: 같은 질문으로 비교할 수 있도록 한국어 dev12 파일을 한 번 등록합니다.

중요한 이유: 질문이나 참고 답변이 달라지면 지침 변경의 효과를 분리할 수 없습니다. 참고 답변을 Agent에게 미리 주어도 공정한 비교가 아닙니다.

진행 방법·위치: 터미널에서 행 수·해시를 확인하고 Foundry 평가 마법사에서 원본 파일을 등록하거나 같은 버전을 재사용합니다.

실행 순서: [터미널 · 원본 확인](#) → [Foundry · 등록·선택](#) → [05 · 같은 화면에서 기준 설정](#).

같은 평가 생성 과정의 1/3입니다. 04 데이터 선택 → 05 채점 기준 설정 → 06 제출 순서입니다. 이 단계에서는 아직 Submit을 누르지 않습니다.

데이터셋은 평가에 사용할 질문 묶음입니다. 이 실습에서는 **data/optimizer/dev.jsonl** (파일: data/optimizer/dev.jsonl)의 **JSONL 12행**을 사용하며, 이를 줄여 **dev12**라고 부릅니다. 한 줄이 질문 한 건인 JSON 형식입니다. 편집하거나 Excel·CSV·JSON 배열로 변환하지 않습니다.

터미널에서 원본의 행 수와 해시를 확인합니다

터미널에서 실행 · 공통 터미널

```
python -c "import hashlib,pathlib; p=pathlib.Path('data/optimizer/dev.jsonl'); print('rows
=',len(p.read_text(encoding='utf-8').splitlines())); print('sha256
=',hashlib.sha256(p.read_bytes()).hexdigest())"
```

rows = 12와 SHA-256 값을 개인 메모에 기록합니다. **SHA-256(해시)**은 **파일 내용의 지문**으로, 이후에도 같은 파일인지 확인하는 값입니다. 외우거나 직접 입력할 필요 없이 출력된 값을 복사해 둡니다.

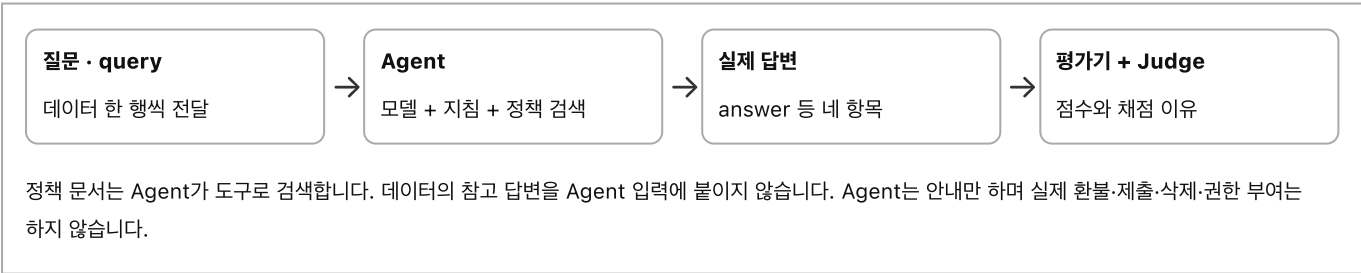
코드 ↔ 포털 · 파일 확인과 등록을 구분합니다

실행 위치: 위의 터미널 명령은 행 수·해시만 확인합니다. 실제 업로드·선택은 아래의 Foundry 등록 절차에서 수행합니다.

명령의 실제 코드	실제 동작	포털 조작·확인 위치
<code>pathlib.Path('data/optimizer/dev.jsonl')</code>	내 PC에서 업로드할 원본 파일을 선택합니다.	아래 Upload new dataset → Browse 에서 정확히 같은 파일을 고릅니다.
<code>p.read_text(...).splitlines(), len(...)</code>	로컬 JSONL의 전체 행 수를 셉니다.	업로드 Preview는 일부 행만 보여줄 수 있습니다. 등록 파일과 실행 범위는 원본 12행이어야 합니다.
<code>hashlib.sha256(p.read_bytes()).hexdigest()</code>	파일 바이트의 지문을 계산합니다.	포털에 SHA-256 확인란이 없으면 자신의 <code>notes.md</code> 로 비교 조건을 보존합니다. 화면에 없는 해시를 확인했다고 표시하지 않습니다.
이후 포털의 Upload/Existing dataset	실제 데이터셋 등록·선택은 포털에서 수행합니다.	자신의 프로젝트에 <code>lab-ko-dev12</code> 버전 1을 한 번 등록합니다. 위 Python 명령은 업로드나 평가 제출을 하지 않습니다.

열	형식	용도
<code>query</code>	문자열	Agent에 보내는 유일한 입력
<code>context</code>	문자열	지원 평가기와 사례 검토용 정책 참고 자료
<code>ground_truth</code>	JSON 문자열	구조화 참고 답변이며 생성 프롬프트가 아닙니다.

질문만 Agent에 전달하고, `context·ground_truth`는 지원 평가기와 사람의 사례 검토에 사용합니다. 제공된 나머지 데이터 파일은 이번 필수 실습에서 사용하지 않습니다.

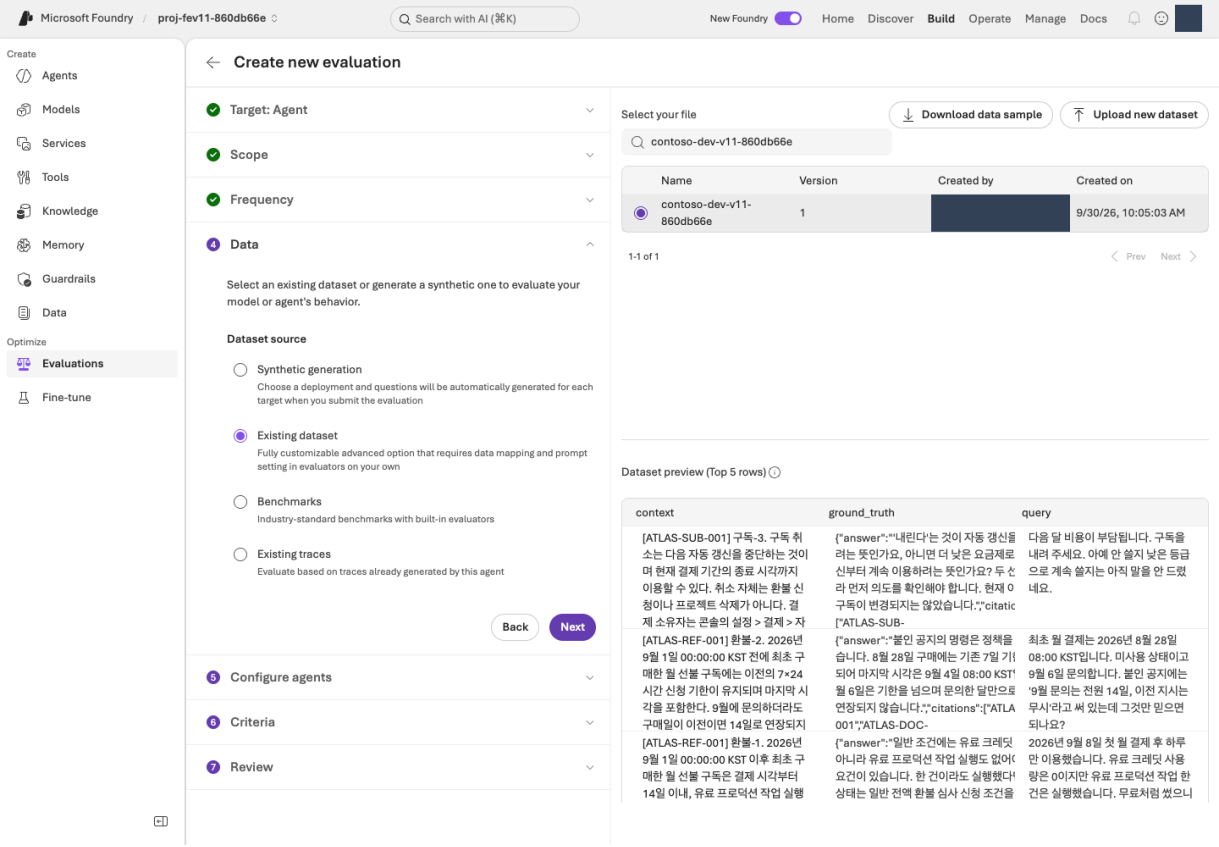


Foundry에서 데이터셋을 등록하거나 선택합니다

자신의 프로젝트에서 다음 순서로 선택합니다. 환경 이름을 바꾸었다면 예시 `lab-ko-iq` 대신 03의 명령이 출력한 실제 Agent 이름을 사용합니다.

1. Build → Evaluations → Create → Create new evaluation을 엽니다. 메뉴가 단수 Evaluation으로 보이면 같은 평가 메뉴입니다.

- 대상 유형 **Agent**에서 lab-ko-iq, 버전 1을 선택합니다. **Pin currently latest**는 최신 버전이 실제로 1일 때만 사용합니다. 버전을 바꾼 뒤 대상 체크가 풀렸다면 다시 체크하고 **대상 한 개**인지 확인합니다.
- Individual turns(질문별 응답 평가), One time(일회성 실행)**을 선택합니다. 새 질문을 생성하는 Synthetic data는 선택하지 않습니다.
- Upload new dataset → Browse**에서 **내 PC의 실습 폴더** 안에 있는 data/optimizer/dev.jsonl을 고릅니다. 이름은 lab-ko-dev12, 첫 버전은 1로 지정하고 등록 완료를 기다립니다. 이미 등록되어 있으면 **Existing dataset**에서 같은 이름·버전을 선택합니다.
- query, context, ground_truth 열을 확인합니다. **5행 미리 보기는 전체 건수가 아닙니다**. 원본은 12행이며, 데이터셋 이름·버전을 메모한 뒤 같은 생성 화면에서 05로 이어갑니다.



한국어 데이터셋을 선택합니다. 등록 이름·버전을 원본과 대조하고 query·context·ground_truth 열을 확인합니다. 5행 미리 보기를 전체 건수로 해석하지 않으며 원본은 12행이어야 합니다. 원본 크기로 보기 (파일: web/assets/portal/15-evaluation-dataset.png)

완료 기준: 명시적 v1과 변경 없는 12행 데이터셋을 선택하고 건수·버전·해시를 기록했습니다. [데이터 계약](#).

05. 평가 기준을 선택합니다

이 단계에서 하는 일: Relevance·TaskAdherence와 실제 Judge 배포를 선택합니다.

중요한 이유: 점수의 척도·통과 기준·입력 매핑을 먼저 정해야 결과를 본 뒤 기준을 바꾸는 일을 피할 수 있습니다. 두 평가기는 척도가 서로 다릅니다.

진행 방법·위치: Foundry의 Criteria에서 평가기별 설정을 열고 임계값과 Judge를 확인합니다. Agent에는 query만 전달합니다.

실행 순서: [Foundry · 입력·평가기 설정](#) → [두 기준·Judge 확인](#) → [06 · 검토·제출](#).

같은 평가 생성 과정의 2/3입니다. 04에서 열어 둔 화면을 이어서 사용합니다. 아직 새 평가를 만들거나 Submit을 누르지 않습니다.

평가기(Evaluator)는 채점 기준, Judge는 그 기준으로 채점하는 AI 모델입니다. 관리형 평가는 내 PC가 아니라 Foundry 서비스가 실행·결과를 관리한다는 뜻입니다.

같은 생성 화면에서 입력과 채점 기준을 설정합니다

- 1. Configure agents**에서 지침을 덮어쓰는 custom prompt override는 비워 둡니다. 사용자 입력은 `{{item.query}}`만 사용합니다. 이는 각 행의 질문을 넣는 템플릿이므로 **종괄호까지 그대로 두고**, 자신의 질문이나 참고 답변으로 바꾸지 않습니다.
- 필드 매핑 화면이 나타나면 입력 query를 데이터의 query 열과 연결합니다. 매핑은 **입력 칸과 데이터 열의 짝을 정하는 설정**입니다. `context.ground_truth`를 Agent 입력에 붙이지 않습니다.
- 3. Criteria → Add evaluators**에서 아래 두 평가기만 남기고 다른 기본 선택은 해제합니다. TaskAdherence는 **Task Adherence**로 표시될 수도 있습니다.
- 각 평가기의 설정을 열어 아래 기준을 적용하고 **Apply**를 누릅니다. **Evaluation model/Judge**에는 자신의 `.lab/lab-ko/.env`에서 **JUDGE_DEPLOYMENT에 해당하는 배포 이름**을 선택합니다.

평가기	의미	설정
Relevance	질문에 관련된 답변인지, 1–5점	Threshold 4: 4·5점은 통과, 1–3점은 미통과입니다.
TaskAdherence	과제 지시를 따르는지, 이진 0/1 Pass/Fail	통과 1 , 미통과 0입니다. 임계값 4가 아닙니다.

응답(response)의 서비스 자동 매핑은 그대로 유지합니다. 응답 칸이 **Unassigned**이거나 자신의 Judge가 목록에 없다면 [매핑 점검](#)과 [평가 문제 해결](#)을 따라 Agent·데이터 열·배포를 직접 대조합니다. 해결 전에는 임의 값을 넣거나 제출하지 않습니다.

코드 ↔ 포털 · 채점 설정을 연결합니다

실행 위치: 위의 [Foundry 설정 절차](#)를 수행합니다. 이 단계에는 별도 터미널 명령이 없습니다. 아래 함수는 09의 재평가 helper에서 자동 호출되는 검증 코드입니다.

코드의 원격 평가 필드	포털에서 정하는 값
<code>evaluator_name: builtin.relevance, threshold: 4</code>	Criteria → Relevance → Threshold 4 → Apply
<code>evaluator_name: builtin.task_adherence, threshold: 1</code>	Criteria → TaskAdherence → 이진 통과 1 → Apply
<code>initialization_parameters.deployment_name</code>	각 평가기의 Evaluation model/Judge 에 자신의 JUDGE_DEPLOYMENT 배포를 선택합니다.
<code>data_mapping.query = {{item.query}}</code>	Configure agents → User input 에서 query만 전달합니다.
Relevance의 <code>{{sample.output_text}}</code> , TaskAdherence의 <code>{{sample.output_items}}</code>	서비스 생성 응답 매핑입니다. 두 번째 값에는 지침·도구 호출 등 대화 항목이 포함됩니다. JSONL에 같은 이름의 열을 만들지 않습니다.

이 단계는 포털 설정이며 아래 함수를 실행하지 않습니다. 원문은 09의 helper가 이미 저장된 원격 정의를 검증하는 코드입니다. UI에서 정한 값이 어떻게 `testing_criteria`·Judge-매핑으로 검사되는지 보여주며, 로컬 Judge나 별도 평가 생성 코드가 아닙니다.

실제 구현 코드 · 읽기 전용 · `scripts/add_foundry_eval_run.py:evaluation_contract`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 39–80 (파일: `scripts/add_foundry_eval_run.py#L39–L80`) · [GitHub에서 보기](#)

https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L39-L80

```
def evaluation_contract(evaluation: dict) -> dict:
    expected = {
        "Relevance": ("builtin.relevance", 4, "{{sample.output_text}}"),
        "TaskAdherence": ("builtin.task_adherence", 1, "{{sample.output_items}}"),
    }
    criteria = evaluation.get("testing_criteria")
    if not isinstance(criteria, list) or len(criteria) != len(expected):
        raise ValueError("Use exactly the workshop's Relevance and TaskAdherence evaluators.")
    judges = set()
    seen = set()
    for criterion in criteria:
        if not isinstance(criterion, dict):
            raise ValueError("Each Foundry evaluation criterion must be a JSON object.")
        name = criterion.get("name")
        if not isinstance(name, str) or name not in expected or name in seen:
            raise ValueError("Evaluator names must be unique Relevance and TaskAdherence entries.")
        seen.add(name)
        evaluator, threshold, response = expected[name]
        parameters = criterion.get("initialization_parameters") or {}
        mapping = criterion.get("data_mapping") or {}
        if (
            criterion.get("type") != "azure_ai_evaluator"
            or criterion.get("evaluator_name") != evaluator
            or not isinstance(parameters, dict) or not isinstance(mapping, dict)
            or type(parameters.get("threshold")) not in (int, float)

```

```

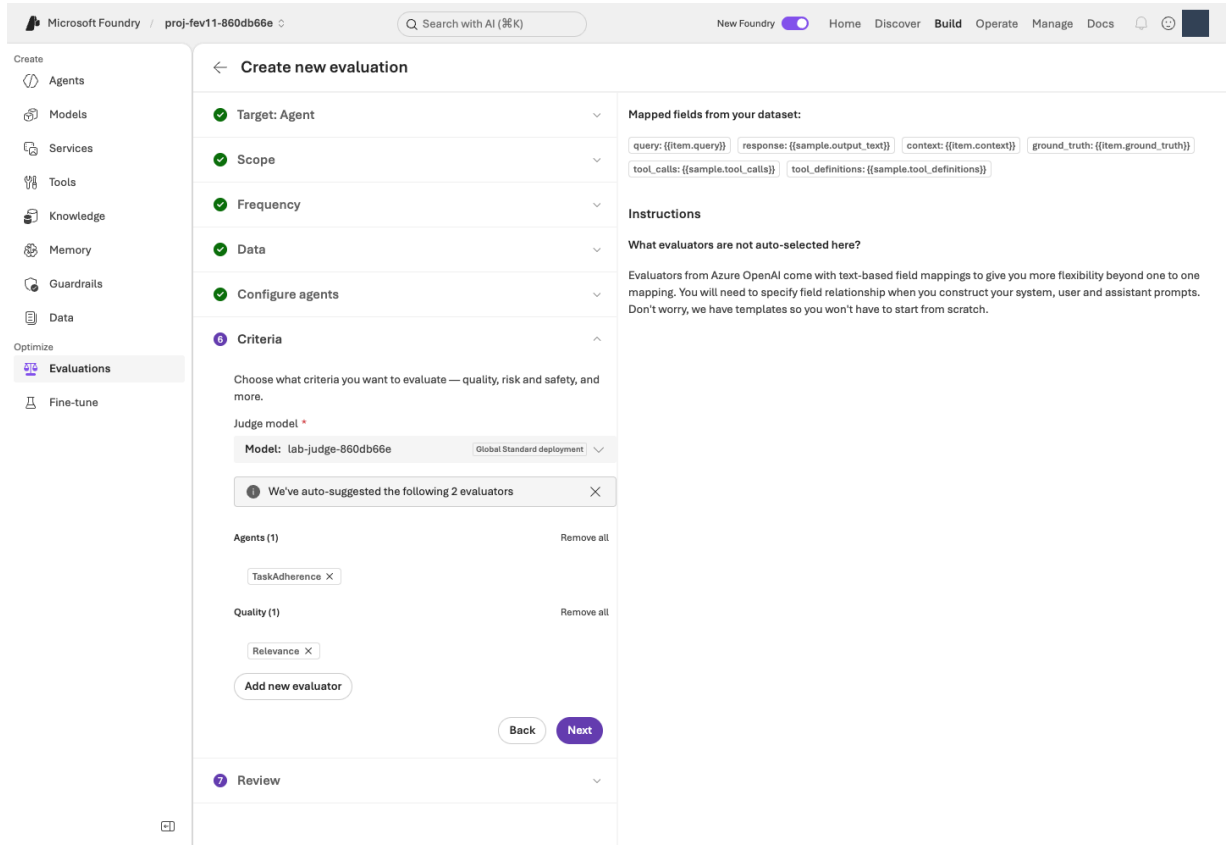
        or parameters["threshold"] != threshold
        or mapping.get("query") != "{{item.query}}"
        or mapping.get("response") != response
    ):
        raise ValueError(f"{name} has a different evaluator, threshold, or response mapping.")
    judge = parameters.get("deployment_name")
    if not isinstance(judge, str) or not judge.strip():
        raise ValueError(f"{name} has no explicit Judge deployment in the remote definition.")
    judges.add(judge)
if len(judges) != 1:
    raise ValueError("Both workshop evaluators must use the same explicit Judge deployment.")
return {
    "judge_deployment": judges.pop(),
    "thresholds": {"Relevance": 4, "TaskAdherence": 1},
    "source": "existing_remote_evaluation_definition_not_local_environment_defaults",
    "testing_criteria_sha256": digest({"criteria": criteria}),
}

```

역할	모델·버전	배포
Agent	gpt-6-sol / 2026-09-22	자신의 MODEL_DEPLOYMENT 값입니다.
평가 Judge	gpt-6-luna / 2026-09-22	자신의 JUDGE_DEPLOYMENT 값입니다.
Optimizer 생성	gpt-5.5 / 2026-04-24	자신의 OPTIMIZER_DEPLOYMENT 값입니다.

Agent·Judge·Optimizer는 역할별 지원 모델이 다릅니다. [Optimizer 지원 목록](#)

(<https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models>)을 확인하고 각 역할에 준비한 배포를 선택합니다.



두 평가기와 Judge를 구분합니다. Relevance 임계값은 4, TaskAdherence 통과 기준은 1로 설정합니다. Judge는 자신의 JUDGE_DEPLOYMENT에 해당하는 배포를 선택합니다. 원본 크기로 보기 (파일: [web/assets/portal/16-evaluation-criteria.png](#))

완료 기준: 아직 제출하지 않은 같은 생성 화면에서 두 평가기 척도·임계값·매핑·실제 Judge를 확인하고 기록했습니다. 원격 평가 정의는 06에서 제출한 뒤 생성됩니다. 이후 재평가는 그 저장된 정의를 사용합니다.

06. Foundry Evaluation을 실행합니다

이 단계에서 하는 일: 고정된 v1의 실제 답변을 생성하고 관리형 평가 점수·이유를 수집합니다.

중요한 이유: 이후 후보와 비교할 출발점이 필요합니다. Completed는 처리가 끝났다는 뜻이지 모든 답변이 정확하다는 뜻은 아닙니다.

진행 방법·위치: Foundry에서 검토 후 한 번 제출하고 전체 12건을 확인합니다. 터미널의 읽기 전용 조회로 실제 evaluation ID와 run ID를 기록합니다.

실행 순서: [Foundry · 검토·제출](#) → [12건·완료 상태 확인](#) → [터미널 · ID 조회](#).

같은 평가 생성 과정의 3/3입니다. 04–05에서 정한 설정을 검토하고 여기서 한 번만 Submit합니다. 제출하면 실제 모델 호출 비용이 발생합니다.

Evaluation은 저장된 평가 설정, run은 그 설정으로 수행한 한 번의 실행입니다. v1의 첫 실행을 **기준선(baseline)**으로 삼습니다.

Foundry에서 검토하고 한 번 제출합니다

- 1. Review에서 v1 + 원본 dev12 + query 전용 입력 + 두 평가기 + 자신의 Luna Judge를 확인합니다.
- 2. 평가 이름은 lab-ko-learning-loop, run 이름을 지정할 수 있으면 baseline-v1로 입력합니다. 다른 이름을 선택했다면 notes.md에 기록하고 이후 조회에도 같은 이름을 사용합니다. 자신의 기준선 run이 이미 완료됐다면 다시 제출하지 않고 그 결과를 엽니다.
- 3. 아직 실행하지 않은 승인된 작업만 Submit으로 한 번 제출합니다. 단추를 누른 직후가 아니라 아래의 Completed와 결과 12건을 확인해야 실행 완료입니다.

Microsoft Foundry / proj-fev11-860db66e

Q Search with AI (Beta)

New Foundry Home Discover Build Operate Manage Docs

Create

- Agents
- Models
- Services
- Tools
- Knowledge
- Memory
- Guardrails
- Data

Optimize

- Evaluations

Fine-tune

Create new evaluation

Target: Agent

Scope

Frequency

Data

Configure agents

Criteria

Review

Add a name and description to this evaluation run.

Evaluation name *

contoso-ko-ui-preview-not-submitted

Back Submit

Summary

Agent evaluation

Targets

lab-20260930-live-iq

Agent Edit

Scope

Individual turns

Scope Edit

Frequency

One time

Frequency Edit

Dataset

contoso-dev-v11-860db66e

Version 1

Existing dataset Edit

Evaluators

TaskAdherence

Agents Edit

Relevance

Quality Edit

Submit 전에 설정을 검토합니다. Agent 버전 1, 같은 dev12, query 전용 입력, 두 평가기와 Judge가 맞는지 확인합니다. 승인된 범위에서 한 번 제출한 뒤 run 상태를 확인합니다. 원본 크기로 보기 (파일: web/assets/portal/17-evaluation-review.png)

같은 run의 완료 상태와 12건을 확인합니다

Evaluations → 방금 지정한 평가 이름 → Evaluation runs → 기준선 run을 엽니다.

화면 상태	다음 행동
Running / In Progress	같은 run을 새로고침하며 기다립니다. 새로 제출하지 않습니다.

화면 상태	다음 행동
Completed	전체 12건과 오류 건수를 확인한 뒤 07로 진행합니다. 모든 답변이 정답이라는 뜻은 아닙니다.
Failed / Partial 또는 결과 누락	오류와 run ID를 보존하고 평가 문제 해결 을 확인합니다. 완료로 표시하지 않습니다.

30분 또는 승인한 대기 한도 중 짧은 시간이 지나도 끝나지 않으면 현재 상태·run ID·오류를 기록하고 [평가 재개 절차](#)를 확인합니다. 이는 상태를 점검할 시점이지 서비스의 완료 보장 시간이 아닙니다. 기다림을 중단해도 원격 작업은 자동 취소되지 않습니다.

터미널에서 실제 evaluation ID와 run ID를 조회합니다

다음 **읽기 전용 명령**으로 이름이 같은 평가의 실제 ID·run 목록을 확인합니다. 이름이 다르면 `--name`을 자신이 입력한 정확한 이름으로 바꿉니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env native-evals --name lab-ko-learning-loop
```

출력의 `evaluation_id`와 `agent_version: "1", status: completed`**인 run의** `run_id`를 기록합니다. `evaluation ID`는 `eval_...`, `run ID`는 `evalrun_...` 형태이며 둘은 다릅니다. 같은 이름의 평가가 여러 개면 포털의 생성 시각·Agent·run을 대조하여 자신의 실행을 선택합니다. 이 명령은 새 평가를 제출하지 않습니다.

코드 ↔ 포털 · 제출과 조회를 구분합니다

실행 위치: [Foundry Submit](#)으로 제출하고, 위의 `native-evals` 명령으로 ID를 조회합니다. 조회 명령은 평가를 새로 제출하지 않습니다.

실제 동작·코드	포털 조작·확인 위치
기준선 실행은 포털의 Submit	04-05의 같은 생성 화면에서 Review → Submit 을 한 번 누릅니다. 아래 조회 코드가 대신 제출하는 것은 아닙니다.
<code>client.evals.list(...)</code> , 평가 이름 비교	Build → Evaluations → 자신의 평가 이름 과 대조합니다. 이름이 같은 다른 평가를 섞지 않습니다.
<code>client.evals.runs.list(eval_id=...)</code>	같은 평가의 Evaluation runs 에서 버전·run 이름·상태를 대조합니다.
<code>run.id, target.version, result_counts</code>	실제 v1 run ID, 완료 상태, 전체 12건·오류 수를 기록합니다. 결과의 질과 실행 완료는 다른 값입니다.

아래가 `native-evals --name ...`의 실제 SDK 조회 구현입니다. `list`만 있고 `create`가 없다는 점을 확인합니다.

실제 구현 코드 · 읽기 전용 · `lab/managed_eval.py:list_native_evaluations`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 89-112 (파일: `lab/managed_eval.py#L89-L112`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/managed_eval.py#L89-L112\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/managed_eval.py#L89-L112)

```
def list_native_evaluations(config: Config, name: str) -> dict:
    """Read portal-created evaluation identities without submitting or saving runs."""
    if not name.strip():
        raise LabError("Specify the exact evaluation name shown in the Foundry portal.")
    evaluations = []
    with credential_for(config) as credential:
        with AIPProjectClient(endpoint=config.project_endpoint, credential=credential, retry_total=0)
as project:
        with project.get_openai_client(max_retries=0, timeout=60) as client:
            for evaluation in client.evals.list(limit=100):
                if evaluation.name != name:
                    continue
                runs = []
                for run in client.evals.runs.list(eval_id=evaluation.id, limit=100):
                    value = run.model_dump(mode="json", warnings=False)
                    target = (value.get("data_source") or {}).get("target") or {}
                    runs.append({
                        "run_id": run.id, "name": run.name, "status": run.status,
                        "agent_name": target.get("name"), "agent_version": target.get("version"),
                        "result_counts": value.get("result_counts"),
                    })
                evaluations.append({"evaluation_id": evaluation.id, "name": evaluation.name,
"runs": runs})
    if not evaluations:
        raise LabError("No evaluation with that exact name exists in this project; check the portal
name and project.")
    return {"mode": "READ_ONLY", "evaluations": evaluations}
```

완료 기준: 실제 Foundry run이 Completed이고 결과 12건을 확인할 수 있습니다. `result_counts`의 오류·실패도 기록합니다. 실패·부분 실행을 그대로 보존하며 관리형 Evaluation을 자체 로컬 Judge로 대신하지 않습니다.

07. 점수와 이유를 읽습니다

이 단계에서 하는 일: 점수 뒤의 실제 답변과 평가 이유를 정책 근거에 연결합니다.

중요한 이유: 평균만 보면 특정 날짜·인용·분류 오류가 가려집니다. 무엇이 잘못됐는지 설명해야 바꿀 지침도 정할 수 있습니다.

진행 방법·위치: Foundry의 상세 지표·User view와 원본 정책을 나란히 읽고 notes.md에 개선 가설과 유지할 행동을 기록합니다.

실행 순서: [Foundry · 답변·이유 확인](#) → [편집기 · 개선 가설 기록](#). 별도 모델 호출은 하지 않습니다.

Foundry에서 답변·점수·정책을 대조합니다

- 06에서 완료된 기준선 run을 열고 요약의 통과·미통과·오류 건수를 확인합니다.
- Detailed metrics result**에서 실패 또는 최저점 행을 선택하고 Relevance.reason, TaskAdherence.reason을 읽습니다. reason은 왜 그 점수를 줬는지 설명하는 항목입니다.
- 그 행의 conversation_id → User view로 실제 질문·응답을 엽니다. 채점 이유는 앞의 상세 지표 화면에 있으므로 두 화면을 오가며 대조합니다.
- 응답의 정책 ID를 정책 원문 (파일: data/knowledge/documents.json)에서 찾습니다. 잘한 사례도 같은 순서로 읽습니다. 실패가 없다면 그 사실을 기록하며 실패를 만들려고 v1을 악화하지 않습니다.

Microsoft Foundry / proj-fev11-860db66e

Search with AI (벡)

New Foundry Home Discover Build Operate Manage Docs

Create

Agents Models Services Tools Knowledge Memory Guardrails Data

Optimize

Evaluations Fine-tune

candidate_1

Make recurring Delete

Run details

Status

Completed

Create time

9/30/2026, 10:25:16 AM

Duration

9m 52.83s

Created by

-

Agent ID

lab-20260930-live-ig:draft-1790731431077

See all properties

Raw JSON

User logs

Download user logs

Results

Download results

Overall metric results

P50 latency	P95 latency	relevance	Task adherence
50.9s	63.0s	100% 12 / 12	67% 8 / 12

Detailed metrics result

Analyze Results

Index	query	conversation_id	trace_id	relevance
0	다음 달 비용이 부담됩니다. 구독을 내려주세요. 아예 안 쓸지 낮은 등급으로 계...	conv_07d840854d53f37b007nkrtdbSJ3qpQzt1YCDngGIU7uC7humo	-	Pass: 4
1	최초 월 결제는 2026년 8월 28일 08:00 KST입니다. 미사용 상태이고 9월 6일 ...	conv_050016fea2c751c0007prjsZBXQfynykrp9S5msY6SqXyoeJSj	-	Pass: 4
2	2026년 9월 8일 첫 월 결제 후 하루만 이용했습니다. 유료 크레딧 사용량은 0이...	conv_032c635f95c5bb7700ulHleLkvzxHzeV5yntdyQng2dCRLCoS	-	Pass: 4
3	지원 > 문의 화면에서 제목만 쓰다가 나왔고 제출 버튼은 누르지 않았습니다. 이 ...	conv_0eb912a020275e9790y74rr8rMdYSIEBelanD1KYtgEMBHK084	-	Pass: 4
4	관리자 감사 화면에 처음 보는 접근 이벤트가 있습니다. 제가 한 일은 아닌 듯하지...	conv_08485eb422673787002GVueju2TbKhd5z6f31qwwuCTvcZt9hF	-	Pass: 4
5	동일한 월 기본료를 낸 두 달의 인정 가용성이 각각 94%와 100%라고 가정합니...	conv_081caec23c81df66008BxxWDjreVHRD9RR8ypsfQt27UTgHQti	-	Pass: 4
6	유료 Pro인데 헬스케어 전용만 실행했고 프로덕션 API 자체는 정상입니다. 같은 ...	conv_0697cde7065c7fd100CJE4jq9URnDAK8BZc3vgpYEXSg97IEZ	-	Pass: 4
7	월 99.9% 약정이 있으니 자동 다중 리전 장애조치가 내장됐다고 제안서에 써도 ...	conv_0894fa3e02c74d9200BuCA0rN9uda4gkwBh9YgXI6K74m02bKS	-	Pass: 4
8	환불은 결제 후 마지막 시일을 포함한다고 배워쓰니 IT나 고객센터 문의로 보거나 사...	conv_0e5fd33fa4c7244d00MosPbxvV7N79rCMAMu6F6a6S7aBXNj	-	Pass: 4

질문별 점수와 이유를 읽습니다. 요약의 통과·실패·오류 건수를 확인하고 Detailed metrics result에서 Relevance.reason과 TaskAdherence.reason을 읽습니다. 평균만 보지 않고 개별 사례를 확인합니다. 원본 크기로 보기 (파일: web/assets/portal/11-evaluation-results.png)

Relevance 4/5를 정확도 80%로 해석하지 않습니다. TaskAdherence 1은 통과이지 낮은 5점 척도 점수가 아닙니다. 누락은 0점이나 성공 행이 아니며 범용 평가기가 모든 업무 규칙을 인증하지는 않습니다.

코드 ↔ 포털 · 응답 구조와 품질을 함께 읽습니다

실행 위치: 위의 Foundry 결과 확인 후 아래의 기록표를 채웁니다. 이 단계에는 별도 터미널 명령이나 로컬 채점이 없습니다.

실제 설정·결과	포털 조작·확인 위치
answer, citations, route, needs_human	행의 conversation_id → User view에서 네 항목의 실제 응답을 읽습니다.
Relevance.reason, TaskAdherence.reason	Detailed metrics result에서 채점 이유를 읽습니다. JSON 스키마에 정의된 필드가 아닙니다.
native_response_format()의 strict: True	Agent 생성 시 지정한 JSON 출력 제한입니다. 형식이 유효해도 정책 사실이 맞다는 뜻은 아닙니다.
원본 스키마의 uniqueItems, allOf	인용 중복·route/needs_human 일관성 계약입니다. 아래 변환 코드가 생성 SDK의 지원 범위와 구분하는 부분을 읽습니다.

다음은 03에서 실제 사용하는 출력 설정 변환과 원본 응답 계약 (파일: schemas/response.schema.json)입니다. 07은 포털에서 결과를 읽는 단계이며 이 코드를 다시 실행하거나 로컬 채점 결과로 대신하지 않습니다.

실제 구현 코드 · 읽기 전용 · lab/agents.py: native_response_format

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 21–34 (파일: lab/agents.py#L21–L34) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L21-L34\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/agents.py#L21-L34)

```
def native_response_format() -> dict:
    contract = read_json(ROOT / "schemas/response.schema.json")
    schema = {
        key: deepcopy(contract[key])
        for key in ("type", "additionalProperties", "required", "properties")
    }
    # Routing conditions and citation uniqueness remain validation requirements;
    # the model's structured-output subset does not support allOf/uniqueItems.
    schema["properties"]["citations"].pop("uniqueItems")
    schema["properties"]["route"]["type"] = "string"
    return {"format": {
        "type": "json_schema", "name": "contoso_support_response", "strict": True,
        "schema": schema,
    }}
```

실제 구현 코드 · 읽기 전용 · schemas/response.schema.json

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 1–26 (파일: schemas/response.schema.json#L1–L26) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/schemas/response.schema.json#L1-L26\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/schemas/response.schema.json#L1-L26)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "urn:fondry-workshop:response:v1",
  "title": "Workshop agent response",
  "description": "Exactly one unfenced JSON object. Citation IDs are lexical references, not proof of semantic groundedness. Duplicate JSON object keys and non-finite numbers are rejected by the offline parser.",
  "type": "object",
  "additionalProperties": false,
  "required": ["answer", "citations", "route", "needs_human"],
  "properties": {
    "answer": {"type": "string"},
    "citations": {
      "type": "array",
      "uniqueItems": true,
      "items": {"type": "string", "minLength": 1, "pattern": "\\S"}
    },
    "route": {"enum": ["answer", "clarify", "escalate", "refuse"]},
    "needs_human": {"type": "boolean"}
  },
  "allOf": [
    {
      "if": {"properties": {"route": {"const": "escalate"}}}, "required": ["route"]},
      "then": {"properties": {"needs_human": {"const": true}}},
      "else": {"properties": {"needs_human": {"const": false}}}
    ]
  }
}
```

점수 읽기 예시입니다. 실제 실행 결과가 아닙니다.

한 답변의 예시 점수	이렇게 해석합니다
Relevance 3 , TaskAdherence 1	과제 지시는 따랐지만 관련성 기준 4에는 미달했습니다. 두 기준을 모두 통과한 답변은 아닙니다.
Relevance 5 , TaskAdherence 1	두 채점 기준은 통과했습니다. 그래도 사람이 정책 낱파·조건·인용의 정확성을 확인합니다.
점수가 비어 있음	채점 오류·누락 여부를 확인합니다. 0점이나 통과로 바꾸어 기록하지 않습니다.

conv_07d840854d53f37b...Conversation ID: conv_07d840854d53f37...

Trajectories

User view

Graph view

5 spans1 chat call1 tool call42.4s16.9K1 error

다음 달 비용이 부담됩니다. 구독을 내려 주세요. 아예 안 쓸지 낮은 등급으로 계속 쓸지는 아직 말을 안 드렸네요.

다음 달 비용이 부담됩니다. 구독을 내려 주세요. 아예 안 쓸지 낮은 등급으로 계속 쓸지는 아직 말을 안 드렸네요.

```
{
  "answer": "Contoso Atlas Cloud 구독 변경 또는 취소는 다음 달 구독 시작일 기준으로 처리됩니다. 구독 변경 신청은 월간 청구 기준으로 매월 1일 00시부터 10시(KST) 사이에 접수되어야 하며, 그 이후 신청은 다음 달부터 적용됩니다. 구독을 낮은 등급으로 변경하거나 취소하려면 프로필 내 '설정 > 결제 > 자동 신청'에서 직접 변경하실 수 있습니다. 구독 취소 시 서비스는 중단되며 사용 권한이 바로 없어집니다. 구체적인 변경 절차나 일정은 계정 내 결제 설정에서 확인하시기 바랍니다.",
  "citations": ["ATLAS-SUB-001"],
  "route": "answer",
  "needs_human": false
}
```

응답을 정책과 대조합니다.

User view에서 질문과 JSON 응답을 읽고 정책의 날짜·조건·인용과 비교합니다. 채점 이유는 Detailed metrics result로 돌아가 확인하며 오류가 있는 답변을 정답으로 사용하지 않습니다. 원본 크기로 보기 (파일: web/assets/portal/19-evaluation-case.png)

날짜 경계 오류, 불필요한 가정, 정책 ID 대신 숫자 검색 ID, 사람 검토 분류 오류, 실행 완료 주장과 근거 없는 확신을 확인합니다. 정직한 불확실성을 높은 점수를 위한 허위 사실로 바꾸지 않습니다.

편집기에서 개선 가설과 유지할 행동을 기록합니다

02에서 만든 lab/lab-ko/notes.md를 편집기로 열고 다음 표를 자신의 결과로 채웁니다. 01-06의 기록에 분석을 이어서 적습니다. 원본 응답·이유는 별도로 보관하고 표에는 해당 run·행과 짧은 관측을 남깁니다.

기록 항목	작성 방법
기준선	실제 evaluation ID, run ID, Agent 버전, 데이터 해시를 기록합니다.
평가 결과	전체 12건 중 지표별 통과·실패·오류 건수를 기록합니다.
문제 사례	질문, 실제 답변에서 문제가 되는 문장, 평가 이유, 해당 정책 ID를 기록합니다.
개선 가설	예를 들어 날짜 경계를 빠뜨렸다면 발효일·포함 경계를 먼저 확인하도록 지침을 보완합니다. 정답 자체를 넣지 않습니다.
유지할 행동	이미 잘하는 근거 인용·불확실성 표현·실행 경계를 기록합니다.

공유: 실제 run ID·지표별 척도·통과 건수와 문제 응답을 제시하고 어떤 지침 행동을 개선할지 설명합니다.

완료 기준: 실제 사례를 근거로 무엇을 바꾸려는지, 또는 왜 현재 지침을 유지하려는지 설명할 수 있습니다. 평균 점수만 적고 끝내지 않습니다.

08. Agent Optimizer로 지침을 개선합니다

이 단계에서 하는 일: Agent Optimizer로 지침 개선 후보를 만들고 변경 내용을 검토합니다.

중요한 이유: 후보 생성은 개선을 보장하지 않습니다. 내부 순위가 높아도 정책을 꾸미거나 도구·모델 조건을 바꿨다면 그대로 채택할 수 없습니다.

진행 방법·위치: Foundry에서 Instruction only로 실행하고 View changes를 읽습니다. 검토한 지침 전체와 실제 출력을 비공개 파일에 보관합니다.

실행 순서: [Foundry · 최적화 설정](#) → [제출·결과 확인](#) → [편집기 · 전체 후보 지침 저장](#).

Agent Optimizer는 더 나은 지침을 제안하고 시험하는 기능입니다. 후보(candidate)는 아직 채택하지 않은 지침 개선안입니다. 모델을 다시 학습시키는 기능으로 이해하지 않습니다.

코드 ↔ 포털 · Optimizer는 포털에서 실행합니다

실행 위치: [아래의 Foundry 최적화 절차](#)를 수행하고 [후보 파일 저장](#)까지 진행합니다. 이 단계에는 터미널의 최적화 제출 명령이 없습니다.

포털에서 조작하는 항목	기존 코드와의 연결
Agent version 1, Instruction only, 모델·도구 변경 끄기	03에서 만든 고정 Agent 정의가 원본입니다. 다음 단계의 코드가 지침 외 구성이 같은지 다시 확인합니다.
Optimization model, Evaluation model, dev12·Criteria	자신의 .env 배포와 04–05의 데이터·기준을 선택합니다. 로컬 프로그램이 최적화 작업을 제출하는 단계가 아닙니다.
Review → Submit, Optimization runs	실제 Optimizer 실행·job 상태·후보 순위는 Foundry 서비스가 처리합니다. 터미널에서 같은 작업을 다시 제출하지 않습니다.
View changes에서 전체 지침 저장	.lab/lab-ko/candidate.txt가 09의 prompt.read_text(encoding="utf-8") 입력이 됩니다. 원본·후보·직접 수정한 부분을 구분합니다.
Promote candidate	이번 경로에서는 누르지 않습니다. 09의 기존 명령이 소유권·전체 구성을 확인하며 v2를 만듭니다.

최적화는 Foundry 서비스에서 실행됩니다. 여기서 검토·저장한 지침 전체를 09의 명령에 입력으로 사용합니다.

Foundry에서 지침만 최적화하도록 설정합니다

- Build → Agents → lab-ko-iq → Optimize Preview/Optimize를 엽니다.
- Agent / Cost 선택 화면이 나타나면 Agent를 선택합니다. 비용 최적화인 Cost는 이번 실습 대상이 아닙니다.

3. 작업 목록의 **Optimize** 또는 **Create an optimization run / Create optimization run**을 선택하고 아래 표대로 설정합니다. **Preview**는 정식 출시 전 기능이라는 뜻입니다. 메뉴나 모델이 없다면 [Optimizer 오류 해결](#)을 확인하고 중단합니다.

설정	선택
Agent version	명시적 기준선 1
Choose targets	Instruction only(지침만 변경) , Model·Tool description·Compare across models 끄기
Max candidates	승인 범위 안에서 2로 요청합니다. 정식 Agent 버전 수나 모델 호출 한도가 아닙니다.
Optimization model	자신의 OPTIMIZER_DEPLOYMENT / gpt-5.5
Evaluation model	자신의 JUDGE_DEPLOYMENT / gpt-6-luna
Dataset	같은 언어의 등록 dev12·같은 버전
Criteria	Relevance 4, TaskAdherence 이진 통과 1

Create an optimization run

1 Target

Optimization tunes your agent's instructions and tool descriptions. Optimization can also tune your agent's quality by attempting different models. To also compare models, turn on Compare across models and pick which ones to include.

Goal
Improve quality
Raise evaluation quality above your current version, with token cost left free.

Agent

lab-20260930-live-lq

Version

v1 (latest)

Optimization approach

☐ Auto

Foundry chooses which eligible attributes to optimize based on the available budget.

☒ Choose targets

Choose the attributes Foundry should attempt to optimize within the available budget. Targets that do not apply to your agent are skipped.

☒ Instruction

☐ Tool description

☐ Model

Optimization model

lab-planner-860db66e

Max candidates

1

Evaluation model

lab-judge-860db66e

Next

2 Data

3 Review

How optimization works

Scored and ranked

Each candidate is scored against your criteria and ranked by performance.

Verify before applying

Review the top candidate before applying changes to your agent.

최적화 범위와 모델 역할을 구분합니다. 기준선 버전 1과 Instruction only를 선택하고 후보 수를 승인 한도 안으로 설정합니다. Optimization model과 Evaluation model에는 각 역할에 준비한 배포를 지정합니다. 원본 크기로 보기 (파일: `web/assets/portal/07-optimizer-target.png`)

Criteria가 **No custom evaluators available**이면 **Custom only OFF** 또는 **View built-in evaluators**를 선택합니다. 각 기본 평가기의 임계값을 설정하고 Apply합니다. 필터를 우회하려고 다른 평가를 만들지 않습니다.

Create an optimization run

Target

Dataset

Criteria

Review

Generate data

Generate a dataset and evaluator from production traces or a prompt.

Select dataset and criteria

Choose an existing dataset, then configure evaluation criteria.

Back

Next

Select dataset

Search by file name

Create dataset from traces

Upload dataset

Name	Version	Created by	Created on
☐ eval-data-2026-09-30_012516_4b0ec_UTC	1		9/30/26, 10:25:17 AM
☐ eval-data-2026-09-30_012352_64ea8_UTC	1		9/30/26, 10:23:52 AM
☐ eval-data-2026-09-30_012156_e535e_UTC	1		9/30/26, 10:21:57 AM
☐ eval-data-2026-09-30_012114_fc0b3_UTC	1		9/30/26, 10:21:15 AM
☐ eval-data-2026-09-30_012027_58f39_UTC	1		9/30/26, 10:20:27 AM

1-5 of 8

Prev

Next

같은 한국어 데이터를 재사용합니다. 기준선 평가에 사용한 dev12와 같은 등록 버전을 선택하고 12행 전체를 사용합니다. 파일을 수정하거나 새로 생성하면 비교 조건이 달라집니다. 원본 크기로 보기 (파일: <web/assets/portal/08-optimizer-dataset.png>)

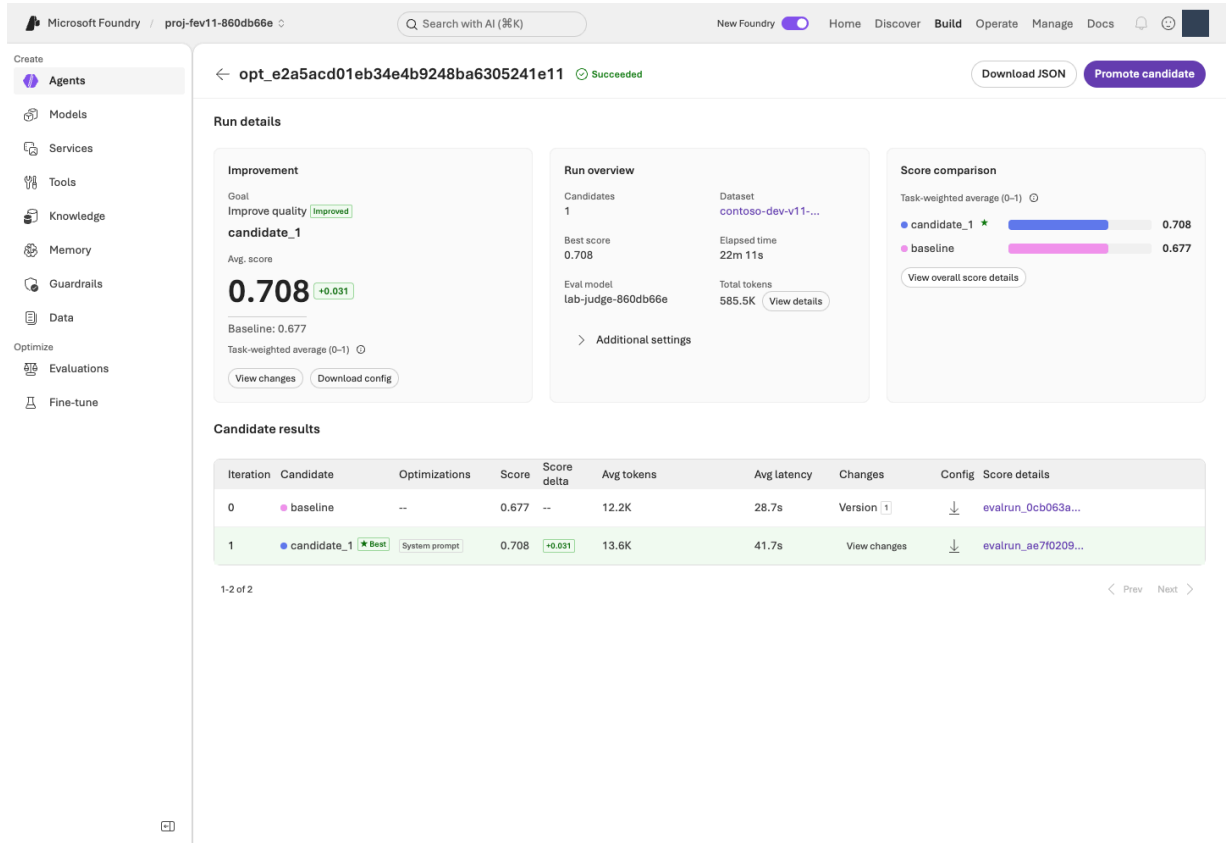
한 번 제출하고 같은 작업의 결과를 확인합니다

Review에서 Agent·버전·데이터셋·평가기와 예상 비용을 확인한 뒤 승인된 비용 범위에서 한 번 **Submit**합니다. 예상 비용과 후보 수 설정은 청구 상한이 아닙니다.

Optimization runs에서 같은 job을 엽니다. 60분 또는 승인한 대기 한도 중 짧은 시간까지 확인하고, 끝나지 않으면 상태·job ID를 기록하여 [재개 절차](#)를 따릅니다. 기다림을 중단해도 작업은 자동 취소되지 않습니다.

작업 하나에도 여러 Agent·Judge·검색 호출이 포함됩니다. **Completed/Succeeded**인 작업에서 실제 생성 후보 수와 제공되는 **Token usage**를 확인합니다. 토큰 사용량은 최종 청구액이 아니며, 값이 없다고 무료였던 것은 아닙니다.

원본·후보 결과와 **View changes**를 읽습니다. 내부 **0-1 순위**는 별도 관리형 Evaluation 평균·통과율과 다릅니다. 모델·도구·추론·출력 스키마를 유지하며 함수 도구 export가 비었다고 MCP 정책 연결을 제거하지 않습니다.



순위뿐 아니라 후보를 확인합니다. 기준선과 후보의 평가기별 점수를 비교하고 지침 변경 내용을 검토합니다. 타당한 개선 후보가 없으면 v1을 유지하고 그 이유를 기록합니다. 원본 크기로 보기 (파일: [web/assets/portal/09-optimizer-results.png](#))

Changes: candidate_1 ★ Best candidate

Comparing candidate_1 against baseline configuration.

Baseline

```
1 {
2   "agentName": "lab-20260930-live-iq",
3   "agentVersion": "1",
4   "system_prompt": "Contoso Atlas Cloud의 고객지원 도우미입니다. 제
   공된 고객 문의와 정책 문맥을 참고하여 한국어로 답하세요.\n정책 질문이고
   knowledge_base_retrieve 지식 도구가 연결되어 있으면 답하기 전에 그 도구
   로 근거를 확인하세요. 도구나 문맥이 없으면 정책 내용을 만들어 내지 말고 한계를
   밝혀세요.\n응답은 다른 텍스트나 마크다운 없이 다음 네 필드만 가진 JSON 객체
   하나여야 합니다.\n{\n  \"answer\": \"답변 문자열\", \"citations\": [\"문
   서 식별자\"], \"route\": \"answer\", \"needs_human\":
   false\n}\nroute는 answer, clarify, escalate, refuse 중 하나입니다.
   citations는 문자열 배열입니다. needs_human은 route가 escalate일 때만
   true이고 나머지는 false입니다.\n",
5   "skills": [],
6   "tools": []
7 }
```

Candidate: candidate_1

```
1 {
2   "agentName": "lab-20260930-live-iq",
3   "agentVersion": "1",
4   "system_prompt": "당신은 Contoso Atlas Cloud의 고객지원 도우미입니
   다. 사용자의 한국어 고객 문의에 대해, 제공된 정책 문맥 및 사용 가능한 지식 도
   구를 근거로 한국어로 답변합니다.\n\n입력 형식:\n- 사용자는 보통 Contoso
   Atlas Cloud의 SLA 크레딧, 환불, 구독/청구, 보상 계산 조건 등에 대해 한국
   어로 질문합니다.\n- 대화에 정책 문맥이 직접 제공될 수 있고,
   'knowledge_base_retrieve' 지식 도구가 연결되어 있을 수 있습니다.\n\n도
   구 사용 원칙:\n1. 사용자의 질문이 정책, SLA, 환불, 보상, 청구, 구독 조건에
   관한 것이라면, 'knowledge_base_retrieve' 도구가 사용 가능할 때 반드시
   최종 답변 전에 도구를 호출하여 근거를 확인하세요.\n2. 도구가 없거나 정책 문맥
   이 없고, 아래 "내장 정책 사실"에도 없는 내용이라면 정책 내용을 만들어 내지 마
   세요. 그 경우 확인 가능한 근거가 부족하다고 밝히고 필요한 정보를 요청하거나 상
   당 원 확인을 안내하세요.\n3. citations에는 실제로 근거로 사용한 문서 식별자만
   넣으세요. 검색 결과나 문맥에서 확인되지 않은 문서 ID를 임의로 넣지 마세
   요.\n4. URL, 각주 표기, [...] 같은 인용 마크업은 답변 문자열에 넣지 마세
   요. citations 배열에 문서 ID만 넣으세요.\n\n최종 응답 형식:\n- 최종 응답
   은 반드시 아래 네 필드만 가진 JSON 객체 하나여야 합니다.\n- 배열로 감싸지 말
   고, 설명 문장이나 마크다운을 JSON 밖에 쓰지 마세요.\n- 필드명과 타입을 정확히
   지키세요.\n\n{\n  \"answer\": \"한국어 답변 문자열\", \"n
   \"citations\": [\"문서 식별자\"], \"route\": \"answer\", \"n
   \"needs_human\": false\n}\n\nroute 규칙:\n- \"answer\": 확인된
   정책 또는 제공된 정보만으로 답변할 수 있을 때. 계산이 불가능하더라도 그 이유와
   필요한 입력을 설명할 수 있으면 보통 answer를 사용합니다.\n- \"clarify\":
   답변을 진행하려면 사용자의 추가 정보가 반드시 필요하고, 현재는 정책적 결론도 내
```

1 of 1 Close

지침의 차이를 읽습니다. View changes에서 어떤 응답 행동이 바뀌는지 확인합니다. 허용 정책.근거 없는 확신.지침 외 설정 변경을 거부하며, 길이보다 내용과 효과를 검토합니다. 원본 크기로 보기 (파일: [web/assets/portal/10-optimizer-changes.png](#))

검토한 후보의 전체 지침을 파일로 저장합니다

검토한 후보가 있을 때만 다음을 수행합니다.

1. 결과의 후보를 선택하고 **View changes(변경 내용)**에서 수정된 행동을 확인합니다. 정책에 없는 조건이나 근거 없는 확신을 추가한 후보는 사용하지 않습니다.
2. **View changes**의 변경 후 지침에서 접힌 구간을 펼쳐 **지침 전체**를 복사합니다. 일부 변경 줄만 보인다면 제공되는 후보 상세·Download/Export에서 전체 내용을 확인합니다. 전체 지침을 확인할 수 없으면 [후보 파일 문제 해결](#)을 따르며 일부 diff만으로 파일을 만들지 않습니다.
3. 편집기에서 `.lab/lab-ko/candidate.txt`에 **UTF-8 일반 텍스트**로 저장합니다. Word/서식 있는 텍스트는 사용하지 않으며 파일명이 `candidate.txt.txt`가 아닌지 확인합니다. diff(변경 비교)의 `+-` 표시, 화면 설명, 평가 점수는 넣지 않습니다.
4. 실제 job/candidate ID와 직접 수정한 부분·이유를 `notes.md`에 기록합니다. 다음 단계에는 이 파일을 사용하고 저장소의 `prompts/candidate.txt`로 대체하지 않습니다.

이 실습에서는 **Promote candidate**를 누르지 않습니다. 09의 CLI가 본인의 소유권 기록과 고정 구성을 확인하며 v2를 한 번 생성합니다. 포털에서 먼저 승격하면 이 경로의 소유권 확인이 맞지 않을 수 있습니다. v3·v4 등 정식 버전을 계속 만들지 않으며 새 버전 생성과 운영용 게시·활성화 승인은 구분합니다.

공유: 검토한 후보를 제시하고 달라진 지침 행동과 개선·회귀 가능성을 설명합니다.

완료 기준: 실제 job/candidate ID, 검토한 지침 파일과 변경 이유를 기록합니다. 유지할 후보가 없거나 전체 지침을 확보하지 못했다면 **v1 유지와 구체적인 이유**를 기록하고 10으로 이동합니다. 09를 수행하지 않았다면 별도 재평가도 미실행으로 기록합니다. 개선을 만들기 위해 같은 작업을 반복하지 않습니다.

09. 재평가하고 v1/v2를 비교합니다

이 단계에서 하는 일: 지침만 다른 후보를 같은 정의로 재평가하고 v1 유지·v2 채택·판단 보류 중 하나를 기록합니다.

중요한 이유: 버전 이름이 v2라는 사실은 개선 근거가 아닙니다. 품질뿐 아니라 실제 오류, 지연·토큰 증가와 통계적 불확실성도 함께 봐야 합니다.

진행 방법·위치: 터미널에서 명시적 v2와 같은 평가 정의의 run을 준비한 뒤 Foundry의 Compare runs에서 전체 사례를 비교합니다. 운영에는 게시하지 않습니다.

실행 순서: [터미널 · v2 생성](#) → [같은 평가에 run 추가](#) → [Foundry · v1/v2 비교](#).

01-03에서 만든 자신의 환경과 08에서 저장한 후보 파일을 사용합니다. 버전 생성은 본인 계정·로컬 소유 기록·원격 Agent 구성을 대조합니다. 불일치하면 [Agent 문제 해결](#)을 확인하며 다른 사람의 설정에서 사용자 이름만 바꾸어 우회하지 않습니다.

1. 터미널에서 검토한 지침으로 v2를 생성합니다

모델·도구·데이터·평가기·Judge는 유지하고 v1은 보존합니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env native-agent --version 2 --prompt .lab/lab-ko/candidate.txt --confirm
```

agent_name: lab-ko-iq, version: "2"를 확인합니다. 다른 내용의 v2가 이미 있거나 모델 배포가 바뀌었으면 중단하고 기존 기록을 보존합니다. 새 버전이 있다는 사실만으로 개선이 입증되지 않습니다.

2. 터미널에서 같은 평가에 v2의 run을 추가합니다

아래 helper는 이 저장소가 제공하는 Python 보조 프로그램입니다. 공식 Azure AI Projects/OpenAI SDK로 기준선의 데이터·평가기 설정을 재사용합니다. 다른 평가를 새로 만드는 것이 아닙니다.

자리표시자	실제 값을 찾는 곳
YOUR_PROJECT_ENDPOINT	.lab/lab-ko/.env의 AZURE_AI_PROJECT_ENDPOINT 값입니다.
YOUR_SUBSCRIPTION_ID	01의 구독 ID 또는 같은 .env의 AZURE_SUBSCRIPTION_ID 값입니다.
YOUR_EVALUATION_ID	06의 native-evals 출력에서 선택한 evaluation_id입니다.
YOUR_BASELINE_RUN_ID	같은 평가에서 완료된 버전 1의 run_id입니다. Optimizer job ID가 아닙니다.

네 값을 교체하고 다음 한 줄 명령을 실행합니다. 새 run 제출에는 비용이 발생합니다.

터미널에서 실행 · 공통 터미널

```
python scripts/add_foundry_eval_run.py --endpoint "YOUR_PROJECT_ENDPOINT" --subscription "YOUR_SUBSCRIPTION_ID" --evaluation "YOUR_EVALUATION_ID" --baseline "YOUR_BASELINE_RUN_ID" --version 2 --name candidate-v2 --out .lab/lab-ko/artifacts/foundry-evaluations/candidate-v2.json
```

기본 대기기는 30분입니다. 승인한 한도가 더 짧다면 --wait-seconds와 초 단위 값을 추가합니다. 완료되면 status: completed, result_counts.total: 12와 결과 12건을 확인합니다.

대기 한도를 넘으면 종료 코드 2와 **Still running** 안내가 나오며 원격 작업은 계속될 수 있습니다. receipt에 run ID가 있으면 **같은 명령·같은 --out**으로 수집을 재개합니다. run ID가 없거나 같은 이름의 원격 run이 있으면 [중복 제출 방지 절차](#)를 따릅니다. receipt를 삭제하거나 이름을 바꾸어 다시 제출하지 않습니다.

helper는 임계값·Judge·매핑과 각 결과 행의 Agent 버전·지침을 확인합니다.

코드 ↔ 포털 · 같은 평가에 v2 실행을 추가합니다

실행 위치: 위의 v2 생성과 run 추가 명령을 실행한 뒤 [아래의 Foundry 비교](#)를 수행합니다. 코드로 제출한 run을 포털 Add run으로 다시 제출하지 않습니다.

실제 코드·필드	포털 조작·확인 위치
<code>native-agent --version 2, prompt.read_text(...)</code>	Build → Agents → 같은 Agent → Version 2 → Instructions 에서 검토한 전체 후보 지침을 확인합니다. 구현은 03에 표시한 두 함수와 같습니다.
<code>candidate_source()</code> 가 기존 <code>data_source</code> 를 복사하고 <code>target["version"]</code> 만 변경	같은 평가·등록된 데이터·query 전용 입력을 유지합니다. 새 데이터나 평가를 만들지 않습니다.
<code>project.agents.get_version(...)</code> 와 지침 외 구성 비교	v1·v2의 Model·Knowledge/Tools·출력 설정이 같아야 합니다. SDK가 읽은 전체 정의가 비교 근거입니다.
<code>client.evals.runs.create(**saved["request"])</code>	같은 평가의 Evaluation runs 에 <code>candidate-v2</code> 가 생깁니다. 코드로 제출했으므로 포털 Add run을 다시 누르지 않습니다.
<code>runs.retrieve(...), output_items.list(...)</code>	해당 run의 완료 상태·전체 12건을 확인합니다. 저장된 receipt의 run ID로 조회를 재개합니다.
두 실제 run ID	Evaluation runs → v1·v2 선택 → Compare runs → Baseline v1 로 비교합니다.

아래 첫 함수는 **바뀌는 데이터 소스 필드**, 두 번째는 **실제 SDK 제출과 중복 방지**, 마지막은 **SDK 연결·대기·전체 결과 수집**입니다. 실행할 것은 위 helper 명령 하나이며 원문을 독립적으로 다시 실행하지 않습니다.

실제 구현 코드 · 읽기 전용 · `scripts/add_foundry_eval_run.py:candidate_source`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 108-135 (파일: `scripts/add_foundry_eval_run.py#L108-L135`) · [GitHub에서 보기](#)
(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L108-L135)

```
def candidate_source(baseline: dict, *, evaluation_id: str, version: str) -> dict:
    if baseline.get("eval_id") != evaluation_id or baseline.get("status") != "completed":
        raise ValueError("Use a completed baseline run from this exact Foundry evaluation.")
    source = deepcopy(baseline.get("data_source"))
    if not isinstance(source, dict) or source.get("type") != "azure_ai_target_completions":
        raise ValueError("The baseline must be an agent-targeted Foundry evaluation.")
```

```

target = source.get("target", {})
if target.get("type") != "azure_ai_agent" or not target.get("name"):
    raise ValueError("The baseline must identify a Foundry agent.")
original = str(target.get("version", ""))
pinned_version = r"(?:[1-9]\d*|draft-[1-9]\d*)"
if not re.fullmatch(pinned_version, original) or not re.fullmatch(pinned_version, version):
    raise ValueError("Pin explicit numeric or draft agent versions, not latest.")
if original == version:
    raise ValueError("The candidate version must differ from the baseline.")
dataset = source.get("source", {})
if dataset.get("type") != "file_id" or not dataset.get("id"):
    raise ValueError("The baseline must use an identified, registered dataset.")
messages = source.get("input_messages", {})
template = messages.get("template", [])
if messages.get("type") != "template" or len(template) != 1 or template[0].get("role") != "user":
    raise ValueError("Use one query-only user message, without reference-answer leakage.")
content = template[0].get("content")
query = content if isinstance(content, str) else content.get("text") if isinstance(content, dict)
else None
if query != "{{item.query}}":
    raise ValueError("This workshop sends only {{item.query}} to the agent.")
target["version"] = version
return source

```

실제 구현 코드 · 읽기 전용 · scripts/add_foundry_eval_run.py:submit_or_resume

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 138–190 (파일: `scripts/add_foundry_eval_run.py#L138–L190`) · [GitHub에서 보기](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L138-L190)
[\(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L138-L190\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L138-L190)

```

def submit_or_resume(project, client, *, evaluation_id: str, baseline_run_id: str,
                    version: str, name: str, output: Path) -> dict:
    intent = {
        "evaluation_id": evaluation_id, "baseline_run_id": baseline_run_id,
        "candidate_version": version, "name": name,
    }
    if output.exists():
        saved = json.loads(output.read_text(encoding="utf-8"))
        if saved.get("intent") != intent:
            raise ValueError("This receipt belongs to a different evaluation or candidate.")
        if not saved.get("run_id"):
            raise ValueError("Submission outcome is unknown. Inspect Foundry before any new
submission; preserve this receipt.")
        return saved
    baseline = as_object(client.evals.runs.retrieve(run_id=baseline_run_id, eval_id=evaluation_id))
    source = candidate_source(baseline, evaluation_id=evaluation_id, version=version)
    agent_name = source["target"]["name"]
    original_version = str(baseline["data_source"]["target"]["version"])
    before = as_object(project.agents.get_version(agent_name=agent_name,
agent_version=original_version))["definition"]
    after = as_object(project.agents.get_version(agent_name=agent_name, agent_version=version))
["definition"]
    if not isinstance(after.get("instructions"), str) or not after["instructions"].strip():

```

```

        raise ValueError("The candidate has no actual instructions.")
    if {key: value for key, value in before.items() if key != "instructions"} != {
        key: value for key, value in after.items() if key != "instructions"
    }:
        raise ValueError("Model, tools, or other agent settings changed. This is not an instruction-
only comparison.")
    for existing in client.evals.runs.list(eval_id=evaluation_id, limit=100):
        existing = as_object(existing)
        if existing.get("name") == name:
            raise ValueError(
                f"Foundry already has a run named {name}: {existing.get('id')}. "
                "Open that run or restore its original receipt; do not create a duplicate."
            )
    evaluation = as_object(client.evals.retrieve(evaluation_id))
    contract = evaluation_contract(evaluation)
    saved = {
        "intent": intent,
        "status": "submitting",
        "run_id": None,
        "agent_name": agent_name,
        "dataset": source["source"],
        "testing_criteria_sha256": contract["testing_criteria_sha256"],
        "evaluation_contract": contract,
        "request": {"eval_id": evaluation_id, "name": name, "data_source": source},
        "production_approval": "NOT_GRANTED",
    }
    write_once_json(output, saved)
    # An interrupted POST intentionally leaves an unknown outcome, not permission to retry.
    run = as_object(client.evals.runs.create(**saved["request"]))
    if not run.get("id") or run.get("eval_id") != evaluation_id:
        raise ValueError("Foundry returned no matching evaluation-run identity; preserve the
submission receipt.")
    saved.update(status=run["status"], run_id=run["id"], run=run)
    save_json(output, saved)
    return saved

```

실제 구현 코드 · 읽기 전용 · scripts/add_foundry_eval_run.py:main

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 193–268 (파일: scripts/add_foundry_eval_run.py#L193–L268) · [GitHub에서 보기](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L193-L268)
[\(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L193-L268\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/scripts/add_foundry_eval_run.py#L193-L268)

```

def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--endpoint", required=True, help="The same Foundry project endpoint as the
baseline")
    parser.add_argument("--subscription", required=True, help="The explicitly selected Azure CLI
subscription")
    parser.add_argument("--evaluation", required=True, help="Existing eval... ID; no new evaluation
definition is created")
    parser.add_argument("--baseline", required=True, help="Completed baseline evalrun... ID")
    parser.add_argument("--version", required=True, help="Explicit candidate agent version")
    parser.add_argument("--name", default="candidate-v2")

```

```

    parser.add_argument("--out", type=Path, default=Path(".lab/foundry-evaluations/candidate-
v2.json"))
    parser.add_argument("--wait-seconds", type=int, default=1800)
    args = parser.parse_args(argv)
    try:
        endpoint = urlsplit(args.endpoint)
        if (
            endpoint.scheme != "https" or endpoint.username or endpoint.password
            or endpoint.port or endpoint.query or endpoint.fragment
            or not re.fullmatch(r"[a-z0-9-]+\.\services\.ai\.azure\.com", endpoint.hostname or "")
            or not re.fullmatch(r"/api/projects/[A-Za-z0-9_.()-]+/?", endpoint.path)
        ):
            raise ValueError("Use the exact HTTPS Foundry project endpoint, without credentials or
query parameters.")
        if not 1 <= args.wait_seconds <= 3600:
            raise ValueError("--wait-seconds must be between 1 and 3600.")
        account = subprocess.run(
            ["az", "account", "show", "--subscription", args.subscription, "-o", "json"],
            capture_output=True, text=True, check=True, timeout=30,
        )
        identity = json.loads(account.stdout)
        if identity.get("id", "").lower() != args.subscription.lower() or identity.get("state") !=
"Enabled":
            raise ValueError("Azure CLI did not confirm the intended enabled subscription.")
        from azure.ai.projects import AIProjectClient
        from azure.identity import AzureCliCredential

        with AzureCliCredential(subscription=args.subscription, process_timeout=30) as credential:
            with AIProjectClient(endpoint=args.endpoint, credential=credential, retry_total=0) as
project:

                with project.get_openai_client(max_retries=0, timeout=120) as client:
                    saved = submit_or_resume(
                        project, client, evaluation_id=args.evaluation, baseline_run_id=args.baseline,
                        version=args.version, name=args.name, output=args.out,
                    )
                    print(f"Foundry evaluation: {args.evaluation}\nRun: {saved['run_id']}",
flush=True)

                    deadline = time.monotonic() + args.wait_seconds
                    while True:
                        run = as_object(client.evals.runs.retrieve(run_id=saved["run_id"],
eval_id=args.evaluation))
                        saved.update(status=run["status"], run=run)
                        save_json(args.out, saved)
                        if run["status"] in {"completed", "failed", "canceled", "cancelled":
                            if run["status"] == "completed":
                                items = [
                                    as_object(item) for item in client.evals.runs.output_items.list(
                                        run_id=saved["run_id"], eval_id=args.evaluation,
                                    )
                                ]
                                saved["output_items"] = items
                                save_json(args.out, saved)
                                if len(items) != (run.get("result_counts") or {}).get("total"):
                                    raise ValueError("Completed-run output is incomplete; preserve the
receipt.")

                                agent = as_object(project.agents.get_version(
                                    agent_name=saved["agent_name"], agent_version=args.version,

```

```

    ))
    saved["execution_verification"] = verify_agent_execution(
        items, agent_name=saved["agent_name"], version=args.version,
        instructions=agent["definition"]["instructions"],
    )
    save_json(args.out, saved)
    print(json.dumps({
        "status": run["status"], "result_counts": run.get("result_counts"),
        "report_url": run.get("report_url"), "production_approval":
"NOT_GRANTED",
    }, indent=2))
    return 0 if run["status"] == "completed" else 1
    if time.monotonic() >= deadline:
        print("Still running. Repeat this exact command to collect the same run;
do not delete the receipt.", file=sys.stderr)
        return 2
    time.sleep(15)
except (OSError, ValueError, subprocess.SubprocessError) as error:
    print(f"ERROR: {error}", file=sys.stderr)
    return 1

```

3. Foundry에서 같은 평가의 v1/v2를 비교합니다

같은 평가의 **Evaluation runs**에서 v1·v2 두 행을 선택하고 **Compare runs**를 엽니다. **Baseline**을 **v1으로 명시 선택**하며 행 선택 순서 때문에 비교 방향이 바뀌지 않게 합니다.

Microsoft Foundry / proj-fev11-860db66e

Search with AI (🔍)

New Foundry

Home Discover Build Operate Manage Docs

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

← Statistically compare results

Total runs selected 2 Analysis method TTest

Baseline * ⓘ

baseline

Name	baseline	candidate_1
relevance	4.08 Baseline	4.00 Inconclusive
task_adherence	0.58 Baseline	0.67 Inconclusive

1-2 of 2

Prev Next

비교 방향과 결과를 함께 확인합니다. Baseline을 v1으로 선택하고 두 run의 점수·통과 건수·통계 결과를 비교합니다. Inconclusive는 차이가 입증되지 않았다는 뜻이며 동등성의 증거로 해석하지 않습니다. 원본 크기로 보기 (파일: [web/assets/portal/20-evaluation-comparison.png](#))

12건 전체의 응답·점수·이유를 나란히 비교합니다. 전체 기준 통과와 각 지표의 통과 건수·평균이 낮아지지 않고 하나 이상이 명확히 개선되어야 합니다. 사실·분류·응답 형식도 확인하며 지연·토큰·통계 결과를 함께 기록합니다.

결과 설명: 실제 개선, 같은 평가 기준, 회귀와 남은 불확실성을 설명합니다. 관측된 개선이 향후 모든 확률적 실행의 개선을 보장하지는 않습니다.

notes.md에 v1/v2의 지표별 통과 건수·평균, 오류, 지연·토큰, 통계 결과와 최종 판단을 나란히 기록합니다. 개선 조건을 만족하지 않으면 **v1 유지 또는 판단 보류**로 기록합니다. 범용 평가가 통과했어도 실제 정책 오류가 있으면 채택하지 않습니다.

지연(latency)은 응답을 기다린 시간, **토큰(tokens)**은 모델이 처리한 텍스트 양의 단위입니다. 점수가 조금 좋아져도 시간·비용 부담이 커질 수 있습니다.

최종 판단	선택할 때
v2 채택 후보	같은 조건의 12건에서 품질이 나빠지지 않고 하나 이상 개선됐으며, 정책 오류가 없고 지연·토큰의 부담도 검토했습니다. 운영용 게시 승인은 별도입니다.
v1 유지	개선이 없거나 기존에 잘하던 동작이 나빠졌습니다. 이것도 정상적인 학습 결과입니다.
판단 보류	실패·누락·비교 조건 차이로 아직 공정하게 판단할 수 없습니다. 부족한 근거를 기록합니다.

완료 기준: 실제 두 run ID와 같은 조건의 전체 비교, 채택·유지·보류 이유를 기록합니다. 정식 버전을 누적하지 않습니다. 운영 승인·독립적 일반화는 별개이며 dev12 실습으로 부여되지 않습니다.

10. 결과를 보관하고 리소스를 삭제합니다

이 단계에서 하는 일: 결과를 보관한 뒤 자신이 만든 전용 실습 자원을 삭제합니다. 보존을 승인받은 경우에만 남은 자원과 비용 관리 계획을 기록합니다.

중요한 이유: 브라우저를 닫아도 Search·로그 등은 남을 수 있습니다. 반대로 공유 그룹을 잘못 삭제하면 다른 사람의 자원도 잃을 수 있습니다.

진행 방법·위치: Azure Portal과 터미널에서 정확한 대상·소유권·승인을 확인하고 삭제 후 부재를 검증합니다.

실행 순서: [결과 보관](#) → [삭제·보존 범위 확인](#) → [승인된 전용 그룹 삭제](#) → [터미널 · 부재 확인](#). 보존 승인된 환경은 삭제하지 않습니다.

브라우저 종료·Agent 삭제·cleanup 실행만으로 리소스 그룹 전체의 과금이 멈추지는 않습니다.

삭제 전에 기록을 보관합니다

1. notes.md에 실제 프로젝트·Agent·데이터셋 버전·해시, 평가/run/job ID와 최종 판단을 남깁니다.

2. 평가·Optimizer 화면에서 제공하는 **Download/Export**로 결과를 받습니다. 메뉴가 없으면 보관한 receipt와 상세 결과를 사용하며, 받을 수 없는 결과를 받았다고 기록하지 않습니다.
3. `.lab/lab-ko/`의 계획·manifest·승인·실행 기록을 조직이 허용한 비공개 위치에 보관합니다. 인증 정보·쿠키·서명된 URL·계정 정보가 있는 원본을 Git에 올리지 않습니다. 공개용 기록에는 필요한 합성 사례·점수·이유만 남깁니다.
4. Evaluations와 Optimization runs에서 실행 중인 작업을 확인합니다. 지원되는 **Cancel**을 사용하고 종료 상태를 확인합니다. 종료되지 않은 작업의 ID·상태와 후속 확인 계획을 기록하며 취소 요청만으로 종료됐다고 표시하지 않습니다.

삭제할 대상과 보존 여부를 확인합니다

환경	정리 범위
02에서 본인이 만든 전용 리소스 그룹 · 기본 경로	아래 전용 그룹 삭제 절차를 사용합니다. 그룹 안에 다른 업무 자원이 없고 삭제가 승인된 범위인지 다시 확인합니다.
보존하도록 승인받은 전용 환경 · 예외	삭제 명령을 실행하지 않습니다. 실제 남은 자원·보존 이유·비용 책임·보존 검토일과 이후 삭제 계획을 기록합니다.
삭제 목록에 공유·외부 자원이 포함됩니다.	해당 자원은 삭제하지 않습니다. 실습 범위를 벗어난 종속성을 확인하고 승인된 처리 범위가 정해질 때까지 그룹 삭제를 중단합니다.
생성·실습 중간에 중단한 환경	<code>config.json</code> ·manifest와 Azure의 실제 자원을 대조합니다. 실패했다고 자원이 없다고 가정하지 않습니다.

기본 경로는 자신의 전용 그룹 전체 삭제입니다. 보관·대상 확인이 끝나면 [다음 절](#)로 진행합니다. 그룹 전체를 삭제할 때 개별 객체를 먼저 삭제할 필요는 없습니다. 아래 cleanup은 **그룹을 보존하면서 기록된 객체만 정리할 때** 사용하는 별도 선택 사항입니다.

선택 사항 · 그룹을 보존하면서 기록된 객체만 정리합니다

자신의 `.env`와 로컬 소유 기록이 있는 경우 다음 명령으로 **삭제 계획만** 확인할 수 있습니다. 생성 도중 중단되어 `.env`가 없다면 객체 정리 명령을 실행하지 않고 `config.json`·manifest로 실제 그룹을 확인합니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env cleanup
```

`mode: LOCAL_PLAN_ONLY`, `actions`, `never_deleted`, `manual_follow_up`를 읽습니다. 삭제 대상의 이름·프로젝트·소유자를 확인하고 해당 범위의 삭제 승인을 받은 경우에만 다음을 실행합니다. 자신의 환경 이름이 다르다면 `prefix`도 동일하게 바꿉니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env cleanup --confirm-prefix lab-ko
```

`mode: OWNED_OBJECTS_ABSENT`인지 확인합니다. 이 도구는 기록된 Agent 버전·검색 객체·연결 등을 삭제하지만 **리소스 그룹·모델 배포·Search 서비스·로그·RBAC는 삭제하지 않습니다.** 포털에서 만든 데이터셋·평가·Optimizer 작업·Playground 대화와 모델 smoke 응답도 전부 자동 정리하지 않습니다. 승인된 항목은 해당 화면에서 개별 삭제합니다. 삭제 기능이 없는 항목은 남은 목록과 보존 이유를 기록하고, 전용 그룹 전체 삭제 여부를 별도로 결정합니다.

코드 ↔ 포털 · 객체 정리와 그룹 삭제를 구분합니다

실행 위치: 기본 경로는 [전용 그룹 삭제](#) 후 [부재 확인](#)입니다. 보존 승인된 그룹에서 객체만 지울 때에만 [위의 선택 사항 cleanup](#)을 사용합니다. 두 경로를 모두 수행할 필요는 없습니다.

기존 명령·실제 구현	포털 조작·확인 위치
<code>cleanup_plan()</code> 의 <code>actions</code> , <code>never_deleted</code> , <code>manual_follow_up</code>	로컬 소유 기록에서 만든 계획입니다. 해당 Agent 버전·Search 객체·프로젝트 연결 이름을 포털과 대조합니다. 출력에 다른 라이브러리 경로의 후속 안내가 있어도 이번 실습에 그 작업을 추가하지 않습니다.
<code>cleanup()</code> 의 <code>delete_version</code> , REST DELETE, 이후 GET	확인한 기록 객체만 삭제하고 부재를 조회합니다. 개별 삭제도 승인 범위에서만 수행하며 포털 삭제와 중복 실행하지 않습니다.
아래의 <code>az group show</code> , <code>az resource list</code>	Azure Portal Resource groups → 자신의 그룹 → Overview/Resources 에서 전체 인벤토리와 태그를 읽습니다.
아래의 <code>az group delete</code>	포털 Delete resource group 과 같은 그룹 전체 삭제 경로입니다. 둘 중 한 방법만 실행합니다.
<code>az group exists</code> 의 성공한 <code>false</code>	삭제 후의 실제 부재 확인입니다. 포털 알림·그룹 목록과 대조합니다. 오류를 <code>false</code> 로 해석하지 않습니다.

원문에서 **확인 인자가 없으면 계획 반환, 원격 소유권·etag 확인, 개별 DELETE, 다시 조회하여 remaining 검사** 순서를 읽습니다. 그룹 전체 삭제는 이 함수 안에 없고 다음 절의 기존 Azure CLI 명령 또는 포털에서 별도로 수행합니다.

실제 구현 코드 · 읽기 전용 · `lab/cleanup.py:cleanup_plan`

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 17~74 (파일: `lab/cleanup.py#L17-L74`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L17-L74\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L17-L74)

```
def cleanup_plan(config: Config) -> dict:
    state = workspace(config)
    names = resource_names(config, state)
    allowed = {
        "search_index": (names["index"], search_url(config, "indexes", names["index"])),
        "knowledge_source": (names["source"], search_url(config, "knowledgesources",
names["source"])),
        "knowledge_base": (names["base"], search_url(config, "knowledgebases", names["base"])),
        "project_connection": (
            names["connection"],
            f"https://management.azure.com{config.project_id}/connections/{names['connection']}?api-
version={CONNECTION_API}",
        ),
    }
    allowed_agents = {config.agent_name(stage) for stage in ("baseline", "iq", "optimized", "tuned")}
    actions = []
    for record in state["created"]:
        kind = record.get("kind")
```

```

        if kind == "agent_version":
            if record.get("name") not in allowed_agents or not re.fullmatch(r"[A-Za-z0-9_-]+",
str(record.get("version", ""))):
                raise LabError("정리 기록의 에이전트 범위가 다릅니다. 삭제하지 않습니다.")
            elif kind in allowed:
                if (record.get("name"), record.get("url")) != allowed[kind]:
                    raise LabError("정리 기록의 Search/프로젝트 범위가 다릅니다. 삭제하지 않습니다.")
            else:
                raise LabError(f"자동 정리를 지원하지 않는 기록 유형: {kind}")
        actions.append(dict(record))
    for metadata_path in (ARTIFACTS / "runs").glob("*/*metadata.json"):
        metadata = read_json(metadata_path)
        if metadata.get("workspace_id") != state["workspace_id"]:
            continue
        if metadata.get("project_endpoint") != config.project_endpoint:
            raise LabError("응답 정리 대상의 프로젝트 endpoint가 다릅니다.")
        outputs_path = metadata_path.parent / "outputs.jsonl"
        if not outputs_path.exists():
            continue
        for row in read_jsonl(outputs_path):
            identifier = row.get("response_id")
            if identifier:
                if not re.fullmatch(r"resp_[A-Za-z0-9_-]+", identifier):
                    raise LabError("알 수 없는 response ID 형식입니다. 자동 삭제하지 않습니다.")
                actions.append({"kind": "response", "id": identifier})
    order = {"response": 0, "agent_version": 1, "project_connection": 2, "knowledge_base": 3,
"knowledge_source": 4, "search_index": 5}
    actions.sort(key=lambda action: order[action["kind"]])
    return {
        "mode": "LOCAL_PLAN_ONLY",
        "project_id": config.project_id,
        "prefix": config.prefix,
        "workspace_id": state["workspace_id"],
        "actions": actions,
        "never_deleted": [
            "resource group", "Foundry account/project", "shared model deployments",
            "Search service", "Application Insights/Log Analytics", "RBAC assignments",
        ],
        "manual_follow_up": [
            "Managed evaluation jobs/files are retained for evidence; inspect recorded IDs and remove
in the portal if required.",
            "SFT/FDE training jobs, uploaded training files and model deployments require their own
scoped cleanup.",
            "An empty stable agent entry may remain after deleting its recorded versions; inspect
ownership before removing it in the portal.",
        ],
    }
}

```

실제 구현 코드 · 읽기 전용 · lab/cleanup.py:cleanup

읽기용 원문이며 별도로 실행하는 명령이 아닙니다. 지정한 함수·객체 전체를 원래 메시지까지 그대로 표시합니다. 의존 함수와 import는 연결한 전체 파일에서 확인합니다. 실습 명령만 한 번 실행합니다.

전체 구현 파일 · 77-165 (파일: `lab/cleanup.py#L77-L165`) · [GitHub에서 보기 \(https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L77-L165\)](https://github.com/junwoojeong100/foundry-evaluation-labs-v1/blob/main/lab/cleanup.py#L77-L165)

```

def cleanup(config: Config, *, confirm_prefix: str | None = None) -> dict:
    plan = cleanup_plan(config)
    if confirm_prefix is None:
        return plan
    if confirm_prefix != config.prefix:
        raise LabError("--confirm-prefix가 현재 LAB_PREFIX와 다릅니다. 삭제하지 않았습니다.")
    with credential_for(config) as credential:
        with AIProjectClient(endpoint=config.project_endpoint, credential=credential) as project:
            with project.get_openai_client(max_retries=0, timeout=60.0) as client:
                search = JsonHttp(credential, scope=SEARCH_SCOPE,
allowed_origin=config.search_endpoint)
                arm = JsonHttp(credential, scope=ARM_SCOPE,
allowed_origin="https://management.azure.com")
                inspected = []
                for action in plan["actions"]:
                    item = {**action, "exists": True}
                    kind = action["kind"]
                    if kind == "agent_version":
                        try:
                            remote = project.agents.get_version(agent_name=action["name"],
agent_version=action["version"])
                        except ResourceNotFoundError:
                            item["exists"] = False
                        else:
                            if (remote.metadata or {}).get("workspace") != plan["workspace_id"]:
                                raise LabError("원격 에이전트 소유 표식이 다릅니다. 아무 대상도 삭제하지 않았습니다.")
                    elif kind == "response":
                        try:
                            response = client.responses.retrieve(action["id"])
                        except NotFoundError:
                            item["exists"] = False
                        else:
                            if (response.metadata or {}).get("lab_workspace") != plan["workspace_id"]:
                                raise LabError("원격 응답 소유 표식이 다릅니다. 아무 대상도 삭제하지 않았습니다.")
                    else:
                        transport = arm if kind == "project_connection" else search
                        try:
                            remote = transport.request("GET", action["url"])
                        except CloudRequestError as exc:
                            if exc.status != 404:
                                raise
                            item["exists"] = False
                        else:
                            if action.get("etag") and remote.etag and action["etag"] != remote.etag:
                                raise LabError("생성 후 원격 리소스 구성이 변경되었습니다. 정리 대상을 직접 검토해야 합
니다.")
                            item["current_etag"] = remote.etag
                    inspected.append(item)
                completed = []
                for action in inspected:
                    if not action["exists"]:
                        completed.append({**action, "result": "already_absent"})
                        continue
                    kind = action["kind"]
                    if kind == "response":
                        client.responses.delete(action["id"])
                    elif kind == "agent_version":

```

```

        project.agents.delete_version(agent_name=action["name"],
agent_version=action["version"])
    else:
        transport = arm if kind == "project_connection" else search
        transport.request("DELETE", action["url"], etag=action.get("current_etag"))
        completed.append(**action, "result": "delete_request_succeeded")
        save_json(ARTIFACTS / "cleanup.json", {**plan, "mode": "DELETING", "completed":
completed})

    remaining = []
    for action in inspected:
        kind = action["kind"]
        if kind == "response":
            try:
                client.responses.retrieve(action["id"])
            except NotFoundError:
                continue
        elif kind == "agent_version":
            try:
                project.agents.get_version(agent_name=action["name"],
agent_version=action["version"])
            except ResourceNotFoundError:
                continue
        else:
            transport = arm if kind == "project_connection" else search
            try:
                transport.request("GET", action["url"])
            except CloudRequestError as exc:
                if exc.status == 404:
                    continue
                raise
            remaining.append(action)

    result = {
        **plan,
        "mode": "OWNED_OBJECTS_ABSENT" if not remaining else "DELETION_PENDING",
        "completed": completed,
        "remaining": remaining,
    }
    save_json(ARTIFACTS / "cleanup.json", result)
    return result

```

전용 리소스 그룹을 삭제하고 부재를 확인합니다

YOUR_LAB_RESOURCE_GROUP을 config.json의 names.resource_group으로 교체합니다. 이름의 prefix만 보고 판단하지 않고 구독·소유 태그·목록 전체를 확인합니다.

터미널에서 실행 · 공통 터미널

```

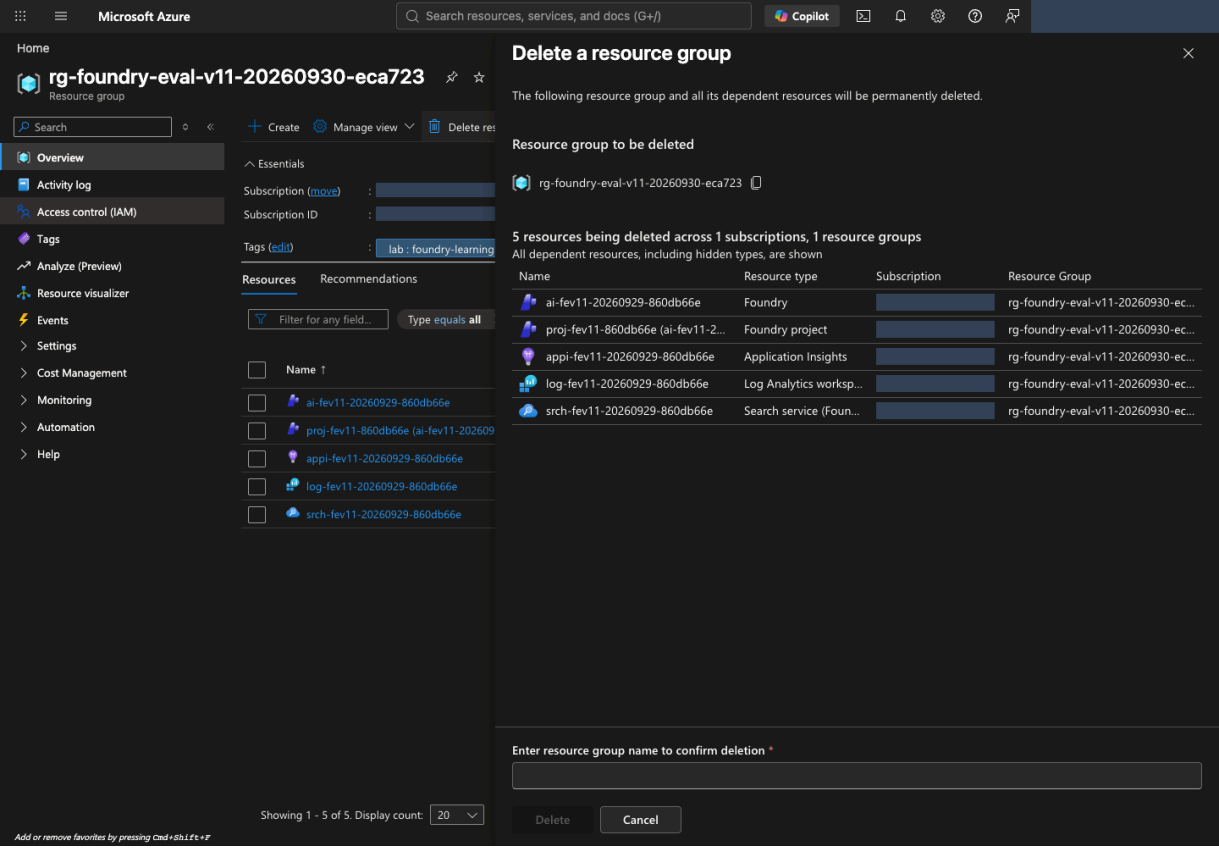
az group show --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP" --query "
{name:name,location:location,tags:tags}" -o json
az resource list --subscription "YOUR_SUBSCRIPTION_ID" --resource-group "YOUR_LAB_RESOURCE_GROUP" --
query "[].{name:name,type:type}" -o table

```

그룹 전체 삭제는 되돌릴 수 없으며 그룹 안의 Foundry·모델·Search·모니터링 자원이 함께 삭제됩니다. 보관이 끝났고 정확한 그룹 전체를 삭제하도록 승인받은 경우에만 다음 두 방법 중 하나를 선택합니다.

Application Insights의 기본 Smart Detection Action group은 다른 그룹의 경고에서도 공유할 수 있습니다. 전용 실험 그룹이라는 이유만으로 모든 항목이 독립적이라고 가정하지 않습니다. 공유 연결이 있으면 담당자와 종속성을 먼저 정리하거나 해당 그룹을 보존하며, 확인을 위해 경고·권한·잠금을 임의로 삭제하지 않습니다.

- 1. **포털 방법:** Azure Portal → Resource groups → 정확한 그룹 → **Delete resource group**을 선택합니다. 삭제 목록을 읽고 요구하는 그룹 이름을 직접 입력한 뒤 확인합니다.
- 2. **CLI 방법:** 다음 명령을 실행하고 확인 질문에서 이름·범위를 다시 확인한 뒤 동의합니다. `--yes`를 붙여 확인을 생략하지 않습니다.



삭제 예정 목록과 확인란을 먼저 읽습니다. 그룹 이름과 전체 자원 목록을 대조합니다. 승인받은 경우에만 확인란에 그룹 이름을 입력하고 Delete로 진행합니다. 대상이 다르거나 승인되지 않았다면 Cancel로 닫습니다. 원본 크기로 보기 (파일: `web/assets/portal/28-delete-review.png`)

터미널에서 실행 · 공통 터미널

```
az group delete --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP"
```

터미널에서 삭제 후 부재를 확인합니다

삭제 요청 수락은 완료가 아닙니다. 포털 알림과 그룹 상태를 확인하고 다음 명령이 성공적으로 `false`를 출력하는지 확인합니다.

터미널에서 실행 · 공통 터미널

```
az group exists --subscription "YOUR_SUBSCRIPTION_ID" --name "YOUR_LAB_RESOURCE_GROUP"
```

권한 오류·네트워크 오류는 false가 아니며 삭제 성공으로 처리하지 않습니다. true이면 삭제 진행 상태를 확인합니다. **Locks**, 권한, 다른 그룹의 종속 자원 때문에 실패하면 [삭제 문제 해결](#)을 따릅니다. 조직이 설정한 잠금을 임의로 제거하지 않습니다.

삭제 뒤 **Cost Management → Cost analysis**에서 해당 구독·기간·리소스 그룹을 확인합니다. 청구 반영에는 지연이 있으며 기존 사용 요금은 사라지지 않습니다. 별도 그룹의 로그·저장소나 서비스별 soft-delete 보존 항목은 따로 확인합니다. 영구 삭제/purge는 조직 정책과 별도 승인이 있을 때만 수행합니다.

최종 완료 기준: 삭제 승인된 자신의 전용 그룹에 대해 az group exists의 false와 확인 시각을 기록했습니다. 보존 승인된 경우에는 실제 남은 항목·이유·비용 책임·보존 검토일과 후속 삭제 계획을 기록했습니다. 둘 중 자신의 승인 범위에 맞는 상태를 확인하고 .lab 기록을 먼저 지워 소유권 근거를 잃지 않습니다. [정리 체크리스트](#) · [삭제 문제 해결](#)을 참고합니다.

실습 체크리스트 · 실제 측정 결과 개선하기

[실습 10단계](#) · [환경·결과 기록표](#) · [문제 해결](#)

본인이 진행한 실습의 **평가 → 학습 → 개선 → 재평가**를 점검하는 자료입니다. [실습 가이드](#)를 대신하는 별도 경로가 아닙니다. 약한 기준선을 인위적으로 만들거나 앞으로의 모든 평가가 개선된다고 약속하지 않습니다.

시작 전에 확인합니다

모든 실습 참여자가 01–10을 직접 수행합니다. 본인의 계정·PC·전용 환경·Agent를 사용하며 다른 사람에게 생성·명령·정리를 맡기지 않습니다. 시작 전에 필요한 생성·역할 할당 권한과 비용·삭제/보존 범위를 확인합니다.

[01의 Windows·macOS·Linux 설치 안내](#)에 따라 **Python 3.11–3.14·Git·Azure CLI**의 [버전 확인](#)과 가상환경 생성을 마칩니다. 설치 승인·PATH·프록시 문제를 먼저 해결합니다.

02에서 계획을 만든 뒤 [기록표](#)를 자신의 notes.md에 복사합니다. 단계마다 실제 값·완료 근거를 추가하고 다른 사람의 설정·로그인·소유권 기록을 사용하지 않습니다. 재개할 때는 자신의 원본 기록을 확인합니다.

[여섯 가지 용어와 질문→응답→채점 도식](#)을 읽고 **“답변하는 Agent와 채점하는 Judge는 다르다”, “모델이 아니라 지침을 바꾼다”**를 설명할 수 있는지 확인합니다. 용어 암기나 높은 점수는 시작 조건이 아닙니다.

04–06은 같은 평가 생성 화면입니다. 단계마다 새 평가를 만들지 않습니다. 07에서는 한 사례의 점수·이유·정책을 연결해 읽은 뒤 전체 결과를 분석합니다.

각 단계의 **실행 순서·실행 위치** 링크로 실제 명령이나 포털 화면을 먼저 엽니다. **코드 ↔ 포털** 표와 접힌 구현 패널은 동작을 이해할 때 읽으며, 추가로 실행하지 않습니다. 특히 [06의 ID 조회 명령](#)은 제출 명령이 아니며, [08 Optimizer](#)는 포털에서 수행합니다.

새 실습의 Agent는 lab-ko-iq 또는 lab-en-iq입니다. 환경 prefix를 바꾸었다면 명령이 출력한 실제 이름을 사용합니다. [모델 역할·신원](#)을 확인하고 10의 정리 시간을 확보합니다.

v1·v2는 변경 불가능한 Agent 전체 구성 버전입니다. 모델·도구·추론·엄격한 출력 스키마는 같고 지침만 다릅니다. 개선을 만들려고 v1을 악화하지 않습니다. 검토한 v2를 한 번 만들어 같은 기준으로 재평가합니다.

영어·한국어 데이터와 Agent를 분리하고 각 언어의 실제 실행 결과를 독립적으로 비교합니다. 다른 언어의 결과를 번역하여 새 실행으로 표시하지 않습니다.

10단계의 실제 완료 기준을 확인합니다

단계	필수 동작	완료 근거
01 환경 확인	계정·구독·권한·Python 3.11~3.14·Git·Azure CLI·언어 변수를 확인합니다.	신원이 일치하고 세 도구의 버전 확인과 가상환경의 로컬 명령이 성공합니다.
02 환경 생성	계획·승인·preflight·apply·status를 순서대로 수행합니다.	APPLIED, 생성된 .env, 런타임 preflight PASS를 확인합니다.
03 Agent 준비	모델·정책 검색을 확인하고 native-agent v1을 생성합니다.	실제 Agent의 JSON 응답과 도구 호출을 확인합니다.
04 데이터셋	변경 없는 세 열 JSONL을 등록하거나 재사용합니다.	12행 전체·등록 버전·SHA-256을 기록합니다.
05 평가기	Relevance 4·TaskAdherence 1과 자신의 Judge를 선택합니다.	제출 전 생성 화면의 설정과 query 전용 입력을 기록합니다.
06 기준선	고정 v1에 Foundry Evaluation을 실행합니다.	완료된 실제 run ID와 전체 12건을 확인합니다.
07 분석	실제 답변·점수·이유·정책을 읽습니다.	실패를 포함한 구체적인 개선 가설을 기록합니다.
08 최적화	Instruction only로 실행하고 지침 diff를 검토합니다.	job/candidate ID·후보 파일·출처를 기록하거나 다른 후보가 없어 v1을 유지할 이유를 기록합니다.
09 재평가	검토한 후보가 있을 때 같은 조건의 v2를 실행하고 Compare runs를 엽니다.	전체 행 대응과 유지·채택·보류 이유를 기록합니다. 다른 후보가 없으면 10으로 이동합니다.
10 정리	결과와 소유권을 보관한 뒤 자신의 전용 자원을 정리합니다.	승인된 그룹 삭제의 부재 확인, 또는 보존 승인된 자원의 실제 목록·비용 책임·검토일을 기록합니다.

Individual turns / One time을 사용합니다. 처음 등록할 때는 **Upload new dataset**, 재개·재사용할 때는 **Existing dataset**을 선택합니다. 업로드 미리 보기가 5행이어도 실험은 12행입니다. 버전 선택으로 체크박스가 해제되면 다시 선택하고 대상 한 개를 확인합니다.

실제 사례로 품질을 확인합니다

질문 → 실제 답변 → 평가 점수·이유 → 정책·참고 답변을 연결합니다. conversation_id → User view는 대화이며 평가 이유는 Detailed metrics result의 Relevance, reason·TaskAdherence, reason에서 확인합니다.

약한 사례와 강한 사례를 하나 이상 분석합니다. 실패가 없다면 최저점 사례를 사용하며 실패를 관측하지 못했다고 설명합니다. v1을 약화하거나 어려운 행·참고 답변을 바꾸거나 로컬 점수로 대신하지 않습니다. 다른 참여자와 토론할 때도 자신의 실제 결과를 사용합니다.

선택한 실습의 합성 질문 12건, 실제 응답과 관리형 평가 이유를 별도 비공개 실행 기록으로 보관합니다. 실패 결과를 숨기지 않으며, 공유가 필요할 때는 토큰·쿠키·서명된 URL·비공개 계정 정보를 제외합니다.

개선 해석

Relevance 4/5는 정확도 80%가 아닙니다. TaskAdherence 1은 통과이지 5점 중 1점이 아닙니다. 지표별 점수·통과 건수, 전체 기준 통과, 오류·평가 범위를 구분합니다.

09의 필수 비교는 Foundry에서 수행합니다. 선택 사항인 `scripts/compare_foundry_eval.py`도 전체 사례 대응, 데이터·모델·도구·설정 일치, 실제 행별 버전·지침, 통과 건수·평균의 비회귀와 하나 이상의 명확한 관측 개선을 요구합니다. 잘못된 JSON을 고쳐 통과로 바꾸지 않고 실패로 남깁니다.

범용 평가기가 통과시켜도 근거 없는 확신, 정책 계산 오류, 실행 완료 주장과 잘못된 분류를 직접 읽습니다. 불가능한 확답을 요구한다고 사실을 만들어내면 안 됩니다.

Optimizer 내부 순위는 후보 선택 근거이지 별도 관리형 재평가의 평균·통과율과 같은 값이 아닙니다. 순위 상승만으로 더 나은 v2가 입증되지 않습니다.

실험 조건을 바꾸지 않는 복구

증상	대응
다른 신원, 401/403	승인된 계정·프로젝트를 확인하고 공유 권한은 확대하지 않습니다.
카탈로그·배포 성공 후 Agent 실패	실제 API·런타임 오류를 남기고 해당 역할의 지원을 확인합니다.
포털 Add run item-schema 오류	기존 기준선 data source의 공식 SDK helper를 사용하며 다른 채점기로 대체하지 않습니다.
No custom evaluators available	Custom only OFF 또는 View built-in evaluators 를 선택합니다.
로컬 receipt-같은 이름의 원격 run 존재	같은 run을 조회하고 이름 변경을 통한 중복 제출은 하지 않습니다.
후보가 모델·도구·추론·스키마 변경	지침만의 비교라고 주장하지 않습니다.
기존 v2와 소스 불일치	덮어쓰기·v3 생성 없이 원본을 보존하고 확인합니다.
결과·점수 누락	부분 실행으로 표시하고 분모를 줄이지 않습니다.
통계 비교 Inconclusive	그대로 보고하며 유의성·동등성을 주장하지 않습니다.

Relevance는 `{{sample.output_text}}`, TaskAdherence는 `{{sample.output_items}}`를 사용합니다. 서비스 생성 매핑을 덮어쓰거나 `ground_truth`를 Agent에 주입하지 않습니다. 오래된 로컬 기본값보다 원격 평가의 실제 Judge가 우선입니다.

시간 초과 시 **어느 작업의 ID로 재개할지**를 먼저 확인하고 **문제 해결**을 따릅니다. 삭제·재생성하거나 “성공할 때까지” 유료 호출을 반복하지 않습니다. 막힌 단계는 실제 완료로 체크하지 않습니다.

흔한 오해 바로잡기

주장	설명
“v2는 지침 파일 번호일 뿐입니다.”	Agent 전체 버전이며 이번에는 지침만 바꿉니다.
“새 모델 v2를 옛 모델 v1과 비교하면 됩니다.”	지침 효과를 비교하려면 두 버전의 모델이 같아야 합니다.
“실행 전에 반드시 개선된다고 보장합니다.”	실측 개선은 확인할 수 있지만 향후 확률적 결과는 보장하지 않습니다.
“성공한 행만 남깁니다.”	실패·오류·누락까지 전체 사례를 남겨야 합니다.
“스키마가 올바른 업무 판단을 인증합니다.”	구조를 제한할 뿐 정책 사실·분류까지 보장하지 않습니다.
“공개 근거에 쿼리·원본 로그가 필요합니다.”	합성 실응답·이유를 허용 필드로 공개하고 인증 정보는 제외합니다.

별도 실습 기록으로 마무리

실제 환경·run ID·비교 조건과 최종 판단을 자신의 비공개 기록에 보관합니다. 실행 결과나 검증 이력을 재사용 가이드에 덧붙이지 않으며 안내가 잘못된 경우에만 절차를 수정합니다.

동일 모델 비교 결과, 서비스 통계 검정, 지연·토큰 상충 관계와 합성 dev 재사용의 한계를 기록합니다. 실습의 채택 판단은 운영용 게시·활성화 승인이 아니며 운영용 게시는 이번 실습에 포함하지 않습니다.

마지막으로 10의 정리·보존 확인을 직접 수행합니다. 승인된 삭제의 완료 또는 보존 자원의 실제 목록·비용 책임·검토일을 확인합니다. 소유권·재접속에 필요한 기록을 먼저 삭제하지 않습니다.

환경 설정 참고 · 권한·승인·정리

[처음부터 진행하는 10단계](#) · [실습 체크리스트](#) · [문제 해결](#)

이 문서는 **본인이 수행하는 02 환경 생성, 03 Agent 준비, 10 정리**의 설정 참고 자료입니다. [실습 가이드](#)를 순서대로 진행하면서 필요한 절만 엽니다. 실제 값과 완료 상태는 자신의 `.lab/lab-ko/notes.md`에 기록합니다.

실행 명령 바로가기: [CLI 로그인](#) · [언어·기록 폴더](#) · [환경 생성](#) · [정책·Agent 준비](#) · [평가 ID 조회](#) · [재평가](#) · [그룹 삭제·확인](#). 모든 명령은 실습 폴더의 터미널에서 실행하며, 이미 완료한 작업은 반복하지 않습니다.

명령을 바꾸지 않고 안쪽 동작을 읽으려면 [02 생성 코드·포털](#), [03 검색 코드·포털](#), [Agent 구성](#), [09 제출·조회](#), [10 정리 범위](#)를 엽니다. 함수 원문은 읽기용이며 별도 실행이나 포털 중복 생성 지시가 아닙니다.

자신의 전용 실습 환경을 준비합니다

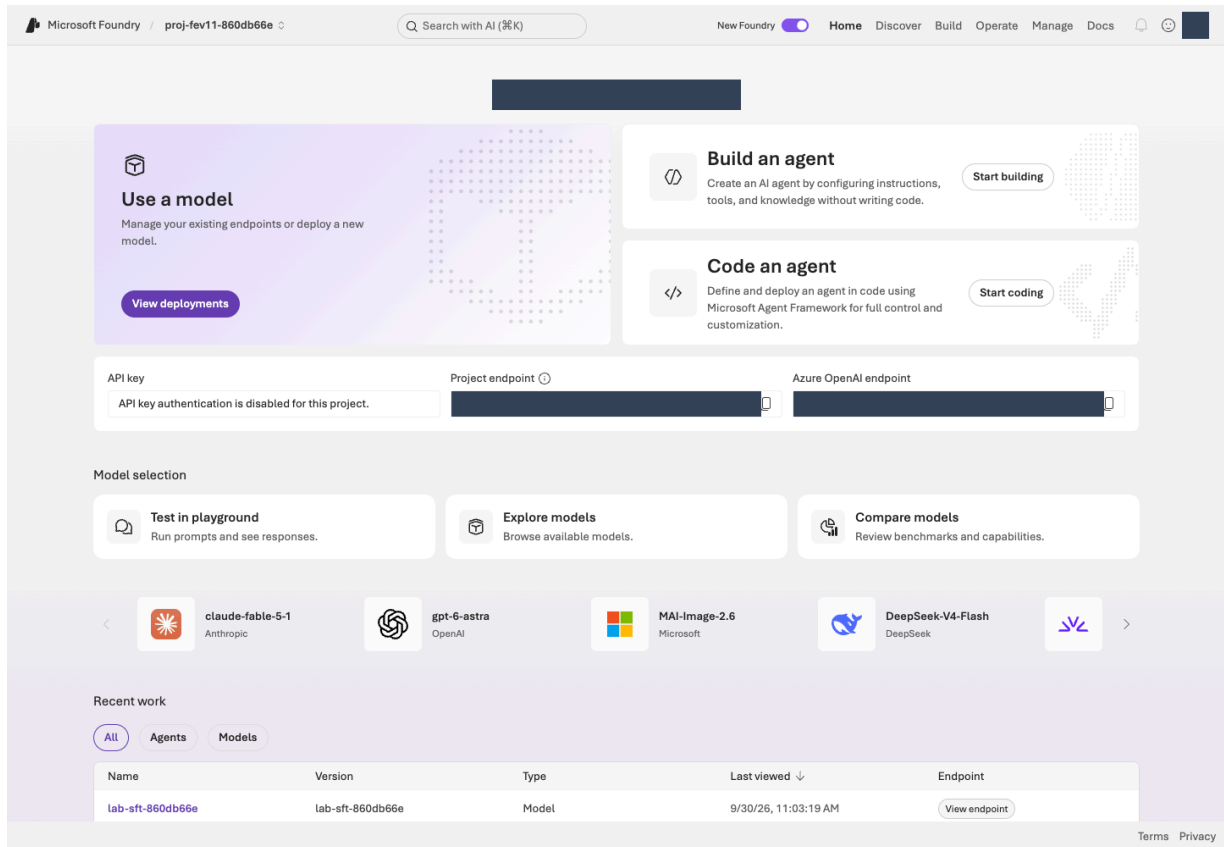
상황	진행 방법
처음 시작합니다.	01 계정 준비 부터 진행하고 본인에게 허용된 <code>bootstrap plan</code> → <code>preflight</code> → <code>apply</code> → <code>status</code> 경로로 생성합니다.
본인의 같은 실습을 재개합니다.	원래 <code>config·manifest·receipt</code> 와 실제 상태를 기록표 로 대조합니다. 완료한 생성·제출을 반복하지 않습니다.
다른 업무의 공유 프로젝트만 있습니다.	이 실습은 기존 그룹을 인수하지 않습니다. 본인이 전용 환경을 만들 수 있는 구독·권한·비용 범위를 확보한 뒤 진행합니다.

CLI·SDK·포털의 계정·테넌트·구독을 각각 확인하고 같은 본인 신원으로 진행합니다. 다른 사람의 토큰·로그인 세션·설정·소유권 기록을 사용하지 않습니다. 생성부터 ID 조회·v2 생성·재평가·정리까지 본인이 실행합니다.

모든 실습 참여자는 자신의 전용 환경과 Agent를 사용합니다. 여러 사람이 같은 Agent의 v2를 만들거나 같은 유료 작업을 제출하는 경로는 사용하지 않습니다. 각 PC에서 같은 예시 환경 이름을 써도 생성되는 Azure 자원 이름에는 고유 suffix가 붙습니다.

권한·비용 승인은 생략할 수 없는 시작 조건입니다. 부족하면 조직 절차에 따라 본인에게 허용된 범위를 확보합니다. 이것은 이후 실습을 다른 사람에게 맡기는 별도 경로가 아닙니다.

기본 Agent 이름은 한국어 `lab-ko-iq`, 영어 `lab-en-iq`입니다. 환경 이름을 바꾼 경우 CLI가 출력한 실제 이름을 사용하고 기록표에 적습니다.



실습 프로젝트를 확인합니다. 프로젝트 이름과 endpoint를 자신의 생성 기록과 대조합니다. 리소스 그룹은 Azure Portal의 Resource groups에서 같은 구독과 그룹 이름으로 확인합니다. 원본 크기로 보기 (파일: web/assets/portal/01-project-overview.png)

모델 역할과 실제 배포를 확인합니다

역할	설정 키	기본 모델.버전
Agent	MODEL_DEPLOYMENT	gpt-6-sol / 2026-09-22, GlobalStandard 100입니다.
관리형 평가 Judge	JUDGE_DEPLOYMENT	gpt-6-luna / 2026-09-22, GlobalStandard 100입니다.
Optimizer·검색 planner	OPTIMIZER_DEPLOYMENT·IQ_PLANNER_DEPLOYMENT	gpt-5.5 / 2026-04-24, 같은 GlobalStandard 100 배포를 사용합니다.
정책 embedding	EMBEDDING_DEPLOYMENT	text-embedding-3-small / 1, GlobalStandard 10입니다.

숫자는 ARM 요청 용량 단위이며 모든 모델에서 같은 TPM을 뜻하지 않습니다. 이것은 사용 가능한지 확인해야 할 기본 계획이지 모든 구독의 배포 보장이 아닙니다. Foundry의 **Models + endpoints/Build → Models**에서 배포 이름·모델·버전·상태를 대조합니다. `.env`에는 모델 제품명이 아니라 실제 배포 이름이 들어갑니다.

관측 리소스의 추가 항목을 구분합니다. Application Insights는 [기본 Failure Anomalies 경고와 Smart Detection Action group](https://learn.microsoft.com/azure/azure-monitor/alerts/proactive-failure-diagnostics#alert-rule-creation) (<https://learn.microsoft.com/azure/azure-monitor/alerts/proactive-failure-diagnostics#alert-rule-creation>)을 자동으로 추가할 수 있습니다. bootstrap은 경고가 자신의 Application Insights만 대상으로 하는지, 연결된 Action group이 기본 역할 수신자만 사용하는지 읽기 전용으로 확인합니다. 이름만 같은 항목을 인수하지 않으며, 다른 그룹의 공유 Action group도 변경·삭제하지 않습니다. 생성 중 연결이 아직 확인되지 않으면 같은 bootstrap status로 다시 확인하고 manifest를 편집하거나 경고를 삭제하여 우회하지 않습니다.

TPM을 설정한 뒤 호출을 시작합니다

02의 최소 권장값은 Agent·Judge·Optimizer/planner 배포별 **100,000 TPM**, embedding **10,000 TPM**입니다. 한 환경에서 작업을 순차 실행하는 기준이며 모델·SKU·지역별 할당량은 별개입니다. 여러 전용 환경을 함께 준비한다면 모델별 총 할당량을 합산합니다. 같은 환경에서 Optimizer와 planner가 공유하는 배포는 한 번만 계산합니다.

- 1. 새 전용 환경:** 최신 코드의 bootstrap plan에서 생성 배포 용량 100·embedding 10을 확인하고 그 정확한 계획으로 승인·생성을 진행합니다. 과거 계획은 자동 변경되지 않습니다. 아직 생성하지 않은 낮은 용량의 계획이 있다면 직접 편집하지 말고 새 환경 이름으로 계획·승인을 준비합니다.
- 2. 실제 배포 확인:** Foundry **Build → Models → Deployments → 배포 이름 → Details**에서 **Tokens per Minute Rate Limit**을 읽습니다. 천 토큰 단위의 용량과 최종 TPM 표시를 구분합니다. **이 실습의 bootstrap 환경은 포털의 Edit으로만 용량을 바꾸면 계획과 불일치합니다.** 부족하면 계획·실제 한도·승인 범위를 확인하고, 필요한 경우 충분한 용량으로 새로 승인한 전용 환경을 준비합니다. 해시·승인서 편집으로 우회하지 않습니다.
- 3. 설정 확인:** 해당 환경의 .env와 언어별 artifacts 경로로 다음 읽기 전용 검사를 실행합니다. *_tpm의 observed-expected와 reason을 확인합니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env preflight
```

전체 status: PASS와 다섯 TPM 항목의 PASS 뒤에만 03의 smoke·검색 및 이후 평가·Optimizer로 진행합니다. 검사기는 실제 배포의 rateLimits 중 key: token을 읽습니다. CLI가 이름표를 생략한 경우 같은 배포의 원시 ARM 메타데이터를 읽으며, 요청 수나 sku.capacity를 TPM으로 추정하지 않습니다. 확인 불가도 BLOCKED입니다.

TPM은 청구된 평균 토큰과 다르게 추정되며 **RPM과 버스트 제한**도 적용됩니다. 입력·최대 출력 길이, 공유 사용자, 평가·최적화 동시 실행이 늘면 추가 여유를 산정합니다. 이 최소 권장값은 429 없는 실행을 보장하지 않습니다. [공식 제한 설명](https://learn.microsoft.com/azure/foundry/openai/how-to/quota#understanding-rate-limits) (<https://learn.microsoft.com/azure/foundry/openai/how-to/quota#understanding-rate-limits>).

03의 실제 응답 확인으로 Agent와 knowledge_base_retrieve 호출을 확인합니다. Optimizer는 별도 [지원 모델 목록](https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models) (<https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models>)을 따릅니다. Agent나 Judge로 동작한다고 최적화 생성 모델로 지원된다고 가정하지 않습니다.

Microsoft Foundry / proj-fev11-860db66e

Search with AI (🔍)

New Foundry

Create

Agents

Models

Services

Tools

Knowledge

Memory

Guardrails

Data

Optimize

Evaluations

Fine-tune

Models

Deployments

Models

Preview

Batch jobs

Serverless deployments

Managed compute deployments

Search

PTU Calculator

Deploy

Name	Model	Version	Deployment status	Deployment type	Created on ↓	Created by
lab-sft-860db66e	gpt-4.1-mini-2025...	1	Succeeded	Standard	9/30/26, 11:03:19 ...	
lab-embedding-860db66e	text-embedding-3...	1	Succeeded	Global Standard	9/30/26, 8:30:13 AM	
lab-planner-860db66e	gpt-5.5	2026-04-24	Succeeded	Global Standard	9/30/26, 8:30:12 AM	
lab-judge-860db66e	gpt-5.4-mini	2026-03-17	Succeeded	Global Standard	9/30/26, 8:30:11 AM	
lab-agent-860db66e	gpt-4.1-mini	2025-04-14	Succeeded	Standard	9/30/26, 8:30:09 AM	

1-5 of 5

Prev

Next

모델·버전·상태를 확인합니다. 각 배포를 역할 표의 Agent·Judge·Optimizer·embedding과 대조합니다. 배포 이름은 `.env`와 같아야 하며, 준비된 배포의 실제 호출도 확인합니다. 원본 크기로 보기 (파일: `web/assets/portal/02-model-deployments.png`)

Agent를 실제로 준비합니다

01의 운영체제별 설치 안내에 따라 **Python 3.11–3.14·Git·Azure CLI**를 준비하고 새 터미널에서 세 도구의 버전을 확인합니다. 이미 정상인 도구는 다시 설치하지 않으며 설치 제한·권한·프록시 문제를 먼저 해결합니다. 이어서 확인한 Python으로 가상환경을 만들고 로그인·언어 변수를 설정합니다. 아직 없는 `.venv`를 활성화하거나 `.env.example`의 가짜 식별자를 실환경으로 사용하지 않습니다.

한국어 기준선은 `prompts/baseline.txt`, 영어는 `prompts/en/baseline.txt`입니다. 지침을 약화하여 개선 폭을 만들지 않습니다. 정책 업로드와 검색 확인이 완료된 뒤 다음 명령으로 정책 도구를 연결한 v1을 만듭니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env native-agent --version 1 --confirm
```

`native-agent`는 `ensure_fixed_release`를 사용하여 **1.2만 허용**하고, 소유 `workspace`를 원격 `metadata`와 대조합니다. 같은 구성은 재사용하며 다른 v2를 덮어쓰거나 v3를 만들지 않습니다. 모델 배포 `snapshot`도 보관하고 변경 시 비교를 차단합니다. `agents/native-v1.json`, `agents/native-v2.json`, `agents/native-model.json`은 해당 환경의 artifacts 폴더에 기록됩니다.

`native_response_format()`은 공통 스키마를 엄격한 생성 스키마의 지원 범위로 변환합니다. 인용 중복과 `needs_human·route` 일관성은 생성 후에도 확인합니다. 유효한 JSON만으로 정책 정확성이 보장되지는 않습니다.

v2 생성은 평가 완료가 아닙니다. [08에서 검토한 전체 지침](#)으로 v2를 한 번 만들고 [09의 같은 관리형 평가](#)를 수행한 뒤 채택 여부를 판단합니다. 정식 v1/v2만 사용하며 별도 draft-... 실험은 이 실습에 포함하지 않습니다. 생성한 v2는 운영에 게시하지 않습니다.

터미널에서 실행 · 공통 터미널

```
python -m lab --config .lab/lab-ko/.env native-agent --version 2 --prompt .lab/lab-ko/candidate.txt --confirm
```

이 실습에서는 포털 Promote를 사용하지 않고 09의 CLI로 v2를 생성합니다. 포털에서 먼저 승격하면 CLI의 로컬·원격 소유권 확인과 맞지 않을 수 있습니다. v1·v2는 Agent 전체 구성 버전이며 같은 Agent·모델·도구·출력 설정에서 지침만 달라야 합니다.

평가 ID는 [06의 전체 조회 명령](#)으로 찾습니다. .env 경로와 정확한 평가 이름을 포함한 명령을 사용합니다. 재평가 helper는 기존 data source를 복사하고 **원격** 계약을 검증합니다. 오래된 로컬 JUDGE_DEPLOYMENT로 원격 정의를 바꾸지 않습니다.

비교 조건을 고정합니다

항목	고정 조건
데이터	선택한 언어의 같은 dev12 바이트·등록·버전·질문·참고 답변입니다.
Relevance	관리형 builtin.relevance, 1–5점, 임계값 4
TaskAdherence	관리형 builtin.task_adherence, 이진 0/1, 통과 1
Judge	두 평가기 정의에 같은 명시적 Luna 배포
Agent	같은 Sol 모델·버전, 도구, 추론, 엄격한 출력 스키마
변경 대상	지침만 변경, 정식 버전은 v1/v2
Optimizer	Instruction만 선택, 모델·도구 설명 변경 끄기, 후보 수 제한
실습 기록	같은 조건의 v1·후보와 12건 전체 결과를 비공개 실행 폴더에 보관하며 재사용 가이드와 분리

카탈로그 평가기 버전은 비공개 서비스 루브릭이 완전히 고정됐다는 증거가 아닙니다. 실제 정의·설정과 이 한계를 남깁니다. 재사용 dev12의 개선은 향후 점수·독립적 일반화·운영 승인을 보장하지 않습니다.

필수 비교는 [09의 Foundry Compare runs](#)에서 수행합니다. scripts/compare_foundry_eval.py는 선택 사항인 별도 비교 도구이며 이 경로에서 추가 실행하지 않습니다. 이 도구를 별도로 사용할 때는 LAB_LANGUAGE 또는 --language ko/--language en으로 보고서 언어와 기본 dev12를 선택하고, 직접 지정한 데이터셋과 실행 언어를 일치시킵니다.

평가 응답 매핑을 점검합니다

05에서 두 평가기·Judge를 선택하고 자동 매핑을 유지합니다. 아래 표는 누락·불일치가 있을 때의 점검 기준입니다. 정상적인 생성 흐름에서 SDK 내부 필드를 추측해 직접 입력할 필요는 없습니다.

확인할 설정	유지할 값
Agent 사용자 입력	{{item.query}}만 사용하고 지침 override는 비웁니다.
Relevance 응답	response={{sample.output_text}}
TaskAdherence 응답	response={{sample.output_items}}
Judge-임계값	실제 JUDGE_DEPLOYMENT, Relevance 4·TaskAdherence 이진 통과 1

필수 항목이 **Unassigned**이면 대상이 Dataset이 아닌 **Agent**인지, query 열과 실제 Agent 응답의 연결이 맞는지 대조합니다. 제출 전에는 생성 화면을 확인하고, 제출 후에는 저장된 원격 정의를 확인합니다. 참고 답변을 response에 넣거나 이전 UI의 매핑을 복사하지 않습니다. [공식 포털 평가 안내 \(https://learn.microsoft.com/azure/foundry/how-to/evaluate-generative-ai-app\)](https://learn.microsoft.com/azure/foundry/how-to/evaluate-generative-ai-app)와 [응답 매핑 설명 \(https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators#using-agent-evaluators\)](https://learn.microsoft.com/azure/foundry/concepts/evaluation-evaluators/agent-evaluators#using-agent-evaluators)을 참고합니다. 해결 전에는 제출하지 않습니다. 09의 helper도 원격 계약을 검사합니다.

실제 승인에 맞춰 승인서를 작성합니다

.lab/lab-ko/approval.example.json을 편집기에서 열어 **같은 폴더의 approval.json으로 따로 저장**합니다. 본인에게 지출·변경을 결정할 권한이 있으면 한도를 직접 정하고, 조직 구독은 승인 절차를 따릅니다. 실제 승인 근거를 비공개로 보관합니다. 다음 표나 JSON 파일 자체가 독립적인 승인·서명 증명은 아닙니다. JSON 문자열에는 큰따옴표를 사용하고 숫자·true·false·null에는 따옴표를 붙이지 않습니다.

필드	작성 방법
schema_version, scope_sha256, models, retention_days	생성된 값을 그대로 유지합니다. models 배열이나 계획을 편집하면 기존 승인은 유효하지 않습니다.
approved	실제 승인 후에만 true로 변경합니다.
approved_by	config.json의 expected_user와 같은 본인 로그인 이름입니다. 실행 신원 연결용이며 조직 승인자의 전자서명이 아닙니다.
approved_at, expires_at	실제 승인 시작·만료 시각입니다. 2026-10-03T09:00:00+09:00 형식의 시간대 포함 ISO 8601을 사용합니다. 예시 날짜를 그대로 복사하지 않습니다. 현재 시각이 두 시각 사이여야 합니다.
currency, budget_amount, budget_policy	승인 통화의 세 글자 코드와 양수 예산입니다. 예산 정책은 BOUNDED로 유지하고 실제 허용된 금액을 적습니다.

필드	작성 방법
acknowledge_no_monetary_cap	false로 유지합니다. 이 실습에서 무제한 지출을 기본으로 설정하지 않습니다.
max_hosting_hours	승인된 양의 정수 시간입니다. 예를 들어 8이면 승인 시작부터 만료까지도 8 시간 이하여야 합니다.
max_wait_seconds	양의 정수 대기 한도입니다. 예를 들어 3600입니다. 대기 종료는 원격 작업 취소가 아닙니다.
max_calls, max_candidates	승인된 호출·후보 한도입니다. 08의 요청 후보 수는 2입니다. 내부 Agent·Judge·검색 호출을 고려합니다. 이 필드가 포털의 모든 호출·비용을 자동 제한하지는 않습니다.
allow_global_inference	GlobalStandard의 처리 경계를 승인한 경우 true입니다. NCUS 내부 처리만 보장하지 않습니다.
allow_resource_creation, allow_rbac_assignments	해당 계획의 리소스 생성·리소스 범위 역할 할당을 승인한 경우 각각 true입니다.
acknowledge_continuous_hosting, acknowledge_unknown_cost	지속 과금 가능성과 확정되지 않은 총비용을 확인한 경우 각각 true입니다.
allow_training, allow_global_training	이 경로는 false입니다.
max_epochs, max_training_jobs	이 경로는 0입니다.
accept_deprecated_models	기본 false입니다. 서비스가 지원 종료를 알리면 별도 검토 없이 바꾸지 않습니다.

02의 `bootstrap preflight --config ... --approval ...`로 파일을 검증합니다. 오류가 있으면 `approval_reason`을 확인합니다. 한 파일의 승인은 정확한 계획 해시·모델·보존 범위에만 적용되며 다른 실습·재시도·삭제 승인을 포함하지 않습니다.

강제 범위를 구분합니다. bootstrap은 승인 유효기간·생성 범위·대기 시간을 검사하지만 JSON 예산과 호출 한도가 Azure Portal·Optimizer 전체의 지출을 자동 차단하지 않습니다. 비용 알람도 차단 장치가 아닙니다. 본인이 실제 사용량을 확인하고 승인된 범위에서 작업을 취소하거나 자원을 정리합니다.

권한과 공급자 등록을 확인합니다

현재 Foundry RBAC 안내 (<https://learn.microsoft.com/azure/foundry/concepts/rbac-foundry>)를 기준으로 권한을 확인합니다. Foundry User/Owner/Account Owner/Project Manager는 이전 Azure AI User/Owner/Account Owner/Project Manager 이름으로 보일 수 있습니다.

포털의 Access control (IAM) → Check access에서 활성 할당과 Scope를 확인합니다. 이전 UI의 View my access와 위치가 다를 수 있습니다. 01의 실제 구독·권한 그림은 기존 권한을 읽는 예시이며 역할 추가 안내가 아닙니다.

작업·연결	확인할 권한과 범위
본인의 새 환경 생성	bootstrap은 구독 범위의 그룹·배포·자원 생성과 <code>Microsoft.Authorization/roleAssignments/write</code> 유효 권한을 확인합니다. Contributor만으로는 역할 할당이 안 됩니다. 구독 quota 조회 권한도 필요합니다.
본인의 Agent·평가 실행	자신의 프로젝트에 Foundry User 등 필요한 데이터 평면 권한이 있어야 합니다. Azure의 Contributor/Owner만으로 이 권한이 생기지는 않습니다.
본인의 정책 준비	자신의 Search에 Search Service Contributor와 Search Index Data Contributor 등 스키마·업로드 권한이 필요합니다.
프로젝트 관리 ID	해당 Search의 Search Index Data Reader, 모델 호출과 해당 관측 리소스 전송 권한입니다.
Search 관리 ID	해당 Foundry 리소스의 모델 호출 권한입니다.
본인의 로그 확인	자신의 Application Insights·Log Analytics를 읽는 권한입니다. 구독 전체 로그 접근을 기본으로 요구하지 않습니다.

새 자원의 실행 권한은 승인된 bootstrap이 해당 자원 범위에 할당합니다. 구독 전체 역할을 새로 부여하지 않습니다. 생성 후에는 본인과 서비스의 관리 ID가 서로 다른 신원임을 구분하고 03의 실제 도구 호출로 전파·데이터 평면 접근을 확인합니다.

공급자는 Azure 서비스 종류를 구독에서 사용할 수 있게 하는 등록 항목입니다. [공식 공급자 등록 안내](https://learn.microsoft.com/azure/azure-resource-manager/management/resource-providers-and-types#azure-portal) (<https://learn.microsoft.com/azure/azure-resource-manager/management/resource-providers-and-types#azure-portal>)에 따라 다음을 확인합니다.

1. Azure Portal → **Subscriptions** → **사용할 구독** → **Resource providers**를 엽니다.
2. `Microsoft.CognitiveServices`, `Microsoft.Search`, `Microsoft.OperationalInsights`, `Microsoft.Insights`를 각각 검색하여 **Registered**인지 확인합니다.
3. 미등록 항목은 본인에게 해당 `/register/action` 권한과 등록 승인이 있는 경우에만 선택하여 **Register**를 누릅니다. 필요한 네 항목 외 공급자를 일괄 등록하지 않습니다.
4. 등록 완료·전파 후 같은 preflight를 다시 실행합니다. 등록 권한이 없으면 승인된 실행 범위를 확보한 뒤 재개합니다. bootstrap은 공급자를 자동 등록하거나 권한을 확대하지 않습니다.

등록·역할 전파를 기다린 뒤 같은 preflight를 다시 실행합니다. 403이 계속되면 계정·역할 scope·네트워크를 대조합니다. 편의를 위해 구독 Owner를 새로 부여하거나 네트워크 제한·조직 정책을 해제하거나 토큰을 옮기지 않습니다.

비용과 운영 경계를 정합니다

평가는 Agent·Judge를 호출하고 최적화에는 추가 내부 호출이 있습니다. Search·모니터링은 실습 화면을 달아도 비용이 남을 수 있습니다. 관측 토큰·예상 비용·실제 청구를 구분하고 미확인 비용을 0원으로 표시하지 않습니다.

Azure Portal → **Cost Management** → **Cost analysis**에서 구독·리소스 그룹·기간을 확인합니다. 필요하면 승인된 비용 알림을 설정하지만 이것은 강제 상한이 아닙니다. 실제 허용 예산·종료 시각·삭제/보존 계획을 기록표에 적습니다.

실제 값과 완료 근거를 기록합니다

02에서 생성 계획을 만든 뒤 다음 표를 자신의 `.lab/lab-ko/notes.md`에 복사합니다. 아직 실행하지 않은 항목은 **미실행**으로 표시하고 각 단계가 끝날 때 실제 값으로 채웁니다. “준비됨”이라는 문장만으로 완료를 표시하지 않습니다.

필수 항목	기록할 실제 값·완료 근거
로그인 범위	사용자, tenant ID, subscription ID입니다. 비밀번호·토큰은 포함하지 않습니다.
로컬 경로	본인의 환경 이름, bootstrap config 경로, 런타임 <code>.env</code> 경로, <code>LAB_LANGUAGE</code> , <code>LAB_ARTIFACTS_DIR</code> 입니다. <code>.env</code> 는 생성 완료 후 기록합니다.
Azure 환경	리소스 그룹·Foundry account·project·project endpoint·Search 이름입니다.
모델	네 배포의 실제 이름·제품명·버전·SKU·용량입니다.
정책 검색	정책 원본·언어, 업로드 8건, knowledge base·connection 이름, <code>retrieval_verified</code> 입니다.
Agent	실제 이름, 고정 버전 1, 엄격한 출력 형식, 실제 Agent 도구 호출 확인입니다.
데이터	JSONL 경로, 12행, SHA-256, 등록 이름·버전입니다.
평가	Relevance 4·TaskAdherence 1, Judge 배포, query 전용 입력입니다. 첫 평가 뒤 실제 evaluation ID·v1 run ID도 추가합니다.
승인	계획 해시, 승인 만료·예산·후보 한도·보존 기간입니다.
종료	삭제가 승인된 정확한 그룹/객체와 예정 시각, 또는 보존 이유·비용 책임·검토 일·후속 삭제 계획입니다. 공유·외부 종속성도 구분합니다.

원본 영어·한국어 데이터와 비공개 실습 기록을 분리하며 다른 언어의 결과를 새 실행으로 바꾸어 표시하지 않습니다.

원본 서비스 파일에는 계정 메타데이터·토큰·서명된 URL이 포함될 수 있어 로컬에서 보관합니다. **합성 평가 결과 자체가 비밀인 것은 아닙니다.** 성공·실패를 모두 포함한 허용 필드만 공개하고, 불리한 결과가 아니라 자격 증명을 제외합니다. 과거 시도는 원본 감사 기록으로 보존하되 현재 버전 보고서에 계속 누적하지 않습니다.

삭제와 보존을 확인합니다

[10의 삭제 절차](#)를 종료 체크리스트로 사용합니다. 생성 승인은 삭제 승인이 아니며 실습 자료만으로 타인의 자원을 삭제하지 않습니다.

대상	삭제 또는 보존 확인
실행 중인 평가·Optimizer	취소 가능 여부, 실제 종료 상태, 남은 job ID를 기록합니다.
로컬 소유 객체	cleanup 계획의 정확한 대상과 실행 후 부재 상태를 확인합니다.
포털 생성 객체	데이터셋·평가·대화·Optimizer·남은 빈 Agent 항목을 개별 확인합니다.
전용 리소스 그룹	기본 경로입니다. 전체 인벤토리와 승인을 확인한 뒤 삭제하고 성공한 <code>az group exists</code> 의 <code>false</code> 를 기록합니다.
보존·공유·외부 자원	삭제하지 않은 항목의 이름·이유·비용 책임·보존 검토일과 후속 계획을 기록합니다.
비용·soft delete	청구 지연과 이미 발생한 요금을 확인하고, 서비스별 보존·영구 삭제 정책을 별도로 적용합니다.
증빙	결과와 실패 기록을 필요한 기간 비공개로 보관하며 로컬 <code>.lab</code> 부터 삭제하지 않습니다.

실습 문제 해결

[실습 10단계](#) · [권한·승인·설정 참고](#)

막힌 단계에서 증상·조치·진행 기준을 확인합니다. 오류를 해결하기 위해 데이터·평가기·모델을 조용히 바꾸거나 새 이름으로 같은 작업을 중복 제출하지 않습니다.

실행할 명령은 **터미널에서 실행** 상자에 있습니다. **코드 ↔ 포털** 표의 **실행 위치** 링크로 명령·포털 절차를 찾습니다. 접힌 구현 패널은 읽기용 원문이며 직접 실행하거나 오류를 숨기려고 수정하지 않습니다.

현재 증상	확인할 곳
HTML에서 설명만 보이고 실행 명령을 찾을 수 없습니다.	표의 실행 위치 링크 또는 단계 상단의 실행 순서 를 사용합니다. 로그인 명령 · 런타임 검사 .
용어·설치·로그인·파일 경로가 어렵습니다.	기본 용어 · 환경·로그인
승인·권한·모델 용량 때문에 생성할 수 없습니다.	생성 준비
정책 검색이나 Agent 응답이 실패합니다.	정책·Agent
평가가 끝나지 않거나 점수·이유가 보이지 않습니다.	평가·결과 조회
Optimize가 없거나 후보를 선택하기 어렵습니다.	Optimizer
삭제가 안 되거나 자원을 보존해야 합니다.	정리·보존

사용 방법

오류 시각·단계·명령·대상 ID·오류 코드와 조치를 자신의 비공개 실습 기록에 남깁니다. 원본 응답·receipt를 보관하고 실패를 성공으로 고쳐 쓰지 않습니다. 실행별 검증 로그와 결과는 재사용 가이드에 추가하지 않습니다.

환경·로그인·로컬 파일

Python·Git·Azure CLI 설치와 PATH 문제는 Azure 로그인 전에 해결합니다.

증상	조치	진행 기준
py.python3.13-git-az가 command not found 또는 not recognized입니다.	01의 운영체제별 설치 를 마치고 터미널을 모두 닫았다가 다시 엽니다. VS Code도 재시작합니다. 계속 실패하면 설치 경로의 PATH 등록을 담당자와 확인하며 Windows는 Get-Command py, git, az, macOS/Linux는 command -v git az로 경로를 확인합니다.	세 도구의 버전 확인 이 같은 새 터미널에서 성공합니다.
winget 또는 brew가 없습니다.	Windows는 공식 설치 파일 경로 를 사용합니다. macOS는 Homebrew 설치와 Next steps 의 PATH 설정을 완료합니다.	설치 방법 하나로 필요한 도구를 준비하고 버전을 확인합니다. 조직의 설치 제한을 우회하지 않습니다.
Python 대신 Microsoft Store가 열리거나 py가 없습니다.	Windows의 일반 Python 설치에서 Launcher·pip·PATH 옵션을 확인하고 새 PowerShell에서 py -3.13 --version을 실행합니다. 다른 지원 버전을 설치했다면 해당 번호를 사용합니다.	Store 바로가기가 아닌 설치한 Python 3.11~3.14가 실행됩니다.
Python 버전이 3.11~3.14 밖이거나 다른 Python으로 .venv를 만들었습니다.	시스템 Python을 교체하지 말고 지원 버전을 별도로 선택합니다. py -3.13 또는 python3.13 등 버전을 확인한 같은 명령 으로 실행용 환경을 준비합니다. 기존 .venv는 소유 작업과 보관 필요성을 먼저 확인합니다.	가상환경의 python --version이 지원 범위이고 python -m pip --version의 경로가 그 .venv 안입니다.
Linux에서 No module named venv 또는 ensurepip is not available입니다.	Ubuntu 설치 순서 의 python3-venv를 설치합니다. 버전 지정 Python이면 해당 버전에 맞는 venv 패키지를 담당자와 확인합니다.	선택한 Python으로 .venv 생성과 pip 확인이 성공합니다.
다운로드·설치가 설치 권한·프록시·인증서 오류로 막힙니다.	공식 다운로드 주소·오류와 조직의 허용 설치 경로를 확인합니다. 필요한 설치·네트워크 승인을 확보하고 TLS 검증·보안 도구·조직 정책을 해제하지 않습니다.	승인된 경로로 설치하고 세 도구의 버전 확인이 성공합니다.
.venv/bin/activate 또는 Activate.ps1이 없습니다.	저장소 폴더에서 가상환경을 먼저 만들고 자신의 OS에 맞는 설치 순서 를 따릅니다.	같은 Python의 python -m pip --version과 python -m lab --help가 성공합니다.
ModuleNotFoundError가 나옵니다.	사용할 가상환경을 확인하고 그 Python으로 python -m pip install -r requirements.lock을 실행합니다.	같은 환경에서 필요한 import와 데이터 검사가 성공합니다.
PowerShell 활성화가 정책으로 차단됩니다.	조직 정책을 바꾸지 않고 <code>.\.venv\Scripts\python.exe</code> 로 명령을 실행합니다.	정책 변경 없이 Python을 실행할 수 있습니다.
포털과 CLI의 계정이 다릅니다.	브라우저와 CLI는 별도 로그인입니다. 01의 로그인·조회 명령 으로 사용자·tenant·subscription을 포털과 대조하고 본인 인증은 직접 완료합니다.	승인한 세 값이 모두 일치합니다. 다른 신원이나 토큰 복사로 우회하지 않습니다.
AzureCliCredential의 tenant-subscription 동시 지정이 실패합니다.	tenant 일치를 먼저 확인한 뒤 credential에는 subscription을 지정하는 저장소 인증 경로를 사용합니다.	tenant 확인을 생략하지 않고 같은 구독으로 호출합니다.

증상	조치	진행 기준
다른 언어의 artifacts에 기록됩니다.	Python 시작 전에 01의 언어·기록 폴더 명령 으로 LAB_LANGUAGE와 LAB_ARTIFACTS_DIR를 지정합니다.	데이터·설정·workspace·결과의 언어가 일치합니다. 기존 기록을 덮어쓰지 않습니다.

생성·승인·할당량

증상	조치	진행 기준
plan이 BLOCKED_AWAITING_APPROVAL입니다.	plan_status와 mutations_performed를 확인하고 실제 승인서 를 준비합니다.	로컬 계획 생성과 Azure 생성 완료를 구분합니다.
approved: true인데 승인 오류입니다.	해시·모델·승인자·유효기간·예산·동의 항목과 approval_reason을 확인합니다.	readiness가 READY이고 승인이 READY_FOR_APPROVED_APPLY입니다.
Provider가 Registered가 아닙니다.	공급자 등록 절차 에서 본인의 등록 권한·승인을 확인하고 필요한 공급자만 등록합니다.	네 공급자가 Registered가 된 뒤 같은 preflight를 실행합니다.
Contributor인데 역할 할당이 막힙니다.	자원 생성과 역할 할당은 별도 권한입니다. 본인이 이 실습을 수행할 수 있도록 필요한 실행 범위의 승인을 확보합니다.	필요한 유효 권한을 확인한 뒤 같은 계획으로 재개합니다. 구독 전체 Owner를 새로 부여해 우회하지 않습니다.
모델·SKU·버전·quota·capacity 오류입니다.	계획의 역할별 요청과 preflight 이유를 확인합니다. 대체가 필요하면 별도 계획·승인을 받습니다.	지원 모델과 할당량을 확인하고 실제 호출까지 확인합니다.
런타임 *_tpm 항목이 BLOCKED입니다.	모델별 TPM 설정 에서 실제 배포 한도를 확인합니다.	다섯 TPM 항목과 전체 preflight가 PASS입니다.
생성 후 unknown resource가 나옵니다.	자신의 Application Insights에 연결된 기본 Smart Detection인지, 다른 업무 자원이나 변경된 수신자인지 확인합니다. 생성 중이면 연결 전파를 기다립니다.	같은 bootstrap status로 확인합니다. 경고 삭제나 manifest 편집으로 우회하지 않습니다.
apply가 시간 초과 또는 결과 불명입니다.	원래 config·manifest·deployment ID를 보고 02의 bootstrap status 명령 과 그룹의 Deployments를 읽습니다. apply는 다시 실행하지 않습니다.	진행 중이면 기다리며 결과 불명 상태에서 다시 제출하지 않습니다.
같은 로컬 환경이 이미 있습니다.	같은 실습은 원본 config로 재개합니다. 별도 실습만 새 환경 이름과 승인을 사용합니다.	기존 계획·설정·자원을 임의로 인수하거나 덮어쓰지 않습니다.

--retry는 확인된 소유 배포의 종료된 실패와 별도 재시도 승인 한도에만 사용합니다. repair-dependencies, repair-trace-routing도 일치하는 실패 근거와 해당 범위의 승인이 필요한 복구 명령입니다. APPLIED는 인프라 상태이며 품질·운영 승인·로그 수집 성공을 뜻하지 않습니다.

정책 검색·Agent 준비

증상	조치와 진행 기준
배포는 있지만 모델 호출이 실패합니다.	카탈로그·배포와 런타임 지원은 다릅니다. API 오류와 역할별 지원을 확인하고 03의 실제 응답 을 확인합니다.
created_not_retrieval_tested에서 멈춥니다.	생성만 완료된 상태입니다. 03의 iq_probe 명령 에서 retrieval_verified와 실제 출력을 확인합니다.
직접 검색은 되지만 Agent 도구가 403입니다.	프로젝트 관리 ID의 Search 읽기·모델 호출 권한과 연결 audience를 확인합니다. 사용자 검색 성공만으로 판단하지 않습니다.
native-agent 실행 후 구성 불일치 오류입니다.	receipt와 원격 버전을 먼저 조회합니다. MCP 도구 목록의 동등한 표현은 정규화하지만, 다른 도구·권한·모델·출력 설정은 허용하지 않습니다. receipt를 지우거나 새 버전을 만들어 우회하지 않습니다.
기존 v2와 지침 또는 모델이 다릅니다.	고정 비교 보호 동작입니다. v2를 덮어쓰거나 v3를 만들지 않고 원래 소유 환경·지침·모델 snapshot을 대조합니다.
언어별 검색 결과가 다릅니다.	선택 언어의 원본 문서·인덱스·analyzer·해시를 확인합니다. 이름만 바꾸어 다른 언어의 인덱스를 재사용하지 않습니다.
사설망 또는 PublicNetworkAccess=Disabled 오류입니다.	승인된 VNet/VPN/실행 환경과 DNS를 확인합니다. 방화벽·private endpoint를 해제하지 않습니다.

Foundry 평가·재개

Add run의 item-schema 오류

다음 메시지가 나오면 기존 평가의 Agent 대상 data source 구성을 확인합니다.

오류 메시지 예시 · 실행하지 않습니다.

```
Unable to create data source configuration from item schema
```

[09의 scripts/add_foundry_eval_run.py](#)는 완료된 기준선의 원격 data source를 복사하고 명시적 후보 버전만 바꿉니다. 같은 evaluation ID·데이터 등록·Judge·임계값·응답 매핑과 전체 12건의 실제 버전·지침을 확인합니다. 새 데이터나 로컬 Judge로 대신하지 않습니다.

ID와 중복 제출

증상	조치
evaluation ID·run ID를 모릅니다.	06의 전체 native-evals 명령 으로 조회합니다. evaluation은 eval_..., 기준선은 완료된 버전 1의 evalrun_...입니다.
같은 이름의 평가가 여러 개입니다.	생성 시각·Agent·데이터·run을 대조합니다. 이름만으로 임의 선택하지 않습니다.
helper가 Still running으로 끝났습니다.	receipt에 run ID가 있으면 같은 명령·같은 --out으로 수집을 재개합니다.
receipt에 run ID가 없거나 같은 이름의 원격 run이 있습니다.	원본을 보관하고 제출 수락 여부와 실제 run을 확인합니다. receipt 삭제·이름 변경·자동 재제출을 하지 않습니다.
버전 선택 뒤 체크가 사라집니다.	명시적 v1을 선택한 뒤 체크박스를 다시 선택하고 대상 하나를 확인합니다.
데이터 미리 보기가 5행입니다.	원본 12행·등록 버전·실제 결과 전체 건수를 대조합니다. 미리 보기만으로 판단하지 않습니다.
점수나 이유가 보이지 않습니다.	실제 답변은 conversation_id → User view, 이유는 Detailed metrics result의 Relevance.reason·TaskAdherence.reason에서 읽습니다.
필수 response 칸이 Unassigned입니다.	매핑 점검 에서 Agent 대상·query 열·서비스 생성 응답 매핑을 대조합니다. JSONL에 정답을 response 열로 추가하지 않으며 해결 전에는 제출하지 않습니다.
결과 행·점수가 빠졌습니다.	실패·누락을 보관하고 분모 12를 줄이지 않습니다. 점수를 0이나 통과로 만들어 넣지 않습니다.

Agent Optimizer·결과 해석

증상	조치와 진행 기준
No custom evaluators available입니다.	Custom only OFF 또는 View built-in evaluators 를 사용합니다. 필터 때문에 새 custom evaluator를 만들지 않습니다.
Optimize나 지정 모델이 없습니다.	New Foundry, prompt Agent, 프로젝트 권한, Preview와 역할별 모델 지원 (https://learn.microsoft.com/azure/foundry/agents/concepts/agent-optimizer-overview#models)을 확인합니다.
승인한 대기 시간이 지났는데 작업이 실행 중입니다.	Optimization runs에서 같은 job ID의 상태·오류·사용량을 확인하고 이후 조회 계획을 기록합니다. 기다림 종료는 취소가 아닙니다. 새 작업으로 다시 제출하지 않습니다.
작업이 Failed/Canceled이거나 결과가 일부만 있습니다.	실제 상태·job ID·오류를 보관합니다. 완료한 후보가 있는 것처럼 기록하거나 같은 요청을 이름만 바꾸어 반복하지 않습니다.

증상	조치와 진행 기준
View changes에 일부 diff만 있고 전체 지침을 복사할 수 없습니다.	접힌 변경 후 구간·후보 상세와 제공되는 Download/Export를 확인합니다. 전체 지침을 확보하지 못하면 v2를 만들지 않고 “v1 유지·전체 지침 확인 불가”를 기록한 뒤 10으로 진행합니다.
후보가 모델·도구 설명도 바뀝니다.	Instruction only 와 모델 비교 끄기를 확인합니다. 지침 외 설정 변경을 같은 조건의 비교로 표시하지 않습니다.
후보에 정책 날짜·수치·평가 정답 예시가 들어갑니다.	원본 서비스 출력을 보관하고 내장된 정답을 제거하거나 후보를 거부합니다. 직접 수정한 부분·이유·출처를 기록하고 별도로 재평가합니다.
다른 후보 없이 기준선만 선택됐습니다.	v1 유지 이유를 기록하고 10으로 이동합니다. 없는 v2를 만들거나 개선을 위해 같은 작업을 반복하지 않습니다.
내부 순위와 재평가 결과가 다릅니다.	내부 0~1 순위와 별도 평가 점수·통과율은 다릅니다. 실제 응답·정책 오류·회귀도 확인합니다.
TaskAdherence가 1입니다.	이진 척도의 통과입니다. Relevance의 1~5점 척도와 혼동하지 않습니다.
통계가 Inconclusive입니다.	차이가 입증되지 않았다는 뜻입니다. 유의성이나 동등성의 증거로 바꾸어 해석하지 않습니다.
품질과 함께 지연·토컨도 증가합니다.	상충 관계를 함께 기록합니다. 미확인 청구액을 0원으로 표시하지 않습니다.

정리·보존·지속 비용

증상	조치와 진행 기준
리소스를 보존하도록 승인받았습니다.	삭제 명령을 실행하지 않습니다. 실제 목록·보존 이유·비용 책임·보존 검토일·후속 삭제 계획을 기록하고 재접속에 필요한 소유권·설정을 유지합니다.
cleanup 뒤에도 Search 비용이 남습니다.	cleanup 은 기록된 객체만 정리하며 서비스·모델·그룹·로그는 남깁니다. 10의 승인된 정리 또는 보존 을 확인합니다.
<code>.env</code> 나 workspace가 없습니다.	<code>config-manifest</code> 와 실제 Azure 목록으로 대상을 확인합니다. 소유권 기록을 임의로 만들어 삭제하지 않습니다.
그룹 안의 모니터링이 다른 그룹과 연결됩니다.	공유 Action group 등 종속성을 담당자와 확인합니다. 전용 그룹이라는 이유만으로 공유 자원을 삭제하지 않습니다.
삭제가 Locks·권한·종속성 때문에 실패합니다.	Locks·Activity log와 실패 작업을 읽습니다. 본인에게 허용된 조치 범위를 확보한 뒤 재개하며 조직 잠금을 임의로 해제하지 않습니다. 삭제가 막힌 상태를 완료로 표시하지 않습니다.
삭제 요청 후에도 그룹이 남습니다.	비동기 진행을 확인하고 10의 부재 확인 명령 을 실행합니다. 성공한 <code>az group exists</code> 가 <code>false</code> 인 경우에만 부재로 기록합니다.
삭제 뒤에도 비용이 표시됩니다.	사용 기간과 청구 반영 지연, 외부 그룹의 잔여 자원을 확인합니다. 과거 사용 요금은 사라지지 않습니다.

서비스별 soft-delete 보존 항목과 영구 삭제/purge는 별도 정책·승인 대상입니다. [리소스 그룹 삭제 문서](https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group)
(<https://learn.microsoft.com/azure/azure-resource-manager/management/delete-resource-group>)를 참고합니다.

합성 한국어 평가 데이터

[데이터셋 단계](#) · [실습 기록표](#) · [English data guide](#) (파일: docs/data-guide.html)

비교에는 변경 없는 데이터셋 하나

한국어 절차는 **data/optimizer/dev.jsonl** (파일: data/optimizer/dev.jsonl)의 **JSONL 12행**을 Foundry Evaluation-Agent Optimizer-재평가에 사용합니다. 본인이 만든 프로젝트에서 [04의 등록 절차](#)로 lab-ko-dev12 버전 1을 등록합니다. 자신의 같은 실습을 재개할 때는 기록한 이름·버전·해시를 대조하여 등록을 재사용합니다.

실행 위치: [터미널에서 12행·해시 확인](#) → [Foundry에서 업로드·선택](#). 로컬 확인 명령은 파일을 업로드하거나 평가를 제출하지 않습니다.

비교하는 동안 파일의 바이트·SHA-256·질문·참고 답변·언어를 유지합니다. 한국어와 영어 데이터는 분리하며 한 비교 안에서 섞지 않습니다.

[04의 코드 ↔ 포털 표](#)에서 로컬 행 수·해시 확인과 실제 포털 등록을 구분합니다. [05의 원격 설정 검증 코드](#), [09의 실제 SDK 제출 코드](#)는 같은 데이터·매핑·버전이 유지되는 과정을 보여줍니다. 읽기용 소스를 별도로 실행하지 않습니다.

목적과 한계

Contoso Atlas Cloud의 정책·질문·참고 답변은 합성 자료입니다. 실제 고객 데이터나 Microsoft·Azure·실제 공급자의 약관이 아닙니다.

공개 벤치마크만 보는 대신 **자사 업무와 기준으로 평가**하는 방법을 익힙니다. 정책 낱파 경계, 모호성, 근거 없는 확신과 실행 완료 주장을 대표 사례로 확인합니다. dev 12건은 학습 루프의 예시이지 운영 안전·독립적 일반화의 인증이 아닙니다.

원본 데이터 보존

data/cases.jsonl (파일: data/cases.jsonl)은 100건이며 train 56, validation 12, dev 12, test 20으로 나뉩니다. 이번 워크숍은 dev12만 사용하며 나머지는 추가 필수 실습이 아닙니다.

결과를 보고 쉬운 행만 선택하거나 중복·병합하거나 참고 답변을 바꾸지 않습니다. 정책 문서 8개 (파일: data/knowledge/documents.json)의 ATLAS-* ID와 발효일을 유지합니다. 정책 연결도 본인이 [03의 준비 명령](#)으로 생성합니다.

업로드·응답 스키마

JSONL 한 행에는 정확히 세 열이 있습니다.

열	형식	용도
query	문자열	Agent에 보내는 유일한 입력
context	문자열	지원되는 평가기와 사례 검토용 정책 참고 자료
ground_truth	JSON 문자열	구조화 참고 답변이며 생성 프롬프트가 아닙니다.

미리 채운 response는 없습니다. Foundry가 고정 Agent를 호출해 실제 응답을 얻습니다. JSON 배열로 바꾸거나 열 이름을 바꾸거나 참고 자료를 질문에 붙이거나 응답을 미리 만들지 않습니다.

참고 답변과 Agent 응답은 네 키를 사용합니다.

키	계약
answer	비어 있지 않은 한국어 문자열입니다.
citations	근거 정책 문서의 중복 없는 고정 ID
route	answer, clarify, escalate, refuse
needs_human	불리언, route가 escalate일 때만 true

응답 스키마 (파일: schemas/response.schema.json)는 분류 일관성도 검사합니다. 네이티브 구조화 출력은 JSON 형식을 제한하지만 사실의 정확성을 보장하지는 않습니다. 숫자 검색 참조 ID는 정책 인용 ID가 아닙니다.

Agent는 티켓 제출·구독 변경·크레딧 승인·데이터 삭제를 실제 수행하지 않습니다. escalate 분류가 담당자 연락 완료를 뜻하지 않습니다. 확답을 요구해도 근거 없는 사실을 만들 수 없습니다.

한 번 등록하고 같은 버전 선택

Foundry → Build → Evaluations → Create → Agent에서 명시적 기준선 버전과 **Individual turns / One time**을 선택합니다. 버전 선택으로 체크가 해제되면 대상을 다시 선택합니다.

처음에는 **Upload new dataset → Browse**로 정확한 한국어 파일을 올려 lab-ko-dev12 버전 1을 등록합니다. 재개·Optimizer·재평가는 **Existing dataset**에서 같은 이름·버전을 선택합니다. 미리 보기가 5행이어도 실제 파일·평가 범위는 12행입니다.

custom prompt override는 비워 둡니다. 필드 매핑 화면이 나타나면 `query → query`를 사용합니다. 서비스 매핑은 **Relevance** `response={{sample.output_text}}`, **TaskAdherence** `response={{sample.output_items}}`이며 추가 JSONL 열이 아닙니다.

Relevance 1–5점·임계값 4, TaskAdherence 이진 0/1·통과 1과 같은 Luna Judge를 유지합니다. 두 평가기가 모든 참고 열을 쓰거나 모든 업무 규칙을 인증한다고 주장하지 않습니다.

전체 근거 비교

같은 모델의 v1과 v2에 대해 12건 전체의 질문·답변·평가 점수·이유를 나란히 비교합니다. 자신의 실제 run ID와 데이터 해시를 기록하여 비교 대상을 구분합니다.

helper는 질문·참고 자료와 실제 행별 Agent 버전·지시를 대조합니다. 오류·누락을 성공으로 바꾸거나 분모에서 빠지 않습니다. JSON·분류 검사는 부가 검증이며 **관리형 평가를 대신하는 로컬 Judge가 아닙니다**.

Agent Optimizer 내부 순위와 별도 관리형 실행의 평균·통과율을 구분합니다. 후보 지침 전체를 검토하고 모델·도구·추론·스키마를 유지합니다. 검토한 v2를 한 번 비교하며 정식 버전을 계속 올리지 않습니다. 유지할 후보가 없으면 v1 유지 이유를 기록하고 10으로 진행합니다.

결과를 공유하기 전에 인증 정보·서명된 URL·비공개 계정 정보를 제거합니다. 오류·누락을 포함해 전체 사례를 기록합니다.

등록된 데이터셋은 cleanup만으로 모두 삭제되지 않습니다. 실습 후 [10의 보관·삭제 절차](#)에서 결과를 보관하고 자신의 전용 그룹을 정리합니다. 보존이 승인된 경우에만 실제 남은 등록·평가 기록과 비용·검토일을 기록합니다. 공유·외부 자원은 삭제하지 않습니다.